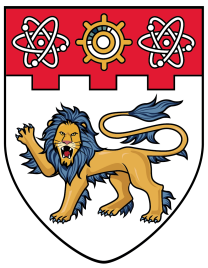

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

Infrastructure Provisioning for SpeechLab Applications with Terraform

Chan Ming Han

U2120477J

Supervisor: Prof Chng Eng Siong

A Final Year Report submitted to School of Computer Science and Engineering, Nanyang Technological University in partial fulfilment of the requirements for the Degree of

BACHELOR OF ENGINEERING SCIENCE (COMPUTER ENGINEERING)

2024/2025 Semester 2

Abstract

As companies move toward containerisation technologies to improve scalability and portability in their applications, Kubernetes has emerged as the industry standard for orchestrating containerised workloads. Despite its advantages, deploying containerised workloads to the cloud is highly complex, and resource intensive. Developers and engineer also need to master the various cloud technologies, such as security and networking requirements, often diverting focus from core application development. This project introduces cloudkube, a lightweight, Python-based command-line tool designed to automate the deployment of Kubernetes clusters in the cloud. Inspired by minikube and leveraging Infrastructure as Code (IaC) principles with Terraform, cloudkube abstracts the complexities of cloud provisioning and Kubernetes setup. It enables developers to focus on core application development, rather than cloud or deployment configurations.. The tool supports customizable configurations, automated local environment setup, and idempotent resource provisioning while ensuring low-cost prototyping by leveraging AWS Free Tier resources. Through cloudkube, deployment times are significantly reduced, and infrastructure provisioning is made accessible to users without deep expertise in cloud technologies. This paper details the tool's architecture, implementation, and workflow, showcasing its potential to enhance productivity, reduce operational overhead, and accelerate the deployment of containerised applications to the cloud.

Keywords: Containerization, Cloud Automation, Infrastructure as Code (IaC), Terraform, Cloud Deployment, Kubernetes Orchestration, Developer Productivity, Lightweight CLI Tools, Cloud-Native Applications

Acknowledgement

I would like to express my deepest gratitude to my academic advisors and mentors, including but not limited to Prof Chng Eng Siong, Research Assistant Vu Thy Ly, and James. Their guidance and invaluable insights have been instrumental throughout the course of this project. Their continuous support and constructive feedback have greatly enhanced the quality of my work. I am particularly grateful for their encouragement and inspiration, which motivated me to tackle complex challenges and explore innovative solutions.

I extend my sincere thanks to my peers, who contributed through stimulating discussions and innovative ideas. Their willingness to share knowledge and exchange ideas fostered an enriching environment that enabled the successful completion of this project.

I extend my special thanks to my course mates, who helped me test this application under various Kubernetes workloads. I also express my gratitude to previous students who have worked on this FYP, as their findings and insights have proved invaluable.

Finally, I would like to acknowledge the funding and infrastructure support provided by Nanyang Technological University. The access to the AWS cloud platform significantly facilitated the implementation and testing phases of this project. Without their generous support, this work would not have been possible.

Table of Contents

Lists of Figures

List of Tables

Chapter 1: Introduction

1.1 Background

In recent years, Kubernetes has emerged as the de facto standard for container orchestration, revolutionizing the way modern applications are deployed and managed [23]. Organizations are incresingly adopting containerized workloads to leverage their multiple benefits, including scalability, portability, and efficiency in managing microservices architectures. Kubernetes provides a robust framework for orchestrating these workloads, ensuring automated deployment, scaling, and operation of such workloads in a highly resilient manner.

Despite the myriad of benefits that Kubernetes offers, the setup, and management of a Kubernetes cluster can often be complex and extremely time-consuming. The deployment process involves provisioning infrastructure, configuring networking, setting up security rules, and configuring various add-ons, all of which require expertise and manual effort. For organizations and developers looking to ship quickly, this complexity introduces delays in application deployment, and reduces team velocity.

The traditional approach to deploying a containerised application and provisioning Kubernetes clusters in the cloud often involves multiple steps, including managing cloud provider integrations, and a lot of configuration. This process can take hours, or even days, depending on the use case, and the specific configuration required. As businesses demand quicker iteration cycles and seamless cloud integration, there is a growing need for simplified, automated provisioning solutions that reduce setup time and eliminate operational friction.

As such, for this final year project, I have decided to build `cloudkube`, which steamlines the provisioning of Kubernetes clusters in the cloud, enabling users to spin up fully configured clusters in the cloud within minutes. By abstracting away the complexities of infrastructure management, `cloudkube` allows developers to focus on deploying applications rather than managing the underlying infrastructure. This report explores the design, implementation, and impact of `cloudkube` in accelerating Kubernetes adoption while maintaining the flexibility and robustness required for production grade deployments.

1.2 Objective

The main objective of this project was to speed up the turnaround time for deployment of new SpeechLab applications. The main idea of this tool is to abstract away in-depth knowledge of cloud technologies from researchers, and allow them to readily and easily deploy their ASR applications to the cloud at the press of a button. This will allow researchers to then concentrate their efforts on the development of the ML model in the ASR application, and not have to worry

about low-level cloud implementation details such as creating a security group role to allow ingress/egress traffic, or configuring their local environment to hit the cluster.

Drawing inspiration from minikube, which is a quick way to spin up a local cluster, for this FYP, I have decided to build cloudkube, which quickly spins up a Kubernetes cluster in the cloud. Users then have a simple barebones Kubernetes cluster hosted in the cloud where they can do dev testing.

The functional requirements for this project are detailed as follows:

a. `cloudkube` should provision infrastructure such as compute instances, storage, networking, and Kubernetes clusters in the cloud.
b. `cloudkube` should be easily extendable to different cloud providers.
c. `cloudkube` should create a Kubernetes environment, including deploying essential components like ingress controllers.
d. `cloudkube` should allow users to easily deploy containerized ASR applications to the Kubernetes cluster.
e. `cloudkube` should be able to deprovision and clean up resources without leaving anything dangling to prevent unnecessary costs.
f. `cloudkube` should maintain the state of provisioned infrastructure to avoid dangling resources in the cloud.

The non-functional requirements for this project are detailed as follows:

a. `cloudkube` should be easy to use, and have a gentle learning curve.
b. `cloudkube` should be user-friendly, with clear command structures and help options.
c. `cloudkube`, the CLI tool, should provision resources and deploy applications within a reasonable time frame.
d. `cloudkube` should provision and deprovision resources idempotently and atomically.
e. `cloudkube` should be able to work across different environments, including Windows, MacOS, and Linux.
f. `cloudkube` should support the addition of new features or integrations.

1.3 Project Scope

The scope of this project is focused on building a minimum viable project for cloudkube, to provision infrastructure for hosting applications in a Kubernetes environment in the cloud. cloudkube aims to simplify the development process by automatic infrastructure provisioning, Kubernetes cluster setup, cloud configuration, as well as application deployment. While the potential of cloudkube is massive, with possibilities for features such as multi-cloud support, advanced monitoring integrations, extensibility of modules and an infrastructure templates, the scope of this project has been deliberately constrained due to the time frame of this FYP.

Given the limited time frame, this project focuses on delivering core functionalities, which are as follows:

a. Efficiently spin up a Kubernetes cluster that applications can be easily deployed to.
b. Allow some customizability of the infrastructure:
    i. Architecture of compute instances.
    ii. Use of ingress controller.
    iii. Use of elastic file system for persistent volumes functionality in Kubernetes.
c. State tracking to ensure idempotent and consistent behavior.
d. Configuration of local environment to communicate with the Kubernetes cluster.
e. Clean deprovisioning and tear down of resources.

1.4 Contributions

This section highlights the key contributions made during the course of this project. The key contributions to this project can be roughly broken down into two parts. The first and main part relates to the infrastructure provisioning, which allows for any containerised workload to be deployed to a Kubernetes cluster hosted on the AWS cloud. The second part relates to optimizing the deployment of the ASR application. While the initial scope of this FYP was to provision infrastructure for SpeechLab applications, this FYP has gone beyond this goal, and `cloudkube` has proved to be scalable, and extendable to different types of Kubernetes workloads.

Contributions toward Infrastructure Provisioning Workflow

a. **Optimized the workflow for creating Kubernetes clusters in the cloud:** In order to facilitate creating Kubernetes clusters in the cloud, I developed the `cloudkube` CLI tool to automate the provisioning of Kubernetes clusters, as well as their corresponding dependencies in the cloud..
b. **Creation of a user-friendly interface:** `cloudkube` was created with clear command structures and help options to ensure ease of use. As a tool aimed at developers who are not experts in cloud, `cloudkube` was built to abstract away in-depth cloud intricacies.
c. **Idempotency and Atomic Provisioning:** To align with the requirements stated above, `cloudkube` was also developed with state tracking, to ensure idempotent and atomic provisioning and deprovisioning of resources to prevent resource leaks and unnecessary costs.
d. **Easy user configurability:** After speaking to developers working on SpeechLab applications, and noting the use cases of different Kubernetes workloads, `cloudkube` provides customization options for infrastructure components such as compute instances, ingress controllers, and persistent volumes.
e. **Optimization of development workflow for authenticating local machine with Kubernetes cluster:** `cloudkube` also facilitates the configuration of local environments to communicate and authenticate seamlessly with the Kubernetes cluster.
f. **Extensive testing:** Extensive testing was also conducted to ensure the reliability and performance of the `cloudkube` tool. This included running multiple Kubernetes workloads against the provisioned cluster, which will be explained in detail in Chapter 5.

Contributions toward Optimizing ASR Kubernetes Deployment

a. **Optimized workflow for copying of models into compute worker instances:** The previous workflow for copying of the models into the EC2 worker instances was via secure copy `scp`. In this project, a more streamlined process was developed, and models were automatically pulled from cloud storage into compute instances [47].

1.5 Report Organisation

This report is structured into 5 different chapters, each detailing different aspects of the project.

Chapter 1 - Introduction: This chapter presents the background and introduction of the project. In this chapter, I elucidate some of the motivations for this project, and a brief overview on the intended outcome of this project.

Chapter 2 - Literature Review: This chapter reviews relevant technologies, and builds a foundational understanding for technical terminology used in future chapters. For completeness, this chapter will also discuss some of the pros and cons of each technology, which become relevant in later chapters.

Chapter 3 - Planning and Design: This chapter focuses on the analysis and design of the proposed solutions. In this chapter, I explore the current methods for handling containerised workloads, and show an optimized workflow for my proposed application.

Chapter 4 - Implementation: This chapter delves into the implementation of the proposed solutions. In this chapter, I detail the code and architecture used for `cloudkube`.

Chapter 5 - Results and Analysis: This chapter discusses the results of my proposed application, `cloudkube`. In this chapter, we will see the deployment of two applications to a Kubernetes environment provisioned by `cloudkube` - 1 ASR Application, and 2 Multi-service Kubernetes Application.

Chapter 6 - Conclusion and Future Work: This chapter presents the conclusions and recommendations for future work.

Chapter 2: Literature Review

To determine the most optimal design and implementation tool, I conducted a comprehensive evaluation of the various cloud technologies. This involved exploring the capabilities of each cloud technology, and diving into the supporting documentation. By understanding the objectives, strengths, and weaknesses of each of the different technologies, I was able to narrow down on the problem that I was trying to solve, and select the best tools that will align with the requirements of the project.

2.1 Infrastructure as Code (IaC)

According to AWS, "Infrastructure as Code (IaC) is the ability to provision and support your computing infrastructure using code instead of manual processes and settings.". In the typical software development life cycle, when it gets to deploying applications, developers have to regularly set up, update, and maintain infrastructure in order to develop, test, and deploy applications [8].

With more and more applications coming up, and companies focusing on high velocity, it has become increasingly important for these manual tasks like infrastructure provisioning to be automated. Furthermore, manual infrastructure provisioning is also more error prone, especially when managing applications at scale (AWS, n.d.). As such, more and more IaC tools have come up, so that infrastructure can be provisioned reliably, quickly, and unambiguously to support our applications in the cloud. Next, we will discuss some of the common IaC tools, their uses cases, strengths as well as pitfalls.

2.1.1 Terraform

Perhaps one of the most popular IaC tools is Terraform. Terraform is an open-source IaC tool created by HashiCorp, which allows users to define and provision infrastructure declaratively. With Terraform, one can specify infrastructure resource in human-readable configuration files that can be reused, distributed, and modified [11].

Benefits

1. **Flexibility and Scalability:** Terraform integrates with over 1700 service providers governing different types of resources and services [11], which means that developers can automate the deployment and management of a wide variety of resources ranging from compute instances to storage solutions.

2. **Cloud Agnostic:** Terraform is cloud agnostic, meaning that a single configuration can manage multiple providers and handle cross-cloud dependencies.

3. **Open Source:** Terraform is open source and free to use.

4. **State Management:** Terraform manages its own state, meaning that cloud resources created and destroyed are all kept track of in a Terraform state file. This eliminates the worry of dangling or failed-to-provision resources.

How Terraform Works

1. **Providers:** At a high level, Terraform creates resources based on the targets exposed application programming interfaces (APIs). Communication with these APIs happens through providers, which are plugins that interact with various platforms and manage their resources. These providers can be seen as a logical abstraction of an upstream API. Today, Terraform supports over 1800 providers, including major cloud providers such as AWS and GCP, as well as providers for Docker, Kubernetes, and more [13]. Providers are normally defined in a top-level `terraform.tf` file as shown below:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.47.0"
    }
  }
  required_version = "~> 1.3"
}
```

2. **terraform plan:** This command creates an execution plan, which generates a preview of the changes that Terraform plans to make to your infrastructure [24]. By default, the following steps are taken:

   (a) Reads the current state of any already-existing remote objects to ensure that the Terraform state is up-to-date.

   (b) Compares the current configuration to the prior state and generates a difference between the current and prior state.

   (c) Proposes a set of changes that, if applied, will make the remote objects match the configuration.

3. **terraform apply:** This command executes the actions proposed in a Terraform plan [24]. By default, the following steps are taken:

   (a) All the steps as detailed in the above section on `terraform plan`.

   (b) Prompt for plan approval. For auto-approval, the flag `--auto-approve` can also be passed to the `terraform apply` command.

   (c) Undertakes the actions as indicated in the plan to modify existing cloud infrastructure to meet the desired current state.

   (d) Modifies the state file to match the current state.

4. **terraform destroy:** This command is used as a way to destroy all remote objects managed by a particular Terraform configuration [24].

5. **The Terraform State File:** To make all of the above possible, Terraform needs to maintain some kind of state, so that it is aware of the current state of resources on the cloud. This is done through the `terraform.tfstate` file, which is automatically generated/modified on every Terraform command that mutates state. For local development of Terraform, the `terraform.tfstate` file is generated locally. For production and other higher environments, it is standard practice for the `terraform.tfstate` file to be stored in the cloud, such as an S3 bucket, with DynamoDB for locking, to ensure effective version control [14].

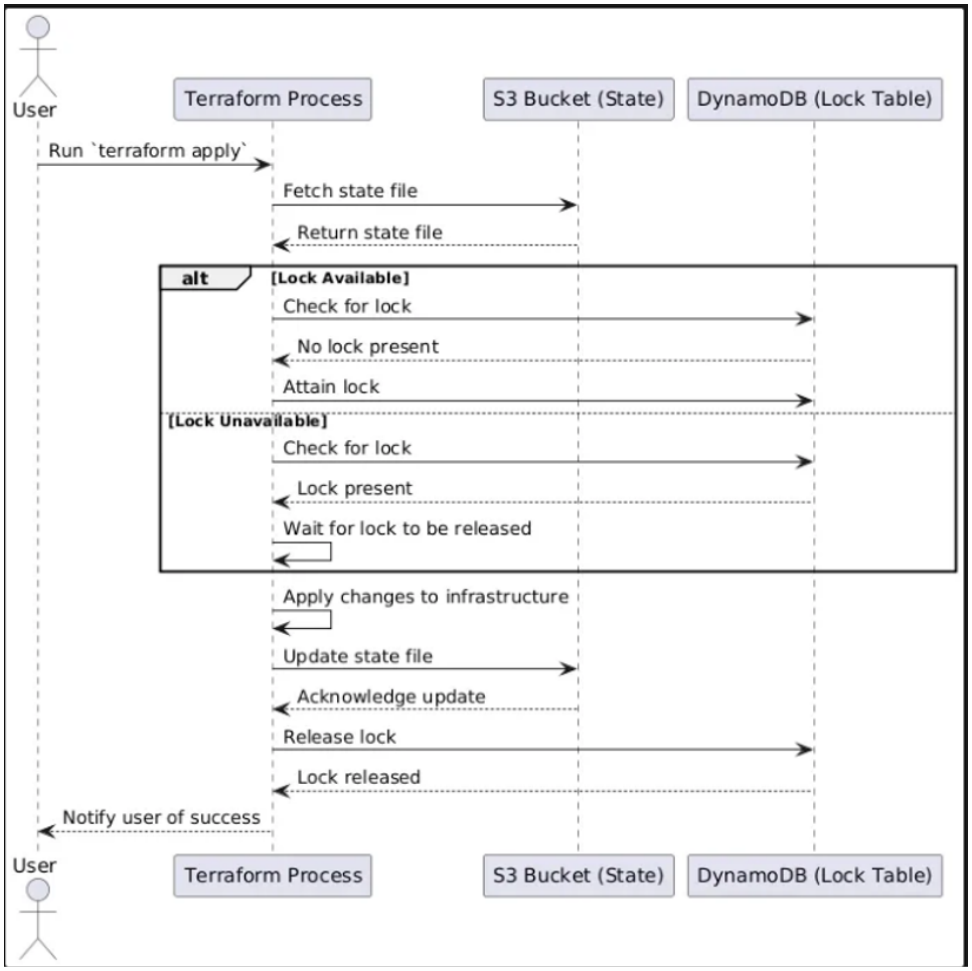

Figure 2-1: Sequence Diagram for Terraform Backend

The above diagram shows the typical workflow in handling terraform state files. In summary, state files are usually saved to some storage such as an S3 bucket. Access to these state files are then synchronized via a lock, which is commonly implemented in DynamoDB [14]. This allows race-free interaction with the provisioned infrastructure.

2.1.2 AWS CloudFormation

AWS CloudFormation is an IaC tools that allows one to manage infrastructure and automate any deployments using code [6]. CloudFormation is also done declaratively, and written in familiar languages such as YAML or JSON. This makes managing, scaling, and automating AWS resources fast and straightforward. However, the greatest downside is that CloudFormation is part of the AWS stack, and is not cloud agnostic. This means that developers are reliant on the AWS cloud should they choose to use AWS CloudFormation.

2.1.3 OpenTofu

OpenTofu is yet another open-source IaC tool designed to help with the provisioning and management of infrastructure [17]. It holds many similarities to Terraform as discussed above, but has an added feature of allowing for state encryption. There are also some differences in their licensing which will not be discussed here.

2.1.4 Ansible

Ansible is an open-source automation tool used for configuration management, application deployment, and task automation. Unlike Terraform and CloudFormation, which are declarative, Ansible uses an imperative approach to define the desired state of the infrastructure. The greatest upside of Ansible is that it is agentless, meaning it does not require any software to be installed on the target machines [25].

2.1.5 Comparison between the different frameworks

Table 2-1 shows the comparison between the different IaC frameworks discussed above [42] [36] [17].

Framework	Terraform	AWS CloudFormation	OpenTofu	Ansible
Type	Declarative	Declarative	Declarative	Imperative + Declarative
Primary Use	Multi-cloud IaC	AWS IaC	Multi-cloud IaC	Configuration Mgmt
Vendor Lock-in	No	Yes	No	No
Multi-Cloud	Yes	No	Yes	Yes
State Mgmt	Yes	Yes	Yes	No
Execution Model	Plan, Apply, Destroy	Stack-based Updates	Plan, Apply, Destroy	Task Execution
Language	HCL	JSON/YAML	HCL	YAML
Extensibility	Providers	AWS-native	Providers	Modules, Plugins
Speed	Fast	Slower	Fast	Fast
Rollback	Manual	Auto Rollback	Manual	Manual
Community	Strong	AWS-focused	Growing	Very Strong
Best Use Case	Multi-cloud infra	AWS infra automation	Open-source Terraform	Config Mgmt, Automation

Table 2-1: Comparison of Terraform, AWS CloudFormation, OpenTofu, and Ansible

2.2 IaC Orchestration and Frameworks

IaC orchestration is used to enforce good programming practices, improving code readability and quality. This provides additional structure, automation, and improves maintainability for complex infrastructure [15]. These tools provide a significant advantage over pure Terraform by improving code readability, promoting consistency across projects, and enabling faster collaboration and development in multi-environment setups.

Several IaC orchestration tools have also been built on top of terraform, such as terragrunt and terraspace [16]. While an IaC orchestration tool will not be used in the implementation due to the main aim of keeping cloudkube portable and easily distributable, it is important to note how these tools fit into the whole cloud ecosystem.

2.2.1 Terragrunt

Terragrunt is a thin wrapper that is written on top of terraform, that provides some tooling and opinions [34]. Terragrunt also provides a way to keep your code DRY by using the generate helper method. Terragrunt also helps in the automated creation of backends, which can be a chore if using raw terraform. Lastly, terragrunt allows for the injection of CLI hooks, allowing the utility to execute custom actions before or after terraform commands

2.2.2 Terraspace

Terraspace is a framework, which allows developer to dynamically generate terraform project in a centralized manner [35]. It is highly opinionated, and strongly enforces modularity and DRYness through stacks. On top of all the benefits offered by terragrunt, terraspace also allows the easy segregation of development and high-level environments through terraspace layering, which allows for the usage of the same code to deploy and create multiple environments.

2.3 Container and Container Orchestration

Containerisation has become the standard practice of the industry today. Allowing for quicker software development cycles, together with more efficient CI/CD practices and portability, containerisation provides us a powerful, and agile way to manage our applications [41]. Along with the growing complexity of distributed systems, container orchestration software such as Kubernetes are also used to automate deployment, handle scalability and fault tolerance, as well as efficient resource utilization. This chapter will discuss some of the common container and container orchestration software, which are pertinent to this project given that most SpeechLab applications have been containerised for the cloud

2.3.1 Docker

Docker is a containerisation software which aids in development, shipping, and deployment. Docker enables the separation of application from infrastructure so that software can be delivered quickly and reliably [1]. With docker, code, as well as version-locked dependencies are baked into a container image. This means that anyone building from this image on any machine in the world, regardless of architecture, will build the same thing, and run the same application. This creates reproducible environments of an application based on a single Docker Image.

2.3.2 Kubernetes

Kubernetes is an open-source container orchestration software that helps to automate deployment, scale, and manage containerised applications. Kubernetes provides a framework to run distributed systems resiliently, taking care of scaling and failover of the application [29].

Key Features of Kuberentes

1. **Self-healing:** Kubernetes restarts, replaces, or kills containers that are unhealthy, and doesn't advertise these containers to clients until they are ready to serve [40].
2. **Storage Orchestration:** Kubernetes allows the automatic mounting of a storage system of the developers choice [40].
3. **Horizontal Scaling:** Kubernetes can scale up or down an application with a simple command, UI, or automatically based on user-defined parameters [40].

4. **Service Discovery and Load Balancing:** Kubernetes can expose a container using the DNS name of the container, or using its own IP address. Kubernetes can also load balance and distribute network traffic to ensure deployment stability [32].

Kubernetes Architecture

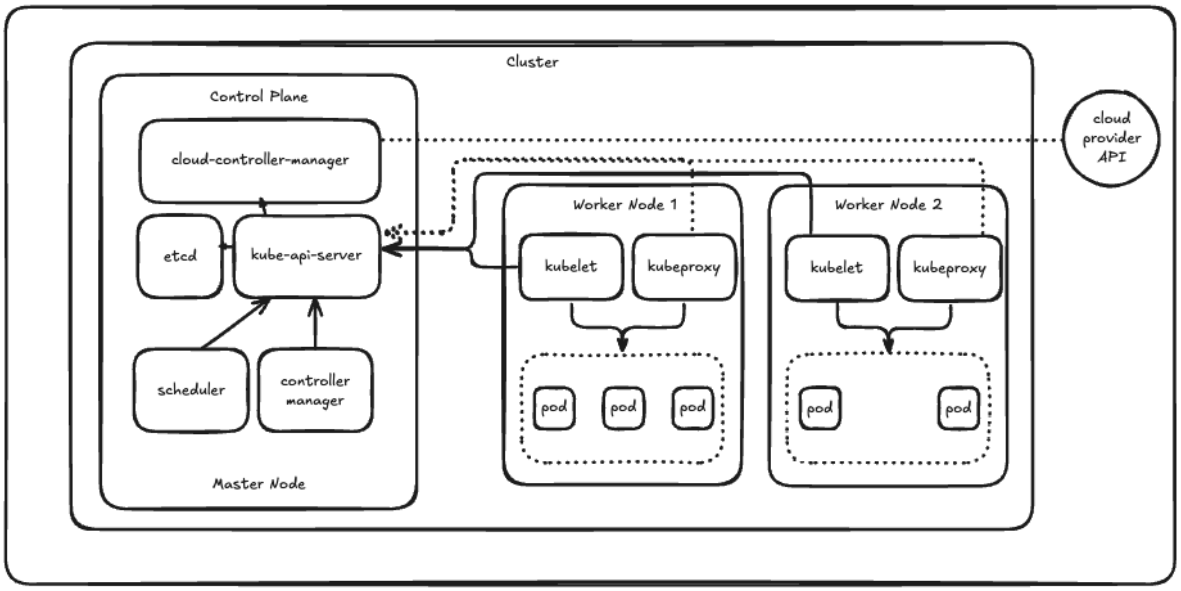

Figure 2-2: Typical Kubernetes Architecture

A Kubernetes cluster typically comprises the Control Plane as well as multiple Worker Nodes. Each of these nodes hold different responsibilities, which are detailed below.

1. **Master Node**

   The Kubernetes master node, also known as the control plane, is the controller of the whole system. This is where all operational decisions are made, including scheduling, balancing, and responding to events [18].

   (a) *kube-api-server:* The Kubernetes API Server is the main module within the master node which receives all REST requests for modifications, thereby serving as the frontend to the cluster [9]. It acts as an interface for interaction with the cluster.

   (b) *kube-controller-manager:* The kube-controller-manager runs a distict number of daemon processes, which help to regulate the shared state of the cluster, as well as perform routine tasks. When changes in configurations occur, the kcm is responsible for spotting the change and working toward the new desired state [9].

   (c) *Kube-scheduler:* The Kubernetes scheduler helps to schedule the pods on the various nodes based on resource utilization [9]. The kube-scheduler is aware of all the resources available, and is responsible for allocating pods to nodes in an efficient manner to avoid resource exhaustion. Should no resources be available, the kube-scheduler puts pods in a pending state until resources are available again.

(d) *etcd:* The etcd is a highly available, distributed key-value store that is use by Kubernetes to store cluster configuration data and state information. It keeps track of all cluster components such as nodes, pods, deployments and network configurations [26].

2. **Worker Nodes**

   The Kubernetes worker node, is a worker machine in the Kubernetes cluster. There can be multiple worker nodes in a Kubernetes cluster. Each node is responsible for running pods, and is managed by the master node. These worker nodes are the workhorses of the Kubernetes cluster, and all application execution logic is carried out in these worker nodes [28].

   (a) *kubelet:* The kubelet is the main service on a node, which regularly takes in pod specifications through the kube-api-server. The Kubelet is responsible for ensuring that pods and containers are healthy, and running in the desired state [9]. The Kubelet is also responsible for health checks.

   (b) *kube-proxy*: The kube-proxy is a service that runs on each worker node, that deals with networking. It is responsible for exposing services to the external world, or to the rest of the cluster. It performs request forwarding to the correct pods across various isolated networks in a cluster [9].

   (c) *Container runtime:* The container runtime is responsible for pulling images from a container image registry (i.e. Dockerhub, Amazon AWS ECR), and starting and stopping containers. The most common plugin used for this is Docker [9].

3. **Pods**

   A pod is the smallest unit that can be scheduled in Kubernetes [9]. Pods help in scalability through replication. In the case of errors or failed health checks, Kubernetes also creates new pods, and destroys unhealthy ones. A single pod can contain multiple containers, and these containers share an IP address and port space, and are able to communicate internally via localhost.

Networking in Kubernetes

Another concept relevant to the development of `cloudkube` is networking in Kubernetes. The following section will two main networking concepts in Kubernetes, which will become relevant to our implementation in Chapter 4.

1. **Service:** A service in Kubernetes is a logical abstraction that provides stable networking for a group of pods running in a cluster. It enables communication between Pods, and ensures consistent access to applications, even if individual pods are created or deleted. These services can act as load balancers to distribute traffic across multiple pods. Pods with type `ClusterIP` work to expose services internally whthin the cluster. Pods with type `NodePort` exposes a service on a static port on each node, and pods with type `LoadBalancer` provisions an external load balancer in the cloud to publicly expose services.

2. **Ingress:** A Kubernetes ingress object is an API object that manages external access to services in a cluster, typically through HTTP/HTTPS. The Ingress object allows us to map traffic to different backends based on rules defined via the Kubernetes API.

3. **Ingress Controller:** In order for an Ingress to work in the cluster, there must be an ingress controller running. Ingress controllers are not started automatically with a cluster, and requires a one to be specified and provisioned for ingress resources to work.

Storage in Kubernetes

Lastly, this section will detail how Kubernetes handles ephemeral and persistent storage. Similarly, this becomes relevant to our implementation as detailed in Chapter 4.

1. **Volumes:** Volumes in Kubernetes provide a way for containers in a pod to access and share data via the filesystem. This can be used for data sharing between two different containers in the same pod, or even between two different pods. There are two broad categories of volumes. Ephemeral volume types have the lifetime of a pod, and follow the lifecycle of a pod, while persistent volumes exist beyond the lifetime of a pod.

2. **Persistent Volumes (PV):** A persistent volume is a piece of storage in the cluster that has been provisioned by the developer. The data in the PV remains even after pods are destroyed or recreated.

3. **Persistent Volume Claims (PVC):** A Persistent Volume Claim is a request for storage by a user. PVCs consume PV resources, and have defined access modes, such as ReadWriteOnce, ReadOnlyMany, ReadWriteMany, or ReadWriteOncePod.

2.3.3 Managed Kubernetes Service

Managed Kubernetes Services are services offered by cloud providers which can be used to run Kubernetes without needing to install, operate, and maintain your own Kubernetes control plane or nodes. The managed Kubernetes service is able to automatically scale control plane instances based on load, detect and replace unhealthy control plane instances, and provide automated version updates and patching for them. These managed service also often offer easy integrability with other cloud services such as file systems for persistent volume storage, and load balancing for load distribution [43].

Cloud Provider Managed Kubernetes Service

- AWS Cloud: Elastic Kubernetes Service (EKS)
- GCP: Google Kubernetes Engine (GKE)
- Microsoft Azure: Azure Kubernetes Service (AKS)

2.4 minikube

minikube is a way for one to quickly set up a local Kubernetes cluster. It is compatible with macOS, Linux and Windows. The main goal of minikube is to help application developers do quick testing of their Kubernetes deployments locally, as well as a tool used for learning by new Kubernetes users [33]. By default, minikube creates a one-node cluster locally, and supports addons such as ingress controllers. As we will see in the following chapter, cloudkube functionality closely mirrors that of minikube, supporting the objective of cloudkube which is to provide a quick and easy testing environment for Kubernetes developers in the cloud. The added benefit, and problem that `cloudkube` solves over minikube is the ability for a Kubernetes cluster to be spun up in the cloud, rather than on a single node environment locally. This can help developers do more robust testing of their applications, and provide DevOps engineers a better testing framework for shipping applicaitons.

2.5 Past FYPs

Past FYPs have worked on the deployment of Speech Recognition Systems to on-prem bare metal resources [46], as well as the AWS Cloud. Benjamin Chen and Poh Kai Kiat worked to set up the CI/CD pipelines for versioning of the ASR system using Jenkins, ArgoCD and Gitlab [10] [38]. Tjandy Putra's work helped to bolster security measures through Kyverno and Falco [39]. Ma Xiao helped to set up the barebones deployment of the Speech Recognition System on the AWS cloud via the AWS console [31]. Lastly, Song Yu and Lee Kai Shern introduced terraform to help handle the infrastructure provisioning [47] [30].

These FYPs have done great work in deploying the ASR application. However, the deployment process is inherently tightly coupled with the ASR application, and changes to source code or structure of the application requires significant recalibration of cloud resources. As such, this FYP aims abstract away the process of infrastructure provisioning, and provide an automated tool for a Kubernetes cluster to be spun up easily in the cloud. As a stretch goal, this FYP also aims to be extendable to other SpeechLab applications. This will allow SpeechLab developers to focus on core application logic, and simply call the `cloudkube` API to spin up a Kuberenetes cluster, and deploy their applications.

Chapter 3: Planning and Design

With the rapid adoption of containerisation technologies, organizations are increasingly turning to containers for their scalability, portability, and efficiency in deploying modern applications [21]. This shift has prompted cloud providers to develop managed container solutions, simplifying the orchestration of containerized workloads.

In this chapter, we will detail the current approach taken in deploying a containerized application to the cloud platform. We will then analyse some of its pitfalls, and show how the proposed solution, `cloudkube`, aims to solve, and alleviate some of these issues. Mainly, each of these subsections will detail the number of steps it takes to get a working container management system up and running, as well as the amount of configuration required. We will then see the much quicker development workflow that `cloudkube` offers.

3.1 Traditional Approaches

Within the AWS ecosystem, there are two primary services offering container orchestration solutions. They are the AWS Elastic Container Service (ECS), as well as a managed Kubernetes service, the AWS Elastic Kubernetes Service (EKS). With 96% of companies who use containerisation in production opting for Kubernetes for container orchestration [44], this report will mainly focus on deploying a containerized workload on Kubernetes. That said, AWS ECS will be included in our analysis for completeness. Before the setup of ECS/EKS, there are 3 main steps that a developer has to take. This would be 1 Configuring the network, 2 Configuring the compute, and 3 Configuring the storage layer. This chapter will discuss the various configurations necessary in subsections 3.1.1 - 3.1.3. Finally, we will see how to deploy our applications to ECS in section 3.1.4, and to EKS in section 3.1.5.

3.1.1 Network Configuration

The first step in deploying a containerised workload to the cloud would be to configure the network. This includes configuring VPC and Subnet settings, as well as their related Access Control Lists, Security Groups, and routing within the cloud.

3.1.1.1 Virtual Private Cloud (VPC)

A virtual private cloud allows the creation of logically isolated sections of the cloud where different resources can be launched in a user-defined virtual network. Developers have full control over the virtual networking environment, including the selection of IP address ranges, creation of subnets, as well as configuration of route tables and network gateways. There will be

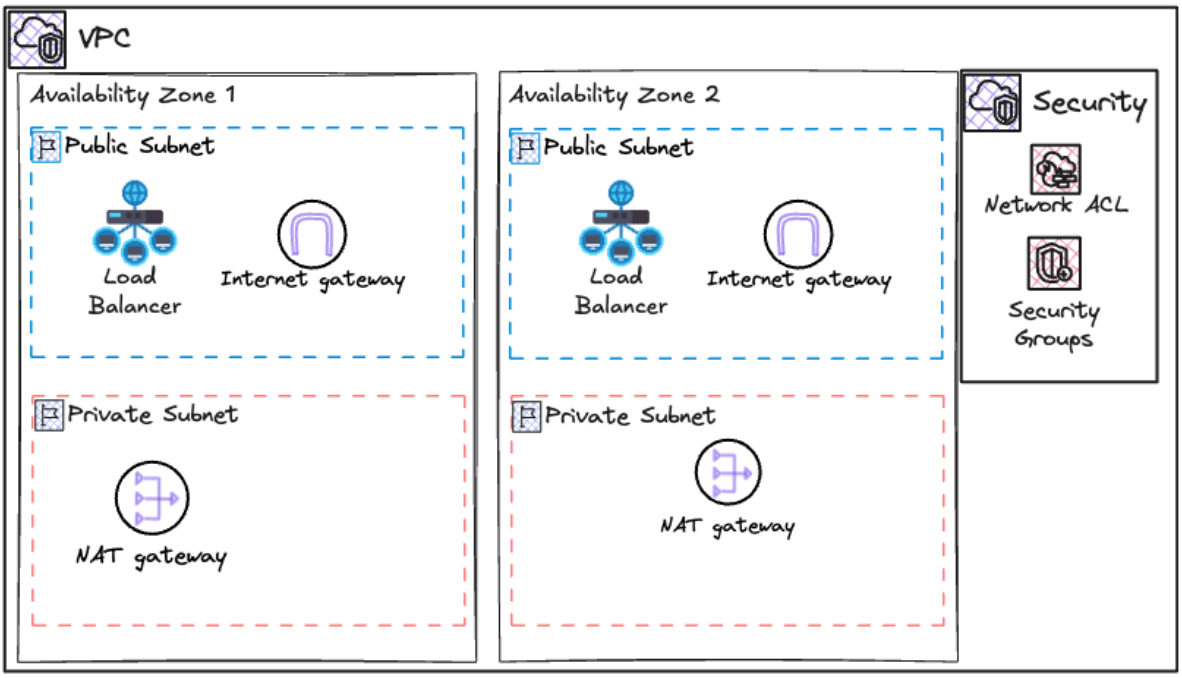

Figure 3-1: Relationship between VPC, Subnets, and other basic cloud components

mentions of some VPC related terminology in the following chapters, all of which are defined here [27].

- **Subnets:** Subnets allow the division of the VPC into different IP address ranges. They can be public or private.
- **Route Tables:** Route Tables allow the control of routing of traffic within the network. This helps in directing traffic between subnets, or between different VPCs.
- **Internet Gateway (IGW):** The Internet Gateway can be attached to a VPC to allow communication between resources in the VPC and the internet.
- **NAT Gateway:** The Network Address Translation (NAT) Gateway enables instances in a private subnet to connect to the internet or other cloud services while preventing the internet from initiating a connection with those instances.
- **Security Groups / Network Access Control Lists (ACLs):** These are virtual firewalls that control ingress and egress traffic at the instance and subnet levels, providing granular security controls.

Cloud Provider VPCs

- AWS Cloud: AWS Virtual Private Cloud
- GCP: Virtual Private Cloud Network
- Microsoft Azure: Virtual Networks

3.1.1.2 Subnets

A subnet is a range of IP addresses in a VPC. Subnets allow us to segment the network inside the VPC to group resources and control their access to each other and to the outside world. A subnet can be thought of as a sub-network within a VPC, where resources like EC2 instances can reside [12].

Key Features of Subnets

1. Public and Private Subnets
    - Public Subnets are subnets with a route to an IGW. Instances within this subnet can directly communicate with the internet, provided that they have a public IP address or an Elastic IP.
    - Private subnets are subnets which do not have a route to an IGW. Instances in a private subnet cannot communicate with the internet directly. They can only access the internet via a NAT Gateway, or NAT instances in a public subnet.
2. CIDR Blocks (Classless Interdomain Routing)
    - Each subnet is associated with a CIDR block that defines its IP address range. CIDR blocks of subnets are always within (longer prefix mask) the CIDR block of the VPC.
3. Availability Zones (AZ)
    - Subnets are created within a specific AZ in a region. Resources within a subnet are guaranteed to be located within the same AZ.
    - Distribution of subnets across multiple AZs help to increase availability and fault tolerance.
4. NACLs
    - Subnets are protected using NACLs, which are stateless firewalls that operate at the subnet level. They control ingress and egress traffic to and from the subnets based on user-defined rules.

3.1.1.3 Security Groups

Security groups are virtual firewalls that control inbound and outbound traffic to and from resources. Security groups are stateful, meaning that if you allow an incoming request from a particular IP address and port, the response is automatically allowed without needing a separate outbound rule. Some cloud providers such as Azure also draw a distinction between security groups which operate at the application layer (operating on compute instances and services), as well as the network layer (operating on ingress and egress IP addresses and protocols) [3].

3.1.1.4 Elastic Network Interfaces (ENI)

An ENI is a logical networking component in a VPC that represents a virtual network card. These ENIs are attached to instances launched in the same AZ. Every VM instance has a primary ENI attached by default, providing the instance with networking capabilities. These ENIs have multiple use cases [4]. Pertinent to this paper, the ENIs are used to facilitate communication between the Kubernetes control plane and worker nodes within the cluster.

3.1.2 Compute Configuration

Once the network is setup, the next step would be to determine the relevant compute needed, and selecting the appropriate machines for use in the VPC. At this point, to support Kubernetes workloads, we also need to configure scaling groups, which are responsible for scaling the number of compute workers up and down depending on the load.

3.1.2.1 Virtual Machines

Kubernetes applications that run in the cloud require compute to be done on virtual machines. Virtual machines are web services offered by cloud providers which allow users to run virtual servers in the cloud. The virtual machines are normally configurable in terms of the choice of processor, storage, networking as well as operating systems [45].

Cloud Provider Virtual Machines

- AWS Cloud: AWS Elastic Compute Cloud (EC2)
- GCP: Google Compute Engine (GCE)
- Microsoft Azure: Azure Virtual Machine (VM)

Pertinent to this paper, EC2 instances will be provisioned as nodes in the Kubernetes cluster. The Kubernetes pods are then ultimately run on these nodes.

3.1.2.2 Scaling Groups

Often times, it is important for compute instances to automatically scale with load. Scaling groups are a collection of compute instances treated as a logical grouping. Its main purpose and objective is to provide for automatic scaling and management. This allows the user to maintain the number of instances in a scaling group. The scaling group starts by launching enough instances to meet its desired capacity. It then maintains this number of instances by conducting periodic health checks on instances in the group. If an instance becomes unhealthy, the scaling group then terminates the instance and launches another instance to replace it [2].

Cloud Provider Scaling Groups

- AWS Cloud: Auto Scaling Groups (ASG)

- GCP: Managed Instance Groups (MIG)
- Microsoft Azure: Azure Virtual Machine Scale Sets (VMSS)

3.1.2.3 Instance Templates

Instance Templates allow users to define the configuration for launch VM instances in a more flexible and reusable way. They allow the specification of instance parameters such as instance type, key pair, security groups and more. These templates can then be used by Auto Scaling Groups, Spot Fleets, and other services to provision VM instances according to the configurations specified [22].

Cloud Provider Instance Templates

- AWS Cloud: AWS Launch Template
- GCP: Instance Templates

Appendix A shows an example AWS Launch Template to provision a VPC for use with Kubernetes.

3.1.3 Storage Configuration

Lastly, some applications required a mounted File System to operate. In the case of SpeechLab applications, a File System is needed in order to house the models. Once the network and compute configuration is done, the File System is then configured.

3.1.3.1 File Systems

File systems provide a simple, serverless, set-and-forget elastic file system for use with cloud resources or on premise infrastructure. They are built to scale on demand to petabytes without disrupting applications, growing and shrinking automatically as files are added or removed, thereby eliminating the need to provision and manage capacity to accommodate growth [7].

Cloud Provider File Systems

- AWS Cloud: Elastic File System (EFS)
- GCP: Cloud Filestore
- Microsoft Azure: Fileshare

3.1.4 AWS Elastic Container Service (ECS)

AWS ECS is a fully managed container orchestration service that helps you to deploy, manage, and scale containerised applications. AWS ECS allows for the running and scaling of container

workloads across AWS regions in the cloud, without the complexity of managing a control plane [5].

Workflow for Deploying a containerized application to ECS

Figure 3-2: Summary of configuration required for ECS

1. Create a Virtual Private Cloud (VPC)

   (a) VPCs are used to launch AWS resources into a user-defined virtual network. They serve as the foundational network infrastructure to deploy cloud resources. In the creation of our VPC, multiple factors need to be considered, including the CIDR blocks, subnet configurations, as well as Availability Zones.

   (b) By default, the VPC supports IPv4 traffic, though IPv6 traffic can be optionally enabled. Users must also decide on tenancy options, which dictate whether resources share hardware with other accounts or if their resources have dedicated access to hardware.

   (c) Subnets, essential for segmenting resources, must also be carefully designed. Public subnets provide direct access to the public internet, and must be configured with an Internet Gateway. On the other hand, private subnets are typically used to access and host internal resources such as databases, and must be configured with a NAT or Egress only Internet Gateways in order to access the public internet.

2. Configure Security groups and network ACLs

(a) Ensuring robust security control is essential when deploying containerised applications. AWS offers two access control mechanisms, namely Security Groups, and Access Control Lists.

(b) Security Groups operate at the resource level, managing inbound and outbound traffic for associated resources, such as EC2 instances. They operate at the instance level, and is stateful, meaning that all return traffic is allowed, regardless of rules. If container instances are launched in multiple regions, then developers need to create a security group in each region, and associate it with the corresponding resource.

(c) On the other hand, Network Access Control Lists allow or deny inbound and outbound traffic at the subnet / network level. These can be used as an additional layer of security, and are stateless, meaning that return traffic must be explicitly allowed by the rules.

3. Decide between Fargate (Serverless) or EC2 Instances

   (a) Developers also need to make an important decision between using AWS Fargate, a serverless option, or EC2 instances, for compute. AWS Fargate allows a way for containers to be run directly, without any virtual machines, or servers to think about. AWS Fargate is a much more managed solution, and automatically manages upgrades and out of date dependencies, but are slightly more expensive than EC2 instances at high utilization [37]. On the other hand, EC2 instances allow the system administration more control over the instances in the cluster. This means that the system administrator has more oversight on maintaining the cluster and optimizing it.

4. Preparing and pushing the container image

   (a) AWS ECS task definitions used Docker images to launch containers on the container instances in the cluster. Assuming we have a ready and runnable image on our local computer, we need to push this image to a container registry. This can be something like AWS Elastic Container Registry (ECR), or Dockerhub.

5. Create a task

   (a) Create a task definition, which specifies the Docker image to use for the containers, how many containers to use in a task, as well as the resource allocation for each container.

   (b) Next, we need to create a service using the task definition, and attach the security group created previously to this service. It is only at this point, with much manual configuration, that our containerised application is deployed to AWS ECS.

3.1.5 AWS Elastic Kubernetes Service (EKS)

With companies moving toward Kubernetes to handle increasingly complex workloads, this paper focuses on deployment of a containerised workload to Kubernetes. AWS EKS is a fully managed Kubernetes service that enables Kubernetes workloads to be run seamlessly on AWS cloud.

Workflow for deploying a containerised application to EKS

Figure 3-3: Summary of configuration required for EKS

1. Creating a VPC
   (a) As mentioned previously, the VPC serves as the foundational network infrastructure that we deploy our cloud resources to. However, the VPC for use with EKS must meet EKS networking requirements in order to ensure proper communication between Kubernetes nodes, as well as between the nodes and the external internet. These requirements include IP addressing requirements, DNS resolution support, and subnet configuration, just to name a few. The major list of requirements are detailed in Appendix B. In order to minimise configuration for developers, AWS has also provided a pre-defined CloudFormation template (Appendix A). However, this template is still very complex, and difficult to find amongst all the AWS developer documentation, making this process an extremely tedious one.
2. Create the EKS
   (a) The EKS can be created via AWS CLI or the console. Briefly, through the setup wizard, we will need to do the following:
      i. Decide on a name of the cluster, and Kubernetes version to be used in the cluster.
      ii. Create and use a Cluster IAM role, which allows the Kubernetes control plane to manage AWS resources on your behalf.
      iii. Create and use a Node IAM role, which will be attached to the EC2 instance launched and registered with the cluster.

iv. Select the VPC created earlier for use with the EKS cluster resources.
v. Select the subnets in the VPC created earlier where the control plane may place elastic network interfaces (ENIs) to facilitate communication with the cluster. These would typically be the private subnets created earlier in the creation of the VPC (one per availability zone).
vi. Choose from a wide selection of AWS add-ons (Appendix C). This is required for configuring ingress controllers, integrating with external file systems, etc.

3. Configure the EKS Node Groups

   (a) Kubernetes works by scheduling pods onto nodes, which are our compute instances. AWS EKS uses EC2 instances with Auto Scaling Groups to satisfy these requirements. At this stage, we would need to choose from the different Amazon Machine Image (AMI) types, Instance Types, as well as determine the scaling configuration for the node group, and set up networking between node groups.

4. Configure our local environment to communicate with the EKS Cluster

   (a) As with every Kubernetes cluster, the local environment needs to attain certificates, and other metadata to apply the Kubernetes config to the cloud environment. In typical production environments, this is handled through git CI/CD pipelines, or a CI/CD orchestrator tool such as Jenkins. In development, to configure the kubeconfig locally, we can run `aws eks --region <region> update-kubeconfig --name <name>` to configure our local configuration so that we are able to communicate with the Kubernetes cluster.

5. Run our Kubernetes application

   (a) Assuming that we have a Kubernetes configuration ready to go, we can then apply the Kubernetes configuration to the cluster, and begin testing our application.

3.2 Challenges

As illustrated above, the traditional approach of deploying a containerised application is highly cumbersome and error-prone. Missing any of the essential steps listed often leads to hours of debugging – whether it's identifying why the website is unresponsive, troubleshooting connectivity issues between nodes or exposing the application to the internet. Each and every step plays a critical role in ensuring that the application runs smoothly and error free. The traditional approaches are also very time-consuming, and have a steep learning curve. Engineers must gain mastery of the different cloud technologies, and know how to integrate them seamlessly, making this task both challenging, and resource-intensive.

Complexity of Configuration

Setting up the environment for containerised application deployment requires a lengthy process of setting up networking within the cluster. This requires developers to understand and gain a mastery over the different cloud components, and learn to apply them within the cloud. A small misconfiguration here could lead to connectivity issues, or resource exhaustion. Furthermore,

configuring the security layer also requires careful planning, on order to ensure both security and functionality, making the process extremely time consuming and tedious.

Manual Work

Administrators are required to perform many repetitive tasks, especially if they are required to provision infrastructure for multiple applications. This requires the repeated configuration, and drastically slows down the deployment process and introduces inefficiencies.

Risk of Errors

Given the numerous configuration options and types, errors in any of the above steps can also lead to failures, downtime, or suboptimal performance of the cluster.

Lack of Streamline Integration

The approaches detailed above are also only used for a barebones configuration of a cluster, and does not integrate other services such as Load Balancing or persistent storage. Integration with these services, should an application require it, will take additional manual setup, and maintenance.

Steep Learning Curve

Provisioning of these infrastructure resources require a deep understanding of container orchestration and services. Without prior expertise, it is easy to misconfigure the cluster, which could lead to security vulnerabilities or broken deployments. Learning the different cloud technologies is very time consuming, and will inevitably result in reduced velocity in teams.

3.3 Proposed Solution

The proposed solution, `cloudkube`, aims to alleviate the above challenges. Besides having an intuitive user interface, `cloudkube` leverages on the strengths of existing IaC frameworks such as terraform. This allows users to abstract in-depth knowledge about terraform and the cloud away, and provides a quick and easy way to deploy a containerised application to the cloud.

3.3.1 Infrastructure

Initially, `cloudkube` was designed to host the SpeechLab ASR Application, focusing on a specific set of infrastructure needs. However, as the system evolved, `cloudkube` was extended to support multiple applications through providing genering infrastructure resources that could be leveraged across different workloads. This subsection will detail the system architecture of the SpeechLab ASR Application. We will then discuss the resources to be provisioned to satisfy each of the Kubernetes components.

Figure 3-4 shows a simplified architecture diagram for the SpeechLab ASR Application. The Kubernetes manifest for deploying this application is appended in Appendix H. The decoding-

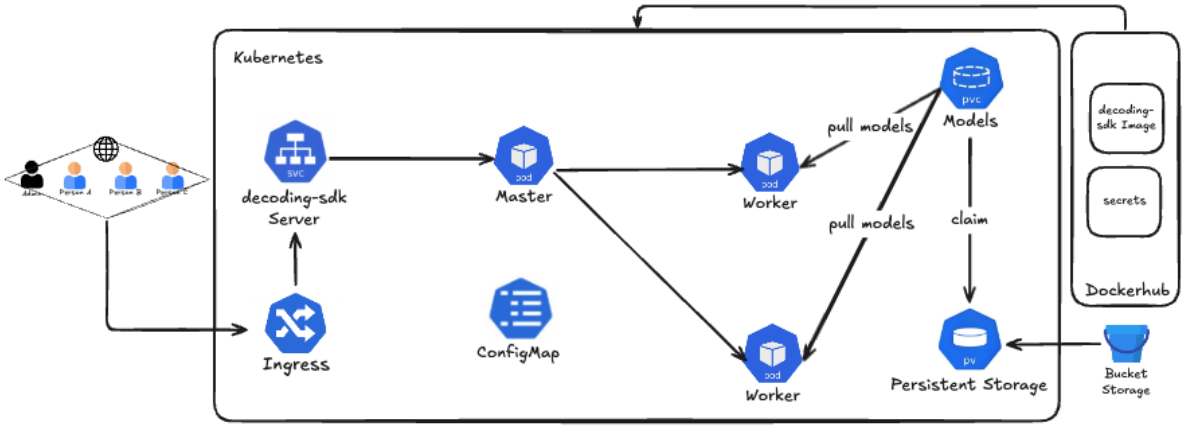

Figure 3-4: Architecture Diagram for SpeechLab ASR Application

sdk images are pushed to Dockerhub, and the machine learning models are hosted within an AWS S3 bucket. On application of the Kubernetes manifest, the images are pulled from Dockerhub. These images represent the code that is to be run in the master and worker pods. Following this, an init container pulls the models from the S3 bucket storage into the Persistent Volume in the Kubernetes environment, which allows master and worker pods to use these models for the live transcription. The ingress then controls the routing to the decoding-sdk server, which exposes the application through a Kubernetes service. Users are able to reach this provided endpoint with their audio file, and receive a live transcription.

To satisfy these Kubernetes components, multiple different types of infrastructure need to be provisioned within the cloud. Here, we note that `cloudkube` only takes care of provisioning infrastructure for Kubernetes components. As such, the bucket storage is not provisioned by `cloudkube`. Table 3-1 shows a mapping of each component to the corresponding infrastructure provisioned.

3.3.2 Key Principles

`cloudkube` is built on three main principles:

1. Simple and Minimal Dependencies: In order to satisfy the simplicity of packaging, distribution, and installation of `cloudkube`, we opt to keep dependencies as lean as possible, to account for compatibility with host architecture as well as conflicting dependencies. The distribution is also kept lightweight, in that `cloudkube` is not meant for a full-blown production setup, but rather a quick and easy way to do development testing in the cloud. As such, more robust frameworks such as terraspace and terragrunt are intentionally excluded. With these, and python installed on a user's machine, a user is able to do a simple 'pip install `cloudkube`' to install `cloudkube` on to their system, and get their first Kubernetes application running in the cloud.

2. Leveraging current IaC best practices and tooling: `cloudkube` is opinionated in the sense that it defines a terraform configuration of the cloud in a specified manner. Used as a tool for beginners, terraform details and implementation are abstracted away, and

Kubernetes Resource	Purpose	Provisioned Infrastructure
`Namespace`	Logical grouping of resources within the cluster	-
`ConfigMap`	Stores configuration data as key-value pairs, injected into pods	-
`PersistentVolume (PV)`	Provides persistent storage within the cluster, ensuring data persistence across pod lifecycles. Typically backed by external storage.	AWS Elastic File System
`PersistentVolumeClaim (PVC)`	Requests dynamically allocated storage from an EFS-backed PV	-
`StorageClass`	Defines dynamic storage provisioning using EFS	-
`Deployment`	Manages pod replicas and lifecycle of pods. These pods are the smallest unit in Kubernetes. Pods run on nodes, which are EC2 machines in the cloud.	AWS Elastic Compute Cloud
`External Service`	Responsible for exposing internal services.	AWS Elastic Load Balancer (ELB)
`Ingress`	Configure routing within the cluster, are only effective if ingress controller is present in the cluster.	NGINX Ingress Controller

Table 3-1: Mapping of Kubernetes Resources to Provisioned Infrastructure

`cloudkube` sets up a Kubernetes cluster in a pre-defined manner, much like what is done in minikube. `cloudkube` also uses the existing methods of tracking terraform state (through the '.tfstate' file). On top of that, to track the installation state of dependencies such as ingress-controllers or persistent volume infrastructure, a configuration file is introduced as well.

3. Intuitiveness and Ease of Use: Drawing inspirations from some of the best-in-class frameworks used today, `cloudkube` is built with a easy to use terminal interface. From create-react-app [19], `cloudkube` implements a setup wizard where users respond to prompts to customize their cluster to fit their development needs. From minikube [33], cloudkube implements imperative methods to configure cluster ingress controllers and persistent volumes.

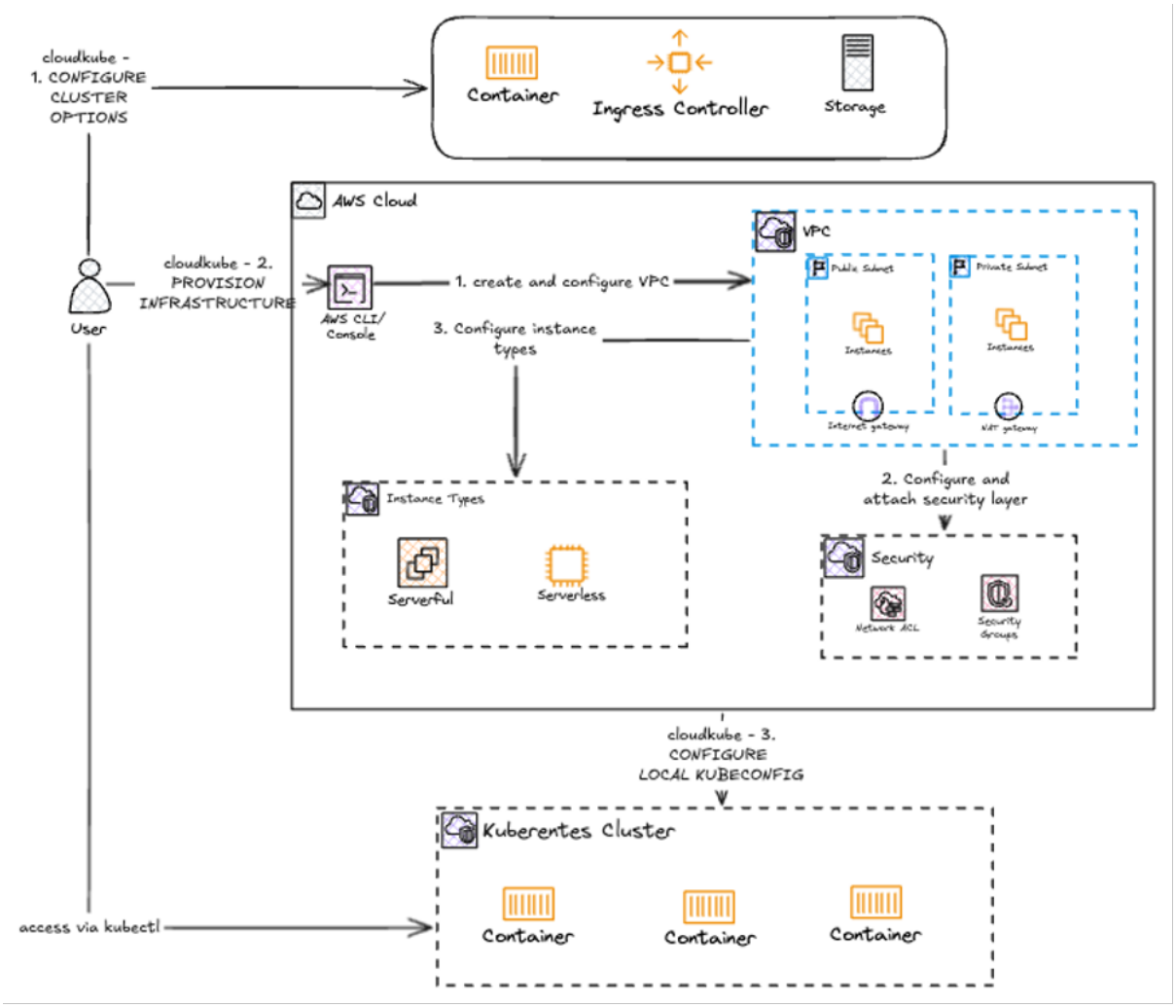

Figure 3-5: Workflow for creating a Kubernetes cluster with `cloudkube`

3.3.3 Workflow

The workflow to get an up and running Kubernetes cluster on the cloud is made much more effortless with `cloudkube`. Besides having to wait for about 10 minutes for the cluster to be provisioned, as well as some configurations for authenticating with the cloud, `cloudkube` automates the provisioning of all cloud resources at the click of a button.

Typical Workflow for Deploying an Application `cloudkube`

1. Packaging and Containerisation
   (a) As with deploying any application to the cloud with Kubernetes, the first step to deploy and application with `cloudkube` is to containerise the application. This can be done with Docker.
   (b) Container images to be used in Kubernetes pods should also be pushed to a container registry which the cloud is able to pull from. Examples include Amazon's Elastic Container Registry, or Dockerhub. Kubernetes secrets for authenticating with these image registries should also be appropriately setup.

2. Defining the `cloudkube` configuration

   (a) `cloudkube` relies on a configuration file `config.json` in its root directory to conditionally provision different types of infrastructure. Depending on the use-case, as well as the requirements of the application, users can select from different computer architecture types, availability zones as well as naming conventions for resources to be created in the cloud.

   (b) Based on the the Kubernetes configuration, users also need to decide whether to provision and ingress controller, as well as what type of ingress controller to provision.

   (c) Lastly, `cloudkube` realises PersistentVolumes in Kubernetes through provisioning an Elastic File System which can be mounted to Kubernetes nodes. This helps to provide persistent storage for Kubernetes deployed applications.

3. Provision Infrastructure

   (a) Before the cloud infrastructure can be applied, the root Terraform module needs to be initialised. `cloudkube` automatically initializes Terraform, and downloads the relevant providers and modules for its inbuilt Terraform configuration.

   (b) The next step taken by `cloudkube` is to plan the infrastructure through calling 'terraform plan'. This creates a plan for the infrastructure to be created in the cloud, and `cloudkube` then outputs the plan, and seeks user approval for creating these resources.

   (c) Upon confirmation by the user, `cloudkube` applies these changes to the cloud, and creates a Kubernetes cluster in the cloud. The infrastructure is created based on the user-specified configuration in the previous step.

4. Configure Local Machine to Communicate with the Kubernetes Cluster

   (a) In typical Kubernetes workflows, users would have to run some cloud-provider specific commands to populate their local Kubernetes configuration in order to communicate with the Kubernetes cluster.

   (b) This step is automatically done by `cloudkube` within the provisioning step. This means that users are able to get straight to applying their application to the Kubernetes cluster.

5. Apply application to Kubernetes Cluster

   (a) Users can apply their Kubernetes configuration to the configured Kubernetes cluster, and begin interacting with their application. Users are also able to interact with their cluster through the various kubectl commands.

Step by step instructions for the initial setup, as well as for developing with `cloudkube`, are detailed in Appendix D.

As we can see from the above workflow, the chore and tedium of infrastructure provisioning is greatly abstracted away from the user. This allows a user to deploy their containerised application to the cloud in a matter of minutes. Specific to deploying the ASR application, the use of `cloudkube` abstracts away many tedious steps as illustrated by Song Yu's work

on *Infrastructure as Code with Terraform and Terragrunt* [47]. More specifically, with `cloudkube`, users do not need to do the following steps:

1. **Initialise Terraform:** `cloudkube` does this behind the scenes for you. Fetching modules, providers, and updating backend configuration are all done automatically by `cloudkube`, through `terraform init`.
2. **Local Kubernetes Context Configuration:** After the infrastructure provisioning stage, `cloudkube` automatically connects to the cluster, and updates your local Kubernetes configuration files. This ensures that the user is able to communicate with the Kubernetes cluster via any `kubectl` commands.
3. **Mounting the EFS on Amazon EC2 Instance:** `cloudkube` automatically mounts an EFS to the Kubernetes nodes in the EKS. After applying the Kubernetes configuration to the cluster, users are able to interact with the file system.

As illustrated, the deployment process of ASR application to the cloud is greatly simplified. Furthermore, this is just an example of a deployment of an ASR application. In the implementation section, we will also see another example of deploying a more complex application with multiple pods, and services that communicate with each other.

With `cloudkube`, development times are greatly decreased, and developers can focus on the application business logic rather than having to debug an obscure permission setting on the cloud, or be a master of cloud technologies. Furthermore, users are also accorded configurability of the cluster, including being able to choose their machine architecture, ingress controller, as well as the provisioning of an EFS volume for PersistentVolume objects in Kubernetes. The configuration of `cloudkube` is also a low-cost one, in that low-spec machines are chosen from the various machine images offered by AWS.

3.3.4 Benefits of `cloudkube`

By leveraging the power of IaC, `cloudkube` is able to maintain many of the guarantees and benefits that terraform provides. `cloudkube` also greatly simplifies and accelerates the provisioning of infrastructure.

High Customizability and Extensibility

In order to meet the needs to different Kubernetes workloads, `cloudkube` is built to be configurable. For example, where Ingress resources are declared in Kubernetes, `cloudkube` offers the installation of a nginx ingress controller to satisfy Kubernetes Ingress resources. Similarly, in order to satisfy PersistentVolume and PersistentVolumeClaim resources, `cloudkube` also offers the installation of an EFS in the cluster. On top of this, users are also able to customize the machine architecture to ensure compatibility with their applications.

Low Cost Prototyping Solution

`cloudkube` is configured with mostly AWS Free Tier resources. As such, creating a Kubernetes cluster with `cloudkube` is cheap, and not resource intensive. Furthermore, `cloudkube` only

provisions what you want. This means that only resources that are required by the Kubernetes application are provisioned. This ensures infrastructure is kept lean, and costs are kept low.

Lean Package Distribution on pyPI

`cloudkube` is conveniently distributed on pyPI, which means that anyone with python and pip installed on their machine and conveniently download `cloudkube` with a single command 'pip install `cloudkube`'.

Preconfigured Permissions, Security Groups, Access Control Lists, IAM Roles

`cloudkube` comes with pre-configured permissions, security groups, ACLs, and IAM Roles. Configured IAM Roles ensures that cluster nodes have sufficient permissions on AWS to communicate with each other as well as the external internet. The pre-configured security groups also ensure that traffic between different cloud components is properly configured, to allow flow of information between these components securely. Ingress and Egress resources also govern traffic into and out of the cluster, ensuring that users can reach their deployed services.

Built on terraform

Being one of the leaders in IaC tooling, terraform is widely used across the industry. It is both robust, and battle-tested by leading firms in the industry. `cloudkube`, being built on terraform, retains all of the benefits that terraform brings. The most notable would perhaps be the tracking of the state file (`terraform.tfstate`). Terraform uses the state to determine the changes to be made to infrastructure. This ensures that the state of the cloud is always reflected on the terraform state file. Prior to any provisioning or teardown commands in `cloudkube`, the existing cloud infrastructure is compared against the state file, and `cloudkube` fails safely if the system is in an inconsistent state. This avoids leaving dangling resources on the cloud.

Chapter 4: Implementation

In this chapter, we will delve deeper and take a closer look at the implementation details of `cloudkube`. This chapter is organized into several sections.

Section 4.1 will discuss the architecture of `cloudkube`, where we will see a high level overview of the different modules and services in `cloudkube`. We will also see the detailed workflow of `cloudkube` via the CLI, which will allow us to see how the different modules interact with one another.

Next, section 4.2 will discuss system design patterns used for the implementation of `cloudkube`. We will rationalise these choices, and explore how these patterns are realised in `cloudkube`.

Section 4.3 will delve deeper into each of the different modules in `cloudkube`, where we will look at detailed code segments to see how the different functions are implemented.

Lastly, section 4.4 will cover the publishing of `cloudkube` to `pip` for easy distribution, as well as the development of github workflows for publishing and realising to `pip`.

4.1 `cloudkube` Architecture

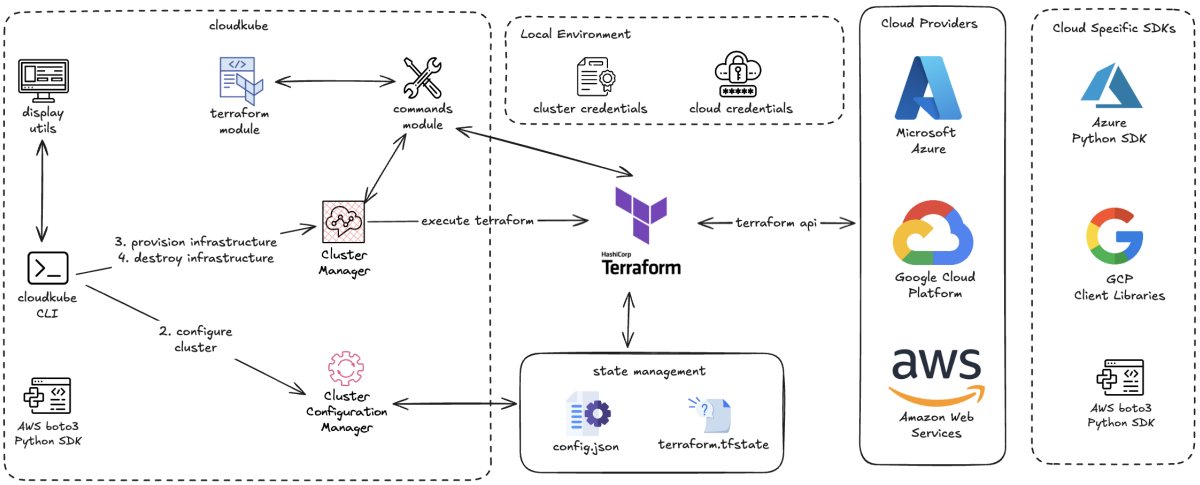

Figure 4-1: `cloudkube` architecture diagram

`cloudkube` is an orchestration tool designed to simplify the development and management of Kubernetes clusters in cloud environments. It leverages terraform for infrastructure provisioning, and provides python scripts for managing the various cloud components. These

python scripts are made available thorugh a simple CLI interface, which will be detailed in a later section. It should be noted that the primary objective of this project was to simplify infrastructure provisioning. Consequently, the focus was placed on designing and implementing robust cloud infrastructure, while the user interface was intentionally kept simple. In order to streamline interactions, `cloudkube` features a straighforward CLI, ensuring functionality and ease of use without unnecessary complexity.

4.1.1 Configuring the cluster

As we see from Figure 4-1, the main entry point into the system is through the `cloudkube` CLI. This entry point (Figure 4-2) can be accessed through the command `cloudkube init`.

Figure 4-2: `cloudkube` entry point

Following this, the first step would then be to select Option 2, where control is handed over to the `Cluster Configuration Manager`. The corresponding UI is show in Figure 4-3. At this stage, the `Cluster Configuration Manager` asks us for the following input:

- **Region:** This refers to the region in the cloud where we wish to provision our resources.
- **Prefix:** This allows us to set a unique prefix, for easy identification of our cloud resources created. This is especially useful for working in large teams, where cloud environments have many resources provisioned.
- **Architecture of Machines:** This allows us to choose between the two major types of machine architectures, `x86` and `arm`.
- **Ingress Controller Types:** This allows us to choose whether we need an ingress controller, as well axs the type of ingress controller to use. If selected, this ultimately provisions an load balancer in the cloud, which satisfies Kubernetes `Ingress` resources.

- **Persistent Volume Setup:** This allows us to choose whether we need File System setup, which ultimately satisfies Kubernetes `Persistent Volume` resources.

```
This will override any previous configuration set in config.json. Continue? [y/N]: y
Select Region (ap-southeast-1): ap-southeast-1
Set prefix of all cloud resources: minghan-eks-cluster
Select the architecture of your machines:
       1. arm (uses AL2_ARM_64)
       2. x86 (uses AL2_x86_64)
Enter a number: 2
Select (if required), and ingress controller type:
        1. None
        2. aws-nginx-ingress-controller
Enter a number: 2
Do you require persistent volume setup? [y/N]y
```

Figure 4-3: `cloudkube` cluster configuration

```
Configuration:
 {
    "region": "ap-southeast-1",
    "prefix": "minghan-eks-cluster",
    "architecture": "AL2_x86_64",
    "ingressController": "aws-nginx-ingress-controller",
    "requireEFS": "True"
}
Configuration successfully saved to config.json!
[hint]: You can now spin up your cluster by selecting option 3.
```

Figure 4-4: `cloudkube` cluster configuration success

```
(fyp) minghanchan@Mings-MacBook-Pro-3 FYP Report % ls -l | grep config.json
-rw-r--r--   1 minghanchan  staff      188 Feb  2 09:58 config.json
```

Figure 4-5: `cloudkube` `config.json` saved to our current working directory

```
(fyp) minghanchan@Mings-MacBook-Pro-3 FYP Report % cat config.json
{
    "region": "ap-southeast-1",
    "prefix": "minghan-eks-cluster",
    "architecture": "AL2_x86_64",
    "ingressController": "aws-nginx-ingress-controller",
    "requireEFS": "True"
}%
```

Figure 4-6: Contents of `config.json`

At each of these steps, the `Cluster Configuration Manager` validates our input. Once our input is verified, the `Cluster Configuration Manager` shows the corresponding configuration file that it has saved to `config.json` (Figure 4-4). It should be noted that thie `config.json` file is saved to our current working directory, which is where we ran `cloudkube init` from.

In the previous step, we ran `cloudkube init` in our `FYP Report` directory, and Figure 4-5 shows the `config.json` in our `FYP Report` directory. Figure 4-6 shows the contents of `config.json`, which is output by the `Cluster Configuration Manager`.

At this point, we have entered all the configuration that `cloudkube` requires, and `cloudkube` is ready to provision our desired infrastructure for us (Figure 4-4).

4.1.2 Provisioning the Infrastructure

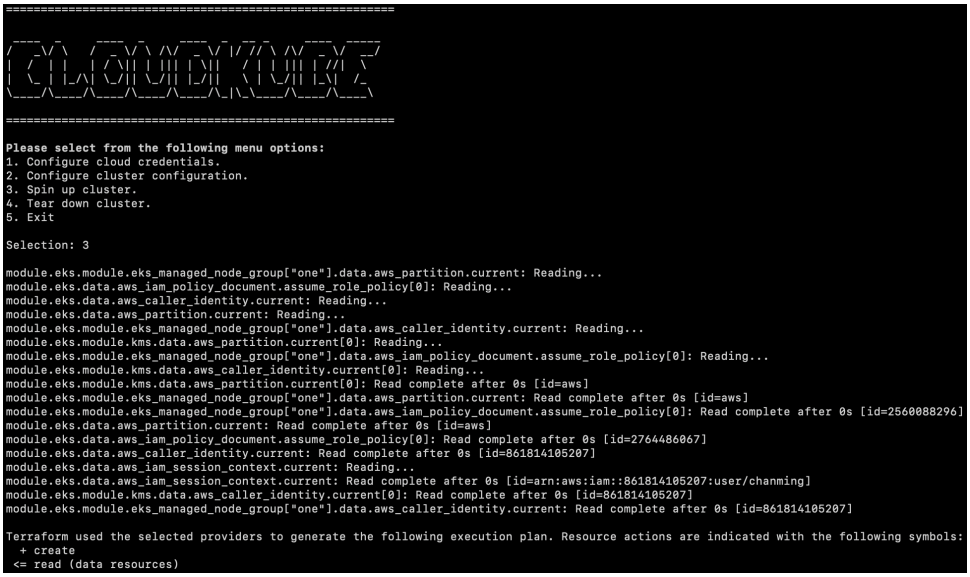

Figure 4-7: Truncated output of terraform plan

Figure 4-8: User input for approval of terraform plan

In order to spin up our infrastructure, we will then select Option 3 (Figure 4-7). `cloudkube` then hands over control to the `Cluster Manager`. The `Cluster Manager` first reads from the configuration file `config.json` that we configured earlier. It then uses these options to provision the corresponding infrastructure on the cloud. The `Cluster Manager` will then output the plan from `terraform` for user approval (Figure 4-8). The full output is attached in Appendix E. Upon user approval, the plan is then applied to the cloud environment. Any errors, info, or warning logs output by terraform are reflected into the `cloudkube` CLI for easier debugging by the user.

Figure 4-9: truncated `cloudkube` output upon successful provisioning

Once the terraform plan has been successfully applied to the cloud, relevant information about the cluster, as well as info logs are output by `cloudkube` (Figure 4-9). The full output by `cloudkube` upon successful infrastructure provisioning is presented in Appendix F.

```
1     Outputs:
2
3 cluster_endpoint = "https://5862339F7E77382B0A74D8C85BD42D31.gr7.ap-southeast-1.eks.
      amazonaws.com"
4 cluster_name = "mh-fyp-eks-wFMZsm5Z"
5 cluster_security_group_id = "sg-0869b037f781256c0"
6 efs_id = "fs-078a7048b2c301c27"
7 region = "ap-southeast-1"
8 [INFO]: Terraform applied successfully
9 Raw Terraform output for 'cluster_name': mh-fyp-eks-wFMZsm5Z
10 Raw Terraform output for 'region': ap-southeast-1
11 region:ap-southeast-1, cluster_name:mh-fyp-eks-wFMZsm5Z
12 [INFO]: kubectl configured successfully
13 [INFO]: install_aws_nginx_controller: Applied resource from https://raw.
      githubusercontent.com/kubernetes/ingress-nginx/controller-v1.12.0-beta.0/deploy/
      static/provider/aws/deploy.yaml successfully
```

Listing 4.1: truncated `cloudkube` output upon successful provisioning

Referring to Listing 4.1, we see at line 8 that the application of the terraform plan has completed successfully. The corresponding outputs, including the provisioned EFS that we configured previously (Figure 4-6), are shown. These outputs can then be used in our Kubernetes manifests.

At line 12 (Listing 4.1), we see the log `kubectl configured successfully`. Under the hood, `cloudkube` authenticates the local environment with the Kubernetes cluster provisioned in the cloud. This populates the `~/.kube/config` file, which allows us to communicate with the Kubernetes cluster from our local environment.

Lastly, given that we required an ingress controller for our set up (Figure 4-6), `cloudkube` provisions an ingress controller, and logs the successfully provisioning of this ingress controller at line 13 (Listing 4.1).

At this stage, we have a fully functional Kubernetes cluster up and running in the cloud of our choice. We can then interact with the cluster, and deploy our application to the cluster. In the subsequent chapter, we will look at the deployment of two types of applications to this cluster.

4.2 System Design Patterns

The implementation of several system design patterns in `cloudkube` was instrumental in ensuring the system was both modular and scalable. In particular, the Command pattern helped to encapsulate user actions such as provisioning and tearing down resources into discrete, reusable commands. On the other hand, the Facade pattern was also employed to abstract the complexities of underlying operation, which helped to simplify the user experience, ensuring seamless interaction with `cloudkube` without requiring knowledge of its internal intricacies.

4.2.1 Command Pattern

The Command pattern is used to encapsulate requests as objects, which allow to parametrization of clients with different requests [20]. This allows for loose coupling between commmand objects as well as the invoker. Specific to `cloudkube`, the command pattern is seen in the menu-driven system `run_wizard`.

```
1  def run_wizard():
2      while True:
3          display_menu_options()
4          selection = input("Selection: ")
5          print()
6          match selection:
7              case "1":
8                  clusterConfig.configure_cloud()
9              case "2":
10                 clusterConfig.configure_cluster_interactive()
11             case "3":
12                 ClusterManager.spin_up_cluster()
13             case "4":
14                 ClusterManager.tear_down_cluster()
15             case "5":
16                 print("exiting...")
17                 break
18             case _:
19                 print("Invalid option selected. Please re-enter.")
20
```

Listing 4.2: Code snippet for main `cloudkube` entrypoint

As seen in the code snippet above, the `match` statement in `run_wizard` maps the user input to specific operations. Each option then corresponds to a command, that is encapsulated as a function all by a controller component. This allows the easy extension of a specific action,

allowing for easy addition of new commands without drastic changes to the core application logic.

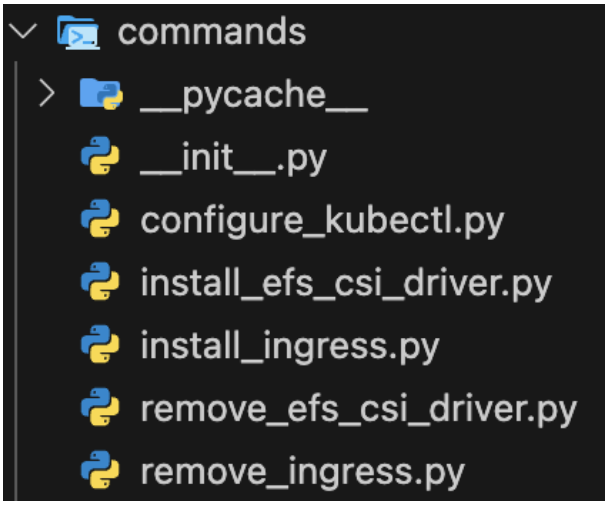

Figure 4-10: Commands Module

While the individual commands in `cloudkube` are not explicit Classes, they are abstracted to a package within `cloudkube` (Figure 4-11). Each of these command objects perform specific actions, and are call upon by the controllers (i.e. `Cluster Manager`). It should also be noted that each of these commands come with a corresponding *undo* command. For example, the `install_efs_csi_driver` has a corresponding `remove_efs_csi_driver`. As we will see later on, this makes it easy to make calls directly to the `cloudkube` CLI to modify cloud configuration.

4.2.2 Facade Pattern

The Facade pattern helps to provide a unified interface, which makes the system easier to use [20]. In the case of `cloudkube`, the `Cluster Manager` and `Cluster Configuration Manager` modules serve as facades for more complex underlying objects. They help to provide a simplified, high-level interface for cluster configuration, as well as spinning up and tearing down clusters.

```
    def spin_up_cluster(path_to_config="config.json"):
        cfg = ClusterManager.__parse_config(path_to_config)
        if not cfg:
            print(
                "Invalid configuration file provided. Have you run '2. Configure
                    cluster configuration?'"
            )
            return

        # Step 1: Apply Terraform
        cluster_name, region = apply_terraform()

        # Step 2: Configure kubectl for the EKS cluster
        configure_kubectl(cluster_name, region)

        # region, prefix, architecture are used in the main.tf terraform code
```

```
16
17          # at this step, we only need to check ingress_controller and require_efs
                state
18
19          if cfg["ingress_controller"] != "None":
20              install_ingress_controller(cfg["ingress_controller"])
21
22          if cfg["require_efs"] == "True":
23              apply_efs_csi_driver(
24                  "kubernetes-sigs/aws-efs-csi-driver", ref="release-2.0"
25              )
26
```

Listing 4.3: Code snippet for ClusterManager class method

For example, in the code snipped above for the class method `spin_up_cluster`, we see a grouping of multiple lower-level operations such as `apply_terraform`, and `configure_kubectl`. This abstracts away the intricate details when provisioning the cluster, making the code more readable, and navigatable when understading the underlying logic in `cloudkube`.

4.3 Module Implementations

4.3.1 commands module

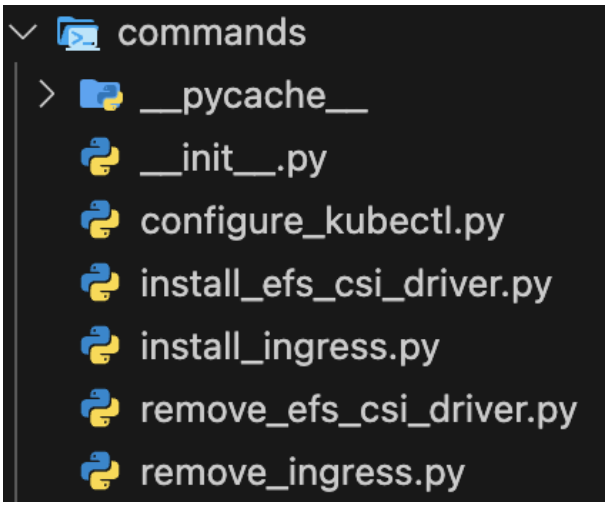

Figure 4-11: Commands Module

The commands module is structured to adhere to the Command Pattern as discussed earlier. Each command is encapsulated as an individual command object. This design ensures that each operation, such as configuring `kubectl`, or installing the EFS CSI driver is modular and self-contained. By adopting this pattern, this module provides ease of extensibility, allowing new commands to be added without modifying existing implementations, or core application logic. This modular approach is particulary beneficial for `cloudkube`, enabling seamless integration of additional cloud-native workflows while maintaining a clean separation of concerns. Each

script within the commands module serves a distinct purpose, making it easier to support flexible deployment strategies. It should also be noted that every command has a corresponding ”reverse” command. This symmetrical command structure (Table 4-1) enhances reversibility and idempotency as it allows `cloudkube` to gracefully rollback configurations if needed.

Command	**Reverse Command**	**Responsibility**
configure_kubectl.py	unconfigure_kubectl.py	Configuration/Cleanup of the local environment to interact with the cluster (~/.kube/config)
install_efs_csi_driver.py	remove_efs_csi_driver.py	Installs/Uninstalls the EFS CSI driver in the cluster for persistent storage.
install_ingress.py	remove_ingress.py	Deploys/Removes an Ingress controller for managing external access to cluster services.

Table 4-1: `cloudkube` Commands and Their Reverse Actions

4.3.2 terraform module

The terraform module hosts all of the terraform code used to spin up the infrastructure. This package is distributed along `cloudkube` to allow for easy installation on any device with a python environment.

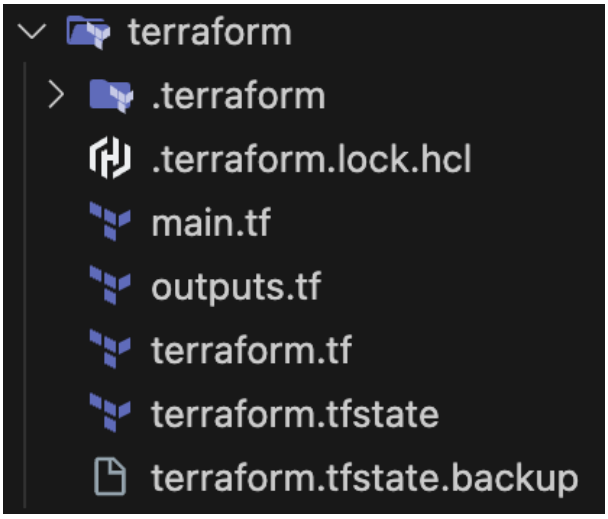

Figure 4-12: Terraform module directory structure

The three main files that will be explained in detail in this subsection are the `main.tf`, `terraform.tf`, and the `outputs.tf` file. The remaining files/folders are autogenerated by terraform. Briefly, they are

1. `.terraform` folder: This is an internal directory created by Terraform automatically during the run time of `terraform init`. It is used to store essential Terraform metadata, downloaded provier plugins, backend configurations, and state lock files.
2. `.terraform.lock.hcl`: This file is a dependency lock file that Terraform use to ensure consistent provider versions across different environments and machines. This prevents unexpected updates which could cause compatibility issues.
3. `terraform.tfstate`: This file helps to track the state of the infrastructure. It contains information about the resources created by Terraform, their current configurations, and mappings between Terraform configurations and real-world infrastructure.
4. `terraform.tfstate.backup`: This file is an automatic backup of the previous state before Terraform applies changes. It ensures that the infrastructure can be restored to the last known good state if anything goes wrong.

`terraform.tf`

This file houses the `terraform` block which is essential for defining the cloud providers. In the context of `cloudkube`, the sole cloud provider that is used is `aws`. The `random` provider is used to generate a random prefix string to prepend to all of our cloud resources for easy identification, as well as reduce probability of clashes. Lastly, the `tls` module is included to generate and manage TLS certificates, private keys, and self-signed certificates, to aid in provisioning cloud resources requiring encryption.

```
terraform {

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.47.0"
    }

    random = {
      source  = "hashicorp/random"
      version = "~> 3.6.1"
    }

    tls = {
      source  = "hashicorp/tls"
      version = "~> 4.0.5"
    }
  }

  required_version = "~> 1.3"
}
```

Listing 4.4: `terraform.tf` source code

`outputs.tf`

The `outputs.tf` file determines the output shown after running the command to provision the infrastructure. In particular, the `efs_id` should be incorporated into the Kubernetes manifest when defining the persistent volume (Line 13 of Listing 4.6). The remaining outputs are used by the `boto3` (AWS Cloud SDK) module to configure the local environment to communicate with the cluster.

```
output "cluster_endpoint" {
  description = "Endpoint for EKS control plane"
  value       = module.eks.cluster_endpoint
}

output "cluster_security_group_id" {
  description = "Security group ids attached to the cluster control plane"
  value       = module.eks.cluster_security_group_id
}

output "region" {
  description = "AWS region"
  value       = local.config.region
}

output "cluster_name" {
  description = "Kubernetes Cluster Name"
  value       = module.eks.cluster_name
}

output "efs_id" {
  description = "Persistent Volume EFS ID"
  value       = aws_efs_file_system.efs.id
}

```

Listing 4.5: `outputs.tf` source code

```
apiVersion: v1
kind: PersistentVolume
metadata:
    name: model-storage
spec:
    capacity:
        storage: 5Gi
    accessModes:
        - ReadWriteMany # EFS supports multi-node access
    storageClassName: efs-sc
csi:
    driver: efs.csi.aws.com
    volumeHandle: fs-015b986c551bc3640
```

Listing 4.6: Definition of Kubernetes Persistent Volume for ASR Application

`main.tf`

The `main.tf` file houses all the main terraform code that is used to provision the Kubernetes cluster in the AWS Cloud.

```
variable "config_file" {
    description = "Path to the JSON configuration file"
    type        = string
    default     = "" # Provide a default if applicable, or leave it empty
}

locals {
    config = jsondecode(file(var.config_file))
}
```

Listing 4.7: Parsing `config.json` as locals in `main.tf`

Firstly, we will parse the `config.json` file which is automatically generated by `cloudkube` to determine the type of infrastructure to spin up. The generation of the `config.json` file will be further elaborated on in the subsection on the Cluster Configuration Manager. This `config.json` file is then decoded via the in-built terraform `jsondecode` function to allow easy access to these variables in terraform code (Listing 4.7).

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.8.1"

  name = "${local.config.prefix}-vpc"

  cidr            = "10.0.0.0/16"
  azs             = ["ap-southeast-1a", "ap-southeast-1b"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  public_subnets  = ["10.0.4.0/24", "10.0.5.0/24"]

  map_public_ip_on_launch = true
  enable_nat_gateway      = true
  single_nat_gateway      = true
  enable_dns_hostnames    = true

  public_subnet_tags = {
    "kubernetes.io/role/elb" = 1
  }

  private_subnet_tags = {
    "kubernetes.io/role/internal-elb" = 1
  }
}

```

Listing 4.8: VPC module in `main.tf`

```
resource "random_string" "suffix" {
  length  = 8
  special = false
}
```

Listing 4.9: Random string generation

Next, we create the EKS compliant VPC. The VPC module is souced from the hashicorp/aws provider, and uniquely identified with a name prepended by a prefix. This prefix is a concatenation of the user-provided prefix in step 2 of Figure 4-3, as well as randomly generated string as shown in Listing 4.9. This uniquely generated string allows users to easily identify their resources in the cloud, and is especially useful for large teams where many resources are spun up within the cloud account. Furthermore, some AWS resources also require unique names. The prepending of this unique string helps to avoid name collisions in the cloud.

The remainder of the code as shown in Listing 4.8 sets up 2 private subnets and 2 public subnets in separate Availability Zones, as well as configures the relevant NAT Gateways for networking. Tags are also used for book keeping, as well as to identify the public and private subnets to Kubernetes during the application of the Kubernetes manifest.

```
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "20.8.5"

  cluster_name    = local.cluster_name
```

```
  cluster_version = "1.30"

  cluster_endpoint_public_access           = true
  enable_cluster_creator_admin_permissions = true
  cluster_enabled_log_types                = ["api", "audit", "authenticator",
      "controllerManager", "scheduler"]


  cluster_addons = {
    kube-proxy = {
      version = "v1.30.0-eksbuild.3"
    }
    coredns = {
      version = "v1.11.1-eksbuild.8"
    }
    vpc-cni = {
      version = "v1.18.1-eksbuild.3"
    }
    eks-pod-identity-agent = {
      version = "v1.3.2-eksbuild.2"
    }
  }

  vpc_id = module.vpc.vpc_id

  // Attach eks to all subnets in vpc (similar to amazon launch template for eks
      vpc)
  subnet_ids = module.vpc.private_subnets


  eks_managed_node_group_defaults = { # This should be user selected based on
      architecture type of docker images
    ami_type = local.config.architecture
  }

  # Extend node-to-node security group rules
  node_security_group_additional_rules = {
    ingress_self_all = {
      description = "Node to node all ports/protocols"
      protocol    = "-1"
      from_port   = 0
      to_port     = 0
      type        = "ingress"
      self        = true
    }
  }
  eks_managed_node_groups = {
    // name the nodes that are created
    one = {
      name = "node-group-1"

      instance_types = (
        local.config.architecture == "AL2_x86_64" ? ["t3.medium"] :
        local.config.architecture == "AL2_ARM_64" ? ["t4g.medium"] :
        []
      )

      min_size     = 2
      max_size     = 2
      desired_size = 2

      // We need to define additional iam policy so that node group can hit the s3
          bucket
```

```
65        iam_role_additional_policies = {
66          "AmazonS3FullAccess" = "arn:aws:iam::aws:policy/AmazonS3FullAccess",
67          "AmazonSSMManagedInstanceCore" :
                "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
68        }
69
70      }
71    }
72  }
73
```

Listing 4.10: EKS Module in `main.tf`

Next, at the core of the infrastructure, we need to create the EKS module in the cloud. This module requires the greatest amount of configuration, and the following will show what each of the above lines in Listing 4.10 are responsible for. Detailed explanations on the various configuration of the EKS module are available at the Hashicorp Documentation.

- **Line 2-10**: This sets up the minimal configuration necessary for the EKS module provided by `hashicorp/aws` provider.
- **Line 13-26**: This configures the relevant add-ons required for the Kubernetes cluster. This is equivalent to configuration of the add ons in Appendix C. The functionality of each of these add-ons are detailed in 4-2.

Addon	**Purpose**
kube-proxy (v1.30.0)	Maintains network rules on nodes for Kubernetes services.
CoreDNS (v1.11.1)	Handles DNS-based service discovery within the cluster.
VPC CNI (v1.18.1)	Manages networking for pods using the AWS VPC.
EKS Pod Identity Agent (v1.3.2)	Enables IAM authentication for pods to access AWS services securely.

Table 4-2: EKS Add-ons and Their Purposes

- **Line 34 - 62**: This configures a node group in the EKS, which represents a group of compute instances in the cluster. The architecture of these nodes (ie arm or x86) is decided here based on the user input when configuring the cluster in step 2 (Figure 4-2). It should also be noted that the nodes within a Kubernetes cluster must be allowed to communicate with each other since the Kubernetes control plane pod may be deployed to any node. As such, in order to allow for commands to be sent between pods, all communication is allowed between nodes (Line 39 - 48).
- **Line 65 - 68**: Lastly, the SpeechLab application requires models to be pulled from an S3 bucket. As such, IAM policies are attached to the nodes within the cluster, to ensure that they are able to access the S3 bucket.

4.3.3 Cluster Configuration Manager

The Cluster Configuration Manager is an interactive tool designed to optimize the configuration of the Kubernetes cluster. It simplifies the process of defining essential configurations, such as

region selection, **resource name prefixes**, **machine architecture**, **ingress controllers**, and **persistent storage options**. Through an interactive wizard that guides users through this setup, the Cluster Configuration Manager ensures consistency and ease of deployment. Once the configuration has been validated, the Cluster Configuration Manager generates a `config.json` file, which serves as the core reference for the infrastructure provisioned. This allows users to efficiently deploy and manage Kubernetes clusters with minimal manual intervention.

```
1
2  def configure_cluster_interactive():
3  """
4  This function provides an interactive setup for the configuration of
5  the cluster.
6
7  It will then generate the ‘variables.tf‘ file, as well as a state file for
8  resources installed on the kubernetes cluster. Examples of such resources
9  include:
10     - nginx-ingress-controller
11     - efs-csi-driver
12     ...
13 """
14     cont = input(
15         "This will override any previous configuration set in config.json.
               Continue? [y/N]: "
16     )
17     if cont != "y":
18         return
19
20     config = {}
21
22     # Configure Region Variable
23     region = input("Select Region (ap-southeast-1): ")
24     # strip whitespace
25     region = "".join(region.split())
26     if len(region) == 0:
27         # use default ap-southeast-1
28         config["region"] = "ap-southeast-1"
29     else:
30         config["region"] = region
31
32     # Configure Prefix Variable
33     prefix = input("Set prefix of all cloud resources: ")
34     config["prefix"] = prefix
35
36     # Configure Machine Architecture
37     printb("Select the architecture of your machines: ")
38     print("\t1. arm (uses AL2_ARM_64)")
39     print("\t2. x86 (uses AL2_x86_64)")
40     arch = input("Enter a number: ")
41     if arch == "1":
42         config["architecture"] = "AL2_ARM_64"
43     elif arch == "2":
44         config["architecture"] = "AL2_x86_64"
45
46     # Configure ingress-controller
47     printb("Select (if required), and ingress controller type: ")
48     print("\t 1. None")
49     print("\t 2. aws-nginx-ingress-controller")
50     choice = input("Enter a number: ")
51     if choice == "1":
52         config["ingressController"] = "None"
53     elif choice == "2":
54         config["ingressController"] = "aws-nginx-ingress-controller"
```

```
55
56      # Configure efs-csi-driver
57      choice = input("Do you require persistent volume setup? [y/N]")
58      if choice == "y":
59          config["requireEFS"] = "True"
60      else:
61          config["requireEFS"] = "False"
62
63      # Save the configuration
64      with open("config.json", "w") as f:
65          json.dump(config, f, indent=4)
66
67      print("Configuration: \n", json.dumps(config, indent=4))
68
69      printgreen("Configuration successfully saved to config.json!")
70      print("[hint]: You can now spin up your cluster by selecting option 3.")
71
```

Listing 4.11: Interactive cluster configuration wizard

As seen from the above code snippet, the Cluster Configuration Manager does some simple string validation, and outputs a `config.json` file in the current working directory that `cloudkube` is run from.

4.3.4 Cluster Manager

Lastly, the Cluster Manager is the main orchestrator in `cloudkube`. It is designed to handle provisioning and teardown of Kubernetes clusters in a structured and efficient manner. The Cluster Manager ensures that the configuration file `config.json` is valid, and provisions the cluster with the predefined setting in `config.json`. By automating the provisioning and tear down workflow, the Cluster Manager reduces manual intervention, and provides a reliable, repeatable mechanism for deploying and managing Kubernetes clusters in the cloud.

```
1  @staticmethod
2  def spin_up_cluster(path_to_config="config.json"):
3      cfg = ClusterManager.__parse_config(path_to_config)
4      if not cfg:
5          print(
6              "Invalid configuration file provided. Have you run '2. Configure
                   cluster configuration?'"
7          )
8          return
9
10     # Step 1: Apply Terraform
11     cluster_name, region = apply_terraform()
12
13     # Step 2: Configure kubectl for the EKS cluster
14     configure_kubectl(cluster_name, region)
15
16     # region, prefix, architecture are used in the main.tf terraform code
17
18     # at this step, we only need to check ingress_controller and require_efs state
19
20     if cfg["ingress_controller"] != "None":
21         install_ingress_controller(cfg["ingress_controller"])
22
23     if cfg["require_efs"] == "True":
24         apply_efs_csi_driver(
```

```
            "kubernetes-sigs/aws-efs-csi-driver", ref="release-2.0"
        )

@staticmethod
def tear_down_cluster(path_to_config="config.json"):
    cfg = ClusterManager.__parse_config(path_to_config)
    if not cfg:
        print(
            "Invalid configuration file provided. Have you run '2. Configure
                cluster configuration?'"
        )
        return
    # User Prompt
    confirm = input(
        "Before tearing down the infrastructure, it is good practice to have
            removed all your kubernetes pods, services, and everything that was
            created using the kubernetes manifest. This is so as to ensure no
            dangling services on the cloud, such load balancers, which could cause
            infrastructure deprovisioning to fail. To confirm proceed, (y/N): "
    )

    if confirm == "y":

        # Clean up procedure
        # Order is important here, we are deleting in the reverse order that we
            created
        # create A -> create B -> create C -> destroy C -> destroy B -> destroy A
        # delete_efs_csi_driver("kubernetes-sigs/aws-efs-csi-driver",
            ref="release-2.0")
        if cfg["require_efs"] == "True":
            delete_efs_csi_driver(
                "kubernetes-sigs/aws-efs-csi-driver", ref="release-2.0"
            )
        if cfg["ingress_controller"] != "None":
            delete_aws_nginx_controller()
        unconfigure_current_kubeconfig()
        destroy_terraform()
        clean_local_config_json()

```

Listing 4.12: Cluster Manager Provisioning and Teardown methods

As seen in Listing 4.12, the infrastructure is provisioned and destroyed in a stack like manner. This ensures a consistent state within the infrastructure, and prevents dangling resources in the cloud. This is important because some resources have dependencies on previous resources. For example, `unconfigure_current_kubeconfig()` removes the Kubernetes cluster context from the local machine. Doing this before removing the ingress controller will result in a dangling load balancer in the cloud, since we are now no longer able to interact with the Kubernetes cluster. As such, the deletion of resources must be in the reverse order that the resources were created.

4.4 Publishing and Distribution

`cloudkube` reimagines the way developers work, particularly in smaller and more agile teams that lack a dedicated infrastructure team. Through consolidating infrastructure provisioning into a streamlined workflow, `cloudkube` ensures that deployments are replicable, consistent,

and easy to manage. This means that at its core, `cloudkube` must be versioned tagged, and be widely distributed to facilitate installation on any developers machine. In this research project, distribution was achieved via `PyPI`, the de facto public repository of Python software packages. By leveraging `PyPI`'s extensive reach, `cloudkube` can be effortlessly installed, updated, and adopted within the Python developer community, fostering wider accessibility and adoption.

The workflow for packaging `cloudkube` is reliant on `setup.py` and `workflow.yml` (Appendix G). The `setup.py` file is responsible for maintaining the metadata of `cloudkube`, as well as tracking its dependencies for installation purposes. The `workflow.yml` file houses the GitHub Actions workflow for automating the publishing process. This ensures that every push to the main branch bumps the version number in `setup.py`, triggers a new release, thereby maintaining the latest version of `cloudkube` on `PyPI`.

Chapter 5: Results and Analysis

In this chapter, we will do a comparison between deploying a Kubernetes workload via the traditional approach and via `cloudkube`. While `cloudkube` was initially developed to meet the needs of the ASR application, this FYP has met the stretch goal of making `cloudkube` extensible to different types of Kubernetes workloads. Consequently, the subsequent sections will show the testing and deployment of the ASR application as well as a multi-service, fullstack application to infrastructure provisioned by `cloudkube`.

5.1 Deployment Timeline

This subsection will compare the deployment timeline using `cloudkube` vs the traditioanal manual approach, highlighting the time savings and operational benefits. The estimated times presented here are based on my experience, as well as anecdotal, unstructured evidence with other students working on deployment of SpeechLab applications. As such, they serve as a general guide rather than absolute values, as actual deployment durations vary depending on specific infrastructure configurations, as well as expertise in cloud technologies. In this section, we see that **deploying with `cloudkube` provides a much gentler learning curve and much quicker deployment times.**

5.1.1 Traditional Method

In the context of this subsection, the traditional method of deploying a Kubernetes workload to the cloud refers to using the AWS CLI or AWS Console for manual setup and management. Due to time and resource constraints, this FYP is focused solely on AWS Cloud, and all deployment/learning times peresented here are based on my personal experience with AWS, and may vary depending on individual familiarity with the platform and cloud specific technologies.

As seen from Table 5-1, deploying a single Kubernetes workload to the cloud involves an extremely steep learning curve. Moreover, even if one has all the relevant expertise with deploying a Kubernetes workload to the cloud, executing it takes up to 4 hours, given the complexity and number of steps involved.

Step	Time to Learn	Time to Execute
Understand Cloud Basics (VPCs, Subnets, Security Groups, etc.)	4-6 hours	N/A
Understand Kubernetes in the Cloud (Elastic Kubernetes Service, Node Groups, etc.)	6-8 hours	N/A
Install prerequisites (kubectl, AWS CLI, Terraform, etc.)	30 minutes - 1 hour	1-2 hours
Configure cloud provider and IAM roles	3-4 hours	1-2 hours
Write and apply Terraform scripts for infrastructure provisioning	5-8 hours	1-2 hours
Deploy Kubernetes cluster using Terraform	2-3 hours	15-30 minutes
Configure kubectl to interact with the cluster	30 minutes	5 minutes
Deploy Ingress controller	1-2 hours	10-15 minutes
Set up persistent storage (EFS CSI driver)	2-3 hours	15-20 minutes
Deploy sample application	1-2 hours	10-20 minutes
Verify and troubleshoot deployment	3-5 hours	Varies
Total Estimated Time	**20 - 30 hours**	**3 - 4 hours**

Table 5-1: Steps to Spin Up a Kubernetes Cluster and Estimated Time (Traditional Method)

5.1.2 `cloudkube` Method

As we can see from Table 5-2, the learning curve with `cloudkube` is made more gentle, and execution times are drastically reduced as compared to the traditional method (Table 5-1). Furthermore, the bulk of the time taken as presented in Table 5-2 is for the installation of prerequisites. Once a developer has set up his/her local environment with `cloudkube` and its corresponding dependencies, it takes **no more than 20 minutes** to get a fully functional Kubernetes cluster in the cloud. The full workflow with `cloudkube` can be found in Appendix D.

Step	Time to Learn	Time to Execute
Install prerequisites (Python, `cloudkube`, AWS CLI, Terraform, kubectl)	30 minutes - 1 hour	1-2 hours
Configure `cloudkube` (`cloudkube init` -> 2)	30 minutes	5-10 minutes
Deploy sample Application	1-2 hours	10-20 minutes
Verify and troubleshoot deployment	3-5 hours	Varies
Total Estimated Time	**2 - 3 hours**	**1 - 2 hours**

Table 5-2: Steps to Spin Up a Kubernetes Cluster Using `cloudkube` and Estimated Time

5.2 Deployment of SpeechLab ASR Application

This section will cover the deployment of the SpeechLab ASR Application via `cloudkube`. In this evaluation, we will walkthrough the end-to-end deployment process, as well as testing the API exposed by the SpeechLab ASR Application to ensure that the infrastructure provisioned suitably fits the needs of the application.

First, `cloudkube` is downloaded via `pip install cloudkube`. We then configure our cloud credentials using `aws configure`. Pictures for this step are omitted form this report to keep the aws credentials secret. Next, based on the Kubernetes manifest of the SpeechLab ASR Application (Appendix H), we see that persistent volumes are required. The ASR application also runs on `x86` architecture as dictated by the decoding-sdk. `cloudkube init` is then run, and `cloudkube` is configured (via option `2`) with `x86` machines, and PersistentVolume setup 5-1. At this point, configuring the cluster has taken no more than 5 minutes. Next, option `3` is run, and `cloudkube` begins provisioning our Kubernetes cluster in the cloud.

```
(fyp) minghanchan@Mings-MacBook-Pro-3 kafka % cloudkube init
=========================================================

 ____ _     ____  _     ____  _  __ _     ____  _____
/   _\/ \   /  _ \/ \ /\/  _ \/ |/ // \ /\/  _ \/  __/
|  /  | |   | / \|| | ||| | \||   / | | |||  | //|  \
|  \_ | |_/\| \_/|| \_/|| |_/||   \ | \_/|| |_\|  /_
\____/\____/\____/\____/\____/\_|\_\\____/\____/\____\

=========================================================

Please select from the following menu options:
1. Configure cloud credentials.
2. Configure cluster configuration.
3. Spin up cluster.
4. Tear down cluster.
5. Exit

Selection: 2

This will override any previous configuration set in config.json. Continue? [y/N]: y
Select Region (ap-southeast-1): ap-southeast-1
Set prefix of all cloud resources: mh-deploy-test
Select the architecture of your machines:
       1. arm (uses AL2_ARM_64)
       2. x86 (uses AL2_x86_64)
Enter a number: 2
Select (if required), and ingress controller type:
        1. None
        2. aws-nginx-ingress-controller
Enter a number: 1
Do you require persistent volume setup? [y/N]y
```

Figure 5-1: Configuring Kubernetes cluster for SpeechLab ASR Application via `cloudkube`

The cluster creation process takes approximately 10 minutes. Once the Kubernetes cluster is successfully created (Figure 5-2), we can then navigate to the Kubernetes manifests to run `kubectl apply -f manifest.yaml`. At this point, our application is deployed to the cloud, and ready for testing. It should be noted that the images are stored on dockerhub, while the models are pulled from an S3 bucket in our AWS cloud.

As we can see from the output of `kubectl get pods -A` (Figure 5-3), which queries the cluster for all pods in all namespaces, we see that all the pods have status of `RUNNING`, meaning that they have successfully started. Of note

```
Outputs:

cluster_endpoint = "https://F941A3CA419A0ED6B0BEB1279DF34F2F.gr7.ap-southeast-1.eks.amazonaws.com"
cluster_name = "mh-deploy-test-eks-NgXDPuyn"
cluster_security_group_id = "sg-0dc7ed32a88a1eadf"
efs_id = "fs-08638c65edca975c3"
region = "ap-southeast-1"
[INFO]: Terraform applied successfully
Raw Terraform output for 'cluster_name': mh-deploy-test-eks-NgXDPuyn
Raw Terraform output for 'region': ap-southeast-1
region:ap-southeast-1, cluster_name:mh-deploy-test-eks-NgXDPuyn
[INFO]: kubectl configured successfully
```

Figure 5-2: `cloudkube` Successful Provisioning

```
minghanchan@Mings-MacBook-Pro-3 decoding-sdk % kubectl get pods -A
NAMESPACE      NAME                                  READY   STATUS    RESTARTS   AGE
decoding-sdk   decoding-sdk-server-6889cd66c5-84d54   1/1     Running   0          2m51s
decoding-sdk   decoding-sdk-worker-5bdd4bd4db-hv8wp   1/1     Running   0          2m51s
decoding-sdk   decoding-sdk-worker-5bdd4bd4db-k7fmh   1/1     Running   0          2m51s
kube-system    aws-node-bfjmc                        2/2     Running   0          45m
kube-system    aws-node-m4gzk                        2/2     Running   0          45m
kube-system    coredns-878d47785-p5gxl               1/1     Running   0          49m
kube-system    coredns-878d47785-zsdl6               1/1     Running   0          49m
kube-system    efs-csi-controller-967cc967c-g6qzc    3/3     Running   0          8m43s
kube-system    efs-csi-controller-967cc967c-t5z7g    3/3     Running   0          8m42s
kube-system    efs-csi-node-6bnkk                    3/3     Running   0          8m42s
kube-system    efs-csi-node-c7qfq                    3/3     Running   0          8m42s
kube-system    eks-pod-identity-agent-7s76c          1/1     Running   0          45m
kube-system    eks-pod-identity-agent-bzs7n          1/1     Running   0          45m
kube-system    kube-proxy-4qqhw                      1/1     Running   0          46m
kube-system    kube-proxy-wh4sk                      1/1     Running   0          46m
```

Figure 5-3: Output of `kubectl get pods -A` for SpeechLab ASR Application

are the pods in the `decoding-sdk` namespace, which show the successful starting of the master pod (`decoding-sdk-server-6889cd66c5-84d54`) as well as the 2 worker pods (`decoding-sdk-worker-5bdd4bd4db-hv8wp` and `decoding-sdk-worker-5bdd4bd4db-k7fmh`). The `kube-system` namespace also contains pods relevant for mounting of the PersistentVolume. These are the `efs-csi-controller` and `efs-csi-node` pods.

```
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  6m30s default-scheduler  Successfully assigned decoding-sdk/decoding-sdk-worker-5bdd4bd4db-k7fmh to ip-10-0-1-114.ap-southeast-1.compute.internal
  Normal  Pulling    6m29s kubelet            Pulling image "amazonlinux"
  Normal  Pulled     6m27s kubelet            Successfully pulled image "amazonlinux" in 1.81s (1.81s including waiting). Image size: 53154540 bytes.
  Normal  Created    6m27s kubelet            Created container copy-models
  Normal  Started    6m27s kubelet            Started container copy-models
  Normal  Pulled     5m50s kubelet            Container image "minghan2000/decoding-sdk:2.0" already present on machine
  Normal  Created    5m50s kubelet            Created container decoding-sdk-worker
  Normal  Started    5m50s kubelet            Started container decoding-sdk-worker
```

Figure 5-4: Output of `kubectl describe pod decoding-sdk-worker-5bdd4bd4db-k7fmh -n decoding-sdk`

Figure 5-4 shows event log for one of the worker pods. In this event log, we see that a container `copy-models` is started. This container pulls the images from the existing S3 bucket that we have in the AWS cloud into the machine. The decoding-sdk server is then started.

All of the above outputs show that the decoding-sdk server has successfully started, and the SpeechLab ASR Application is successfully deployed. Lastly, we will hit the endpoint of the application to confirm that the live transcription works. To do this, we first need to view the load balancer exposed to the public internet. This can be done via `kubectl get services`. As we can see from the output of Figure 5-5, the external IP of the decoding-sdk server is `ad40289733a2844058b1f62d619b7c8e-1622439313.ap-southeast-1.elb.amazonaws.com`.

Finally, we can hit the endpoint and obtain our live transcription as shown in Figure 5-6.

```
minghanchan@Mings-MacBook-Pro-3 decoding-sdk % kubectl get services -A
NAMESPACE     NAME                         TYPE           CLUSTER-IP       EXTERNAL-IP                                                                    PORT(S)                  AGE
decoding-sdk  decoding-sdk-server          LoadBalancer   172.20.184.135   ad40289733a2844058b1f62d619b7c8e-1622439313.ap-southeast-1.elb.amazonaws.com   8010:31987/TCP           13m
default       kubernetes                   ClusterIP      172.20.0.1       <none>                                                                         443/TCP                  61m
kube-system   eks-extension-metrics-api    ClusterIP      172.20.167.238   <none>                                                                         443/TCP                  61m
kube-system   kube-dns                     ClusterIP      172.20.0.10      <none>                                                                         53/UDP,53/TCP,9153/TCP   59m
```

Figure 5-5: Output of `kubectl get services -A`

```
    INFO 2024-10-23 10:14:01,983 Socket opened! <__main__.MyClient object at 0x10106ze40>
    INFO 2024-10-23 10:14:01,984 Start transcribing...
Sending block of data 8000
Sending block of data 8000
Sending block of data 8000
Sending block of data 8000
Sending block of data 8000
    INFO 2024-10-23 10:14:02,792 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 2, 'result': {'hypotheses': [{'transcript': 'ya'}], 'final': False}}
    INFO 2024-10-23 10:14:02,792 ya
Sending block of data 8000
Sending block of data 8000
    INFO 2024-10-23 10:14:03,273 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 3, 'result': {'hypotheses': [{'transcript': '游戏'}], 'final': False}}
    INFO 2024-10-23 10:14:03,274 游戏
Sending block of data 8000
Sending block of data 8000
    INFO 2024-10-23 10:14:03,923 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 4, 'result': {'hypotheses': [{'transcript': '游戏 的'}], 'final': False}}
    INFO 2024-10-23 10:14:03,923 游戏 的
Sending block of data 8000
    INFO 2024-10-23 10:14:04,283 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 5, 'result': {'hypotheses': [{'transcript': '游戏 的'}], 'final': False}}
    INFO 2024-10-23 10:14:04,283 游戏 的
Sending block of data 8000
Sending block of data 8000
    INFO 2024-10-23 10:14:04,600 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 6, 'result': {'hypotheses': [{'transcript': 'you see the whole'}], 'final': False}}
    INFO 2024-10-23 10:14:04,603 you see the whole
    INFO 2024-10-23 10:14:04,729 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 7, 'result': {'hypotheses': [{'transcript': 'you see the whole of me'}], 'final': False}}
    INFO 2024-10-23 10:14:04,730 you see the whole of me
Sending block of data 8000
    INFO 2024-10-23 10:14:04,955 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 8, 'result': {'hypotheses': [{'transcript': 'you see the whole of me'}], 'final': False}}
    INFO 2024-10-23 10:14:04,955 you see the whole of me
Sending block of data 8000
    INFO 2024-10-23 10:14:05,254 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 9, 'result': {'hypotheses': [{'transcript': 'you see the whole of me i tahu'}], 'final': False}}
    INFO 2024-10-23 10:14:05,255 you see the whole of me i tahu
Sending block of data 8000
Sending block of data 8000
    INFO 2024-10-23 10:14:05,646 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 10, 'result': {'hypotheses': [{'transcript': 'you see the whole of me i tahu la'}], 'final': False}}
    INFO 2024-10-23 10:14:05,647 you see the whole of me i tahu la
Sending block of data 8000
    INFO 2024-10-23 10:14:05,889 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 11, 'result': {'hypotheses': [{'transcript': 'you see the whole of my life'}], 'final': False}}
    INFO 2024-10-23 10:14:05,889 you see the whole of my life
    INFO 2024-10-23 10:14:05,966 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 12, 'result': {'hypotheses': [{'transcript': 'you see the whole of my life'}], 'final': False}}
    INFO 2024-10-23 10:14:05,967 you see the whole of my life
    INFO 2024-10-23 10:14:06,052 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 13, 'result': {'hypotheses': [{'transcript': 'you see the whole of my life'}], 'final': False}}
    INFO 2024-10-23 10:14:06,053 you see the whole of my life
Sending block of data 8000
    INFO 2024-10-23 10:14:06,133 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 14, 'result': {'hypotheses': [{'transcript': 'you see the whole of my life'}], 'final': False}}
    INFO 2024-10-23 10:14:06,133 you see the whole of my life
    INFO 2024-10-23 10:14:06,215 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 15, 'result': {'hypotheses': [{'transcript': 'you see the whole of my life'}], 'final': False}}
    INFO 2024-10-23 10:14:06,215 you see the whole of my life
Sending block of data 8000
    INFO 2024-10-23 10:14:06,380 {'status': 0, 'id': '1a226a8f-7c78-41c0-8766-6db0fd261a87', 'segment': 16, 'result': {'hypotheses': [{'transcript': 'you see the whole of my life'}], 'final': False}}
    INFO 2024-10-23 10:14:06,380 you see the whole of my life
```

Figure 5-6: Live transcription with SpeechLab ASR Application

All in all, this deployment process took no longer than 20 minutes. Using `cloudkube`, developers are able to quickly spin up a Kubernetes cluster in the cloud and test their application in a scalable and isolated environment.

5.3 Deployment of Multi Service Dummy Application

Next, we will look at the deployment of a multi-service, dummy application that was created for the purposes of testing the various functionality within the Kubernetes cluster provisioned by `cloudkube`. The architecture diagram of this application is shown in Figure 5-7. Traffic from the public internet is routed through an ingress, which directs traffic between the relevant pods for servicing. The roles and responsibilities of each Kubernetes component is shown in Table 5-3.

To successfully deploy this application, we need to provision an ingress controller to satisfy the ingress Kubernetes resource. Furthermore, this application also runs on arm, and the architecture of the cluster must be changed. To do this, we simply edit the `config.json` file to the following. As we can see, an nginx ingress controller has been chosen, and the architecture has changed to `arm`.

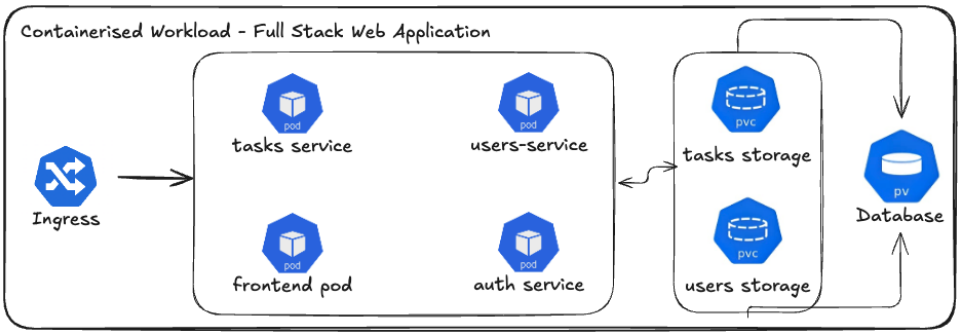

Figure 5-7: Multi Service Tasks Application Architecture Diagram

Name	Kubernetes Component	Responsibility
-	`Ingress`	Routing rules for routing traffic to the various pods
`tasks service`	`Pod`	Creation and Deletion of Tasks
`users service`	`Pod`	Registering and Logging in Users
`frontend pod`	`Pod`	UI layer for interaction with the application
`auth service`	`Pod`	Dummy authentication service to verify tokens during login
`tasks storage`	`PersistentVolumeClaim`	Claims a set amount of storage from the PersistentVolume `Database` for storing of tasks
`users storage`	`PersistentVolumeClaim`	Claims a set amount of storage from the PersistentVolume `Database` for storing of users
`Database`	`PersistentVolume`	Provides persistence of data across pod restarts

Table 5-3: Responsibilities of each component in tasks application

```
{
    "region": "ap-southeast-1",
    "prefix": "mh-dep-test",
    "architecture": "AL2_ARM_64",
    "ingressController": "aws-nginx-ingress-controller",
    "requireEFS": "True"
}
```

Listing 5.1: `config.json` file for Tasks dummy application

Next, we run option 3 of `cloudkube init` to provision the resources. As seen in Figure 5-8, the cluster is modified accordingly. Note that only resources that need modification are changed. Other resources are left alone to avoid redundant re-creation. Once again, the infrastructure provisioning happens in less than 10 minutes, and the Kubernetes manifest of this application can now be applied to the cluster.

```
Plan: 36 to add, 15 to change, 36 to destroy.

Changes to Outputs:
  ~ cluster_endpoint          = "https://F941A3CA419A0ED6B0BEB1279DF34F2F.gr7.ap-southeast-1.eks.amazonaws.com" -> (known after apply)
  ~ cluster_name              = "mh-deploy-test-eks-NgXDPuyn" -> "mh-dep-test-eks-NgXDPuyn"
  ~ cluster_security_group_id = "sg-0dc7ed32a88a1eadf" -> (known after apply)
  ~ efs_id                    = "fs-08638c65edca975c3" -> (known after apply)
```

Figure 5-8: Terraform output for provisioning with `cloudkube`

Once the Kubernetes manifest is applied to the cluster, we can then access the frontend of our application via the ingress provisioned. The address of this ingress can be output by doing `kubectl get ingress`. We can test the functionality of the application through this frontend interface, as well as through postman.

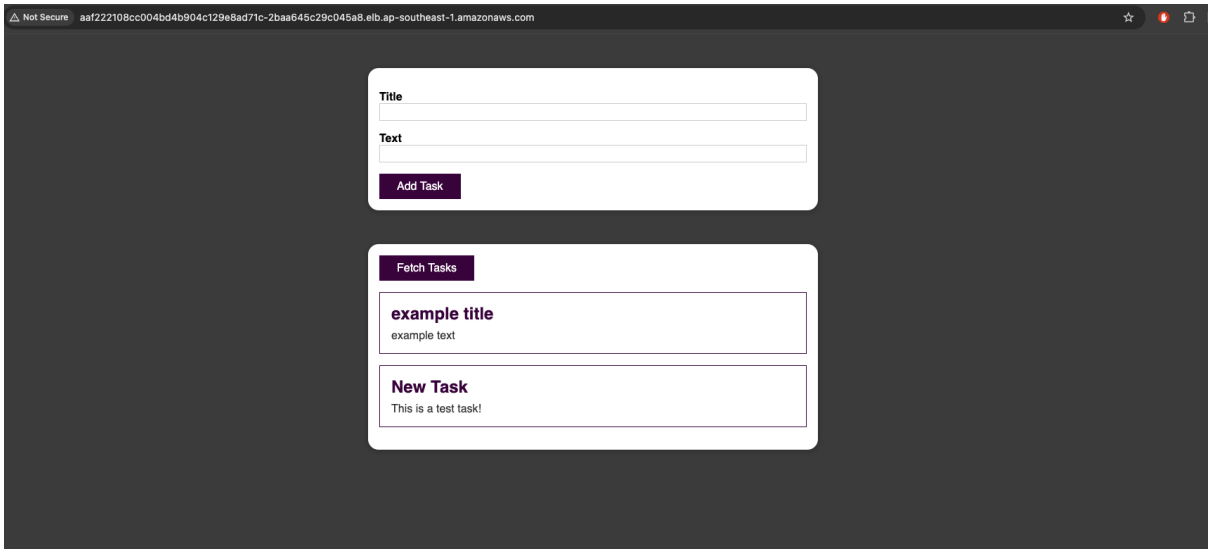

Figure 5-9: Successful creation of tasks

From Figure 5-9, we can see that we are able to create new tasks. Inspecting the nginx ingress controller logs also shows a rerouting to the various services based on the Kubernetes Ingress component. Finally, in order to test that the PersistentVolume setup is working, the pods are intentionally killed. With PersistentVolumes, the tasks persist across pod lifecycles, confirming functionality of the PersistentVolumes which are realised by EFS under the hood of `cloudkube`.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

As supported by our results, `cloudkube` significantly accelerates development time in Kubernetes by simplifying cluster provisioning, deployment, and management. With its streamlined interface and automated setup, developers are able to rapidly spin up fully functional Kubernetes clusters without worrying about complex configurations. This reduces the overhead acssociated with infrastructure management, allowing teams to focus on writing, testing, and scaling their applications. Through providing a seamless and efficient development worklow, `cloudkube` empowers teams to iterate faster, deploy more reliably, and bring applications to market with greater agility and velocity.

6.2 Future Work

6.2.1 Additional Support for Add-ons

`cloudkube` is built generically to support the SpeechLab ASR Application. A multi-service application was also tested here. However, more complex setup might be required based on different types of application requirements. In future, commands can be added to the commands module of `cloudkube` to allow for the addition of more niche Kubernetes add-ons such as Pod Identity Agents, or drivers for run time monitoring.

6.2.2 Multi-Cloud Support

While `cloudkube` is built to be extensible to different cloud providers, the current version of `cloudkube` only supports AWS cloud due to limited time and resources for the completion of this FYP. In future, `cloudkube` can be extended to other cloud providers to provide a just as simple set up of a Kuberenetes cluster in a different cloud provider.

6.2.3 CI/CD Integrations

Currently, `cloudkube` is built to be run locally on a machine. In future, `cloudkube` can be integrated into CI/CD pipelines to automate infrastructure mutation and provisioning.

References

[1] *"What is Docker?"*. Sept. 2024. URL: https://docs.docker.com/get-started/docker-overview/.

[2] AWS. *Auto Scaling groups - Amazon EC2 Auto Scaling*. URL: https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-groups.html.

[3] AWS. *Control traffic to your AWS resources using security groups - Amazon Virtual Private Cloud*. URL: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html#:~:text=A%20security%20group%20controls%20the,with%20a%20default%20security%20group..

[4] AWS. *Elastic network interfaces - Amazon Elastic Compute Cloud*. URL: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-eni.html.

[5] AWS. *What is Amazon Elastic Container Service? - Amazon Elastic Container Service*. URL: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html.

[6] AWS. *What is AWS CloudFormation? - AWS CloudFormation*. URL: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html.

[7] AWS. *What is File Storage? - Cloud File Sharing and Storage Explained - AWS*. URL: https://aws.amazon.com/what-is/cloud-file-storage/.

[8] AWS. *What is Infrastructure as Code? - IaC Explained - AWS*. URL: https://aws.amazon.com/what-is/iac/.

[9] Yash Bindlish. "Under the Hood: An Introduction to Kubernetes architecture". In: (Dec. 2021). URL: https://yashbindlish.medium.com/under-the-hood-an-introduction-to-kubernetes-architecture-bb9d8599f837.

[10] Benjamin Chen. *Docker and Kubernetes: Deploying Speech Recognition System*. Tech. rep. Dec. 2021.

[11] Mike Tyson Of The Cloud. "Terraform Startup Guide: A comprehensive whitepaper for startups". In: (Aug. 2023). URL: https://blog.brainboard.co/terraform-startup-guide-a-comprehensive-whitepaper-for-startups-1d58e0155d42.

[12] Cloudflare. *What is a subnet? — How subnetting works*. URL: https://www.cloudflare.com/en-gb/learning/network-layer/what-is-a-subnet/.

[13] Ivan Cvitkovic. *Introduction to Terraform*. Feb. 2022. URL: https://dev.to/cvitaa11/introduction-to-terraform-2hfe.

[14] Deepeshjaiswal. "Setting Up Terraform with S3 Backend and DynamoDB Locking". In: (Nov. 2024). URL: https://medium.com/@deepeshjaiswal6734/setting-up-terraform-with-s3-backend-and-dynamodb-locking-1e4b69e0b3cd.

[15] Flavius Dinu. *16 Most useful Infrastructure as Code (IAC) tools for 2025*. Feb. 2025. URL: https://spacelift.io/blog/infrastructure-as-code-tools.

[16] Flavius Dinu. *16 Most useful Infrastructure as Code (IAC) tools for 2025*. Feb. 2025. URL: https://spacelift.io/blog/infrastructure-as-code-tools.

[17] Flavius Dinu. *OpenTofu vs Terraform : Key Differences and Comparison*. Aug. 2024. URL: https://spacelift.io/blog/opentofu-vs-terraform.

[18] Flavius Dinu and Jack Roper. *What is a Kubernetes cluster? Key components explained*. Feb. 2025. URL: https://spacelift.io/blog/kubernetes-cluster.

[19] Facebook. *Create React App*. 2022. URL: https://create-react-app.dev/.

[20] Erich Gamma et al. *Design Patterns*. Addison-Wesley, Mar. 2009.

[21] *Gartner's 2024 Perspective on Container Management*. URL: https://www.perfectscale.io/blog/gartners-2024-perspective-on-container-management.

[22] GCP. *Instance templates*. URL: https://cloud.google.com/compute/docs/instance-templates#:~:text=An%20instance%20template%20is%20a,managed%20instance%20group%20(MIG)..

[23] Google. *What is Kubernetes? — Google Cloud*. URL: https://cloud.google.com/learn/what-is-kubernetes?hl=en.

[24] Hashicorp. *Basic CLI features — Terraform — HashiCorp Developer*. URL: https://developer.hashicorp.com/terraform/cli/commands.

[25] Red Hat. *Ansible Collaborative - How Ansible Works*. URL: https://www.redhat.com/en/ansible-collaborative/how-ansible-works.

[26] Ibm. *ETCD*. Jan. 2025. URL: https://www.ibm.com/think/topics/etcd#:~:text=etcd%20is%20an%20open%20source,the%20popular%20container%20orchestration%20platform..

[27] Ibm, Stephanie Susnjara, and Iam Smalley. *VPC*. Dec. 2024. URL: https://www.ibm.com/think/topics/vpc.

[28] Siddhant Khisty. *Understanding kubernetes nodes*. Mar. 2025. URL: https://devtron.ai/blog/understanding-kubernetes-nodes/.

[29] Kubernetes. *Overview*. URL: https://kubernetes.io/docs/concepts/overview/.

[30] Kai Shern Lee. *Deploying ASR System for Scalability and Robustness on AWS*. Tech. rep. May 2022.

[31] Xiao Ma. *Docker and Kubernetes: Deploying Speech Recognition System for Scalability*. Tech. rep. May 2021.

[32] Mumshad Mannambeth. *Kubernetes features every beginner must know*. July 2024. URL: https://kodekloud.com/blog/kubernetes-features-for-beginner/.

[33] *minikube*. URL: https://minikube.sigs.k8s.io/docs/.

[34] Tung Nguyen. *Terraform vs Terragrunt vs Terraspace - BoltOps Blog*. Sept. 2020. URL: https://blog.boltops.com/2020/09/28/terraform-vs-terragrunt-vs-terraspace/.

[35] Tung Nguyen. *Terraspace vs Terragrunt - Terraspace*. URL: https://terraspace.cloud/docs/vs/terragrunt/.

[36] Sumeet Ninawe. *Terraform vs. Ansible: Key Differences and Comparison of Tools*. Jan. 2025. URL: https://spacelift.io/blog/ansible-vs-terraform.

[37] Nathan Peck. *EC2 or AWS Fargate?* June 2023. URL: https://containersonaws.com/blog/2023/ec2-or-aws-fargate/.

[38] Kai Kiat Poh. *GitOps in Kubernetes Clusters*. Tech. rep. Dec. 2022.

[39] Tjandy Putra. *Deploying Automatic Speech Recognition System for Scalability, Reliability, and Security with Kubernetes*. Tech. rep. Dec. 2023.

[40] Quip. *KUBERNETES: FEATURES AND BENEFITS - Criticalcase*. May 2021. URL: https://www.criticalcase.com/blog/kubernetes-features-and-benefits.html.

[41] Rgs. *Cloud Computing: Trends toward Containerization*. Nov. 2023. URL: https://rancher government.com/news/cloud-computing-trends-toward-containerization.

[42] Chris Spitzenberger and Flavius Dinu. *Terraform vs. AWS CloudFormation - Ultimate Comparison*. Oct. 2024. URL: https://spacelift.io/blog/terraform-vs-cloudformation.

[43] R Sujatha. *Managed Kubernetes Services: Comparing the best options for containerized applications — DigitalOcean*. URL: https://www.digitalocean.com/resources/articles/managed-kubernetes-services.

[44] Ozan Unlu. *Latest Kubernetes Adoption Statistics: Global Insights and analysis for 2024*. Dec. 2024. URL: https://edgedelta.com/company/blog/kubernetes-adoption-statistics.

[45] VMWare. *What is a Virtual Machine?* URL: https://www.vmware.com/topics/virtual-machine.

[46] Jonathan Zher Hao Yap. *On-premise Cloud Deployment of ASR System*. Tech. rep. May 2023.

[47] Song Yu. *Deploying speech recognition system using Kubernetes cluster infrastructure as code with terraform and terragrunt*. Tech. rep. 2023.

Appendix A: AWS CloudFormation YAML Template for EKS IPv4

The following is the YAML template used for provisioning the infrastructure for an EKS compliant VPC (IPv4):

```
---
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Amazon EKS Sample VPC - Private and Public subnets'

Parameters:

  VpcBlock:
    Type: String
    Default: 192.168.0.0/16
    Description: The CIDR range for the VPC. This should be a valid private (RFC
        1918) CIDR range.

  PublicSubnet01Block:
    Type: String
    Default: 192.168.0.0/18
    Description: CidrBlock for public subnet 01 within the VPC

  PublicSubnet02Block:
    Type: String
    Default: 192.168.64.0/18
    Description: CidrBlock for public subnet 02 within the VPC

  PrivateSubnet01Block:
    Type: String
    Default: 192.168.128.0/18
    Description: CidrBlock for private subnet 01 within the VPC

  PrivateSubnet02Block:
    Type: String
    Default: 192.168.192.0/18
    Description: CidrBlock for private subnet 02 within the VPC

Metadata:
  AWS::CloudFormation::Interface:
    ParameterGroups:
      -
        Label:
          default: "Worker Network Configuration"
        Parameters:
          - VpcBlock
          - PublicSubnet01Block
          - PublicSubnet02Block
          - PrivateSubnet01Block
          - PrivateSubnet02Block

Resources:
  VPC:
    Type: AWS::EC2::VPC
    Properties:
```

```
49          CidrBlock:  !Ref VpcBlock
50          EnableDnsSupport: true
51          EnableDnsHostnames: true
52          Tags:
53          - Key: Name
54            Value: !Sub '${AWS::StackName}-VPC'
55
56        InternetGateway:
57          Type: "AWS::EC2::InternetGateway"
58
59        VPCGatewayAttachment:
60          Type: "AWS::EC2::VPCGatewayAttachment"
61          Properties:
62            InternetGatewayId: !Ref InternetGateway
63            VpcId: !Ref VPC
64
65        PublicRouteTable:
66          Type: AWS::EC2::RouteTable
67          Properties:
68            VpcId: !Ref VPC
69            Tags:
70            - Key: Name
71              Value: Public Subnets
72            - Key: Network
73              Value: Public
74
75        PrivateRouteTable01:
76          Type: AWS::EC2::RouteTable
77          Properties:
78            VpcId: !Ref VPC
79            Tags:
80            - Key: Name
81              Value: Private Subnet AZ1
82            - Key: Network
83              Value: Private01
84
85        PrivateRouteTable02:
86          Type: AWS::EC2::RouteTable
87          Properties:
88            VpcId: !Ref VPC
89            Tags:
90            - Key: Name
91              Value: Private Subnet AZ2
92            - Key: Network
93              Value: Private02
94
95        PublicRoute:
96          DependsOn: VPCGatewayAttachment
97          Type: AWS::EC2::Route
98          Properties:
99            RouteTableId: !Ref PublicRouteTable
100           DestinationCidrBlock: 0.0.0.0/0
101           GatewayId: !Ref InternetGateway
102
103       PrivateRoute01:
104         DependsOn:
105         - VPCGatewayAttachment
106         - NatGateway01
107         Type: AWS::EC2::Route
108         Properties:
109           RouteTableId: !Ref PrivateRouteTable01
110           DestinationCidrBlock: 0.0.0.0/0
111           NatGatewayId: !Ref NatGateway01
```

```
112
113       PrivateRoute02:
114         DependsOn:
115         - VPCGatewayAttachment
116         - NatGateway02
117         Type: AWS::EC2::Route
118         Properties:
119           RouteTableId: !Ref PrivateRouteTable02
120           DestinationCidrBlock: 0.0.0.0/0
121           NatGatewayId: !Ref NatGateway02
122
123       NatGateway01:
124         DependsOn:
125         - NatGatewayEIP1
126         - PublicSubnet01
127         - VPCGatewayAttachment
128         Type: AWS::EC2::NatGateway
129         Properties:
130           AllocationId: !GetAtt 'NatGatewayEIP1.AllocationId'
131           SubnetId: !Ref PublicSubnet01
132           Tags:
133           - Key: Name
134             Value: !Sub '${AWS::StackName}-NatGatewayAZ1'
135
136       NatGateway02:
137         DependsOn:
138         - NatGatewayEIP2
139         - PublicSubnet02
140         - VPCGatewayAttachment
141         Type: AWS::EC2::NatGateway
142         Properties:
143           AllocationId: !GetAtt 'NatGatewayEIP2.AllocationId'
144           SubnetId: !Ref PublicSubnet02
145           Tags:
146           - Key: Name
147             Value: !Sub '${AWS::StackName}-NatGatewayAZ2'
148
149       NatGatewayEIP1:
150         DependsOn:
151         - VPCGatewayAttachment
152         Type: 'AWS::EC2::EIP'
153         Properties:
154           Domain: vpc
155
156       NatGatewayEIP2:
157         DependsOn:
158         - VPCGatewayAttachment
159         Type: 'AWS::EC2::EIP'
160         Properties:
161           Domain: vpc
162
163       PublicSubnet01:
164         Type: AWS::EC2::Subnet
165         Metadata:
166           Comment: Subnet 01
167         Properties:
168           MapPublicIpOnLaunch: true
169           AvailabilityZone:
170             Fn::Select:
171             - '0'
172             - Fn::GetAZs:
173                 Ref: AWS::Region
174           CidrBlock:
```

```
175             Ref: PublicSubnet01Block
176           VpcId:
177             Ref: VPC
178           Tags:
179           - Key: Name
180             Value: !Sub "${AWS::StackName}-PublicSubnet01"
181           - Key: kubernetes.io/role/elb
182             Value: 1
183
184       PublicSubnet02:
185         Type: AWS::EC2::Subnet
186         Metadata:
187           Comment: Subnet 02
188         Properties:
189           MapPublicIpOnLaunch: true
190           AvailabilityZone:
191             Fn::Select:
192             - '1'
193             - Fn::GetAZs:
194                 Ref: AWS::Region
195           CidrBlock:
196             Ref: PublicSubnet02Block
197           VpcId:
198             Ref: VPC
199           Tags:
200           - Key: Name
201             Value: !Sub "${AWS::StackName}-PublicSubnet02"
202           - Key: kubernetes.io/role/elb
203             Value: 1
204
205       PrivateSubnet01:
206         Type: AWS::EC2::Subnet
207         Metadata:
208           Comment: Subnet 03
209         Properties:
210           AvailabilityZone:
211             Fn::Select:
212             - '0'
213             - Fn::GetAZs:
214                 Ref: AWS::Region
215           CidrBlock:
216             Ref: PrivateSubnet01Block
217           VpcId:
218             Ref: VPC
219           Tags:
220           - Key: Name
221             Value: !Sub "${AWS::StackName}-PrivateSubnet01"
222           - Key: kubernetes.io/role/internal-elb
223             Value: 1
224
225       PrivateSubnet02:
226         Type: AWS::EC2::Subnet
227         Metadata:
228           Comment: Private Subnet 02
229         Properties:
230           AvailabilityZone:
231             Fn::Select:
232             - '1'
233             - Fn::GetAZs:
234                 Ref: AWS::Region
235           CidrBlock:
236             Ref: PrivateSubnet02Block
237           VpcId:
```

```
            Ref: VPC
          Tags:
          - Key: Name
            Value: !Sub "${AWS::StackName}-PrivateSubnet02"
          - Key: kubernetes.io/role/internal-elb
            Value: 1

      PublicSubnet01RouteTableAssociation:
        Type: AWS::EC2::SubnetRouteTableAssociation
        Properties:
          SubnetId: !Ref PublicSubnet01
          RouteTableId: !Ref PublicRouteTable

      PublicSubnet02RouteTableAssociation:
        Type: AWS::EC2::SubnetRouteTableAssociation
        Properties:
          SubnetId: !Ref PublicSubnet02
          RouteTableId: !Ref PublicRouteTable

      PrivateSubnet01RouteTableAssociation:
        Type: AWS::EC2::SubnetRouteTableAssociation
        Properties:
          SubnetId: !Ref PrivateSubnet01
          RouteTableId: !Ref PrivateRouteTable01

      PrivateSubnet02RouteTableAssociation:
        Type: AWS::EC2::SubnetRouteTableAssociation
        Properties:
          SubnetId: !Ref PrivateSubnet02
          RouteTableId: !Ref PrivateRouteTable02

      ControlPlaneSecurityGroup:
        Type: AWS::EC2::SecurityGroup
        Properties:
          GroupDescription: Cluster communication with worker nodes
          VpcId: !Ref VPC

    Outputs:

      SubnetIds:
        Description: Subnets IDs in the VPC
        Value: !Join [ ",", [ !Ref PublicSubnet01, !Ref PublicSubnet02, !Ref
            PrivateSubnet01, !Ref PrivateSubnet02 ] ]

      SecurityGroups:
        Description: Security group for the cluster control plane communication with
            worker nodes
        Value: !Join [ ",", [ !Ref ControlPlaneSecurityGroup ] ]

      VpcId:
        Description: The VPC Id
        Value: !Ref VPC
```

Listing A.1: AWS CloudFormation YAML Template for EKS IPv4

Appendix B: VPC Requirements for EKS Cluster in AWS

The following are the VPC requirements for deploying an EKS Cluster in AWS.

1. The VPC must have sufficient IP addresses for the cluster, any nodes, and Kubernetes resources that are created.
2. If IPv6 is used by Kubernetes, then IPv6 CIDR blocks must be associated with the VPC and corresponding subnets.
3. The VPC must have DNS hostname and DNS resolution support.
4. The subnets must have at least six IP addresses for use by EKS.
5. The subnets must be in at least two AZs.
6. If inbound access is required from internet to pods, then there needs to be at least 1 public subnet with enough available IP addresses to deploy load balancers and ingresses to.

Appendix C: AWS EKS Add-ons

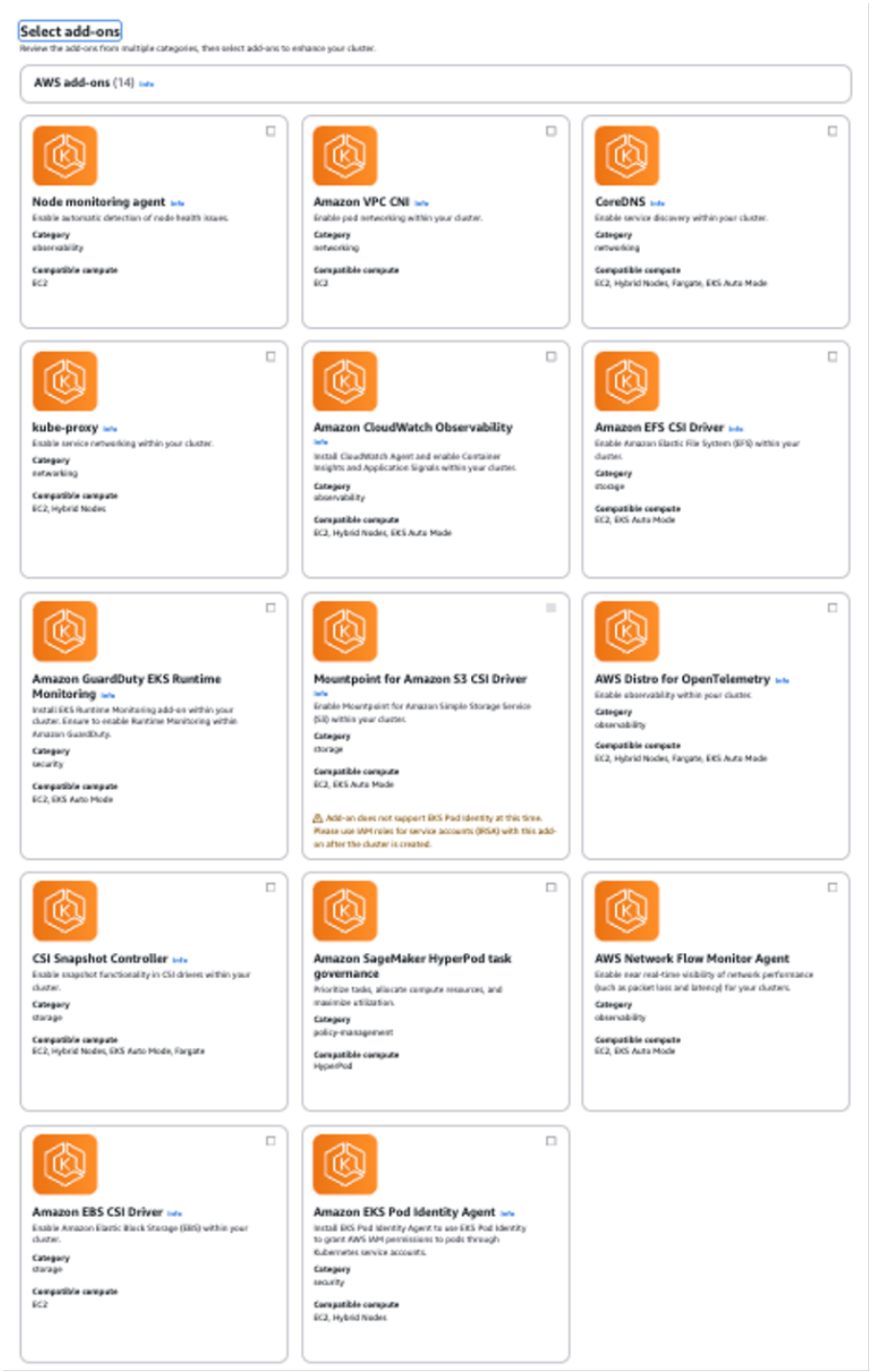

Figure C-1: AWS EKS Optional Add-ons

Appendix C: `cloudkube` README

`cloudkube` Developer Documentation

Initial Setup

The initial setup for `cloudkube` is straightforward and needs to be done only once. Since `cloudkube` runs on Python (`>= 3.11`), it is recommended to use virtual environments to avoid dependency conflicts and keep your global environment clean. Virtual environments can be created using **conda** (recommended), **pipenv**, or **python venv**.

1. **Create and Activate a Virtual Environment:** Use your preferred method (e.g., conda, pipenv, or venv) to create and activate a virtual environment.
2. **Install `cloudkube`:** Run the following command to install `cloudkube` via `pip`.

   ```
   pip install cloudkube
   ```

3. **Verify Installation:** Check if `cloudkube` was installed successfully.

   ```
   pip list | grep cloudkube
   ```

4. **Install AWS CLI:** Follow the official AWS documentation to install the AWS CLI: Getting Started with AWS CLI and configure your cloud credentials.

   ```
   aws configure
   ```

   For additional details, refer to the AWS CLI Welcome Guide.

5. **Install Terraform:**

   - **macOS (Recommended):** Install Terraform using Homebrew:

     ```
     brew tap hashicorp/tap
     brew install hashicorp/tap/terraform
     brew update
     brew upgrade hashicorp/tap/terraform
     ```

   - **Other Platforms:** Terraform can also be installed on other platformrs by following the Terraform Developer Documentation.

6. **Verify Terraform Installation:**

   ```
   terraform -version
   ```

Once all dependencies are installed, you are ready to start using `cloudkube`!

`cloudkube` **Workflow**

1. Initialize `cloudkube`

To start working with cloudkube, run `cloudkube init`. This will bring up an interactive wizard, which will bring you through the provisioning of a cluster.

```
cloudkube init
```

2. Configure Cluster Configuration

a. In the wizard, select **Option 2** to configure the cluster.

b. Provide the following information when prompted:

- Name of the cluster
- Region
- Compute architecture
- Optional add-ons (e.g., persistent volumes, ingress controllers)

c. The configuration details will be saved in a file named `config.json` in the same directory where `cloudkube` is run.

3. Spin Up the Cluster

a. Select **Option 3** in the interactive wizard.

b. `cloudkube` performs the following actions:

- `terraform init`: Installs all required Terraform dependencies and sets up the environment.
- `terraform plan`: Creates a resource plan and seeks user approval before proceeding.
- `terraform apply`: Provisions the required infrastructure, including:
  - An EKS-compliant VPC
  - Relevant permissions and security groups
- Automatically configures the local environment to communicate with the cluster by populating `~/.kube/config`.

4. Verify Cluster Creation

a. Check the cluster creation via the **AWS Console**.

b. Use the following `kubectl` commands for further verification:

- List all pods:

```
kubectl get pods -A
```

- List all nodes:

```
kubectl get nodes
```

5. Apply Kubernetes Manifests

To deploy your application:

a. Ensure your Kubernetes YAML files are stored in the `./kubernetes` directory.

b. Apply the manifests:

```
kubectl apply -f ./kubernetes
```

6. Interact with your Kubernetes Cluster

At this point, we have a fully functional Kubernetes cluster with your deployed application. You should be able to use all kubectl commands to interact with your application. For a list of commands to interact with the cluster, please refer to the Kubernetes Documentation.

7. Teardown and Resource Deprovisioning

Once you are done testing your application:

a. Remove your Kubernetes deployment.

```
kubectl delete -f ./kubernetes
```

b. Run the following command and select **Option 4** in the wizard to deprovision all resources and clean up your local Kubernetes configuration.

```
cloudkube init
```

Notes

- As of version `v1.0`, `cloudkube` only supports **AWS Cloud**. If support for other cloud providers is added in the future, their respective setup steps should be followed.

Appendix D: terraform plan output for `cloudkube` provisioning

The following refers to a part of the output after selection Option 3 in the `cloudkube` CLI. This is the output redirected from `stdout` of `terraform plan`.

```
module.eks.module.eks_managed_node_group["one"].data.aws_partition.current: Reading...
module.eks.data.aws_iam_policy_document.assume_role_policy[0]: Reading...
module.eks.data.aws_caller_identity.current: Reading...
module.eks.data.aws_partition.current: Reading...
module.eks.module.eks_managed_node_group["one"].data.aws_caller_identity.current: Reading...
module.eks.module.kms.data.aws_partition.current[0]: Reading...
module.eks.module.eks_managed_node_group["one"].data.aws_iam_policy_document.assume_role_policy[0]: Reading...
module.eks.module.kms.data.aws_caller_identity.current[0]: Reading...
module.eks.module.kms.data.aws_partition.current[0]: Read complete after 0s [id=aws]
module.eks.module.eks_managed_node_group["one"].data.aws_partition.current: Read complete after 0s [id=aws]
module.eks.module.eks_managed_node_group["one"].data.aws_iam_policy_document.assume_role_policy[0]: Read complete after 0s [id=2560088296]
module.eks.data.aws_partition.current: Read complete after 0s [id=aws]
module.eks.data.aws_iam_policy_document.assume_role_policy[0]: Read complete after 0s [id=2764486067]
module.eks.data.aws_caller_identity.current: Read complete after 0s [id=861814105207]
module.eks.data.aws_iam_session_context.current: Reading...
module.eks.data.aws_iam_session_context.current: Read complete after 0s [id=arn:aws:iam::861814105207:user/chanming]
module.eks.module.kms.data.aws_caller_identity.current[0]: Read complete after 0s [id=861814105207]
module.eks.module.eks_managed_node_group["one"].data.aws_caller_identity.current: Read complete after 0s [id=861814105207]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create
 <= read (data resources)

Terraform will perform the following actions:

  # aws_efs_backup_policy.policy will be created
  + resource "aws_efs_backup_policy" "policy" {
      + file_system_id = (known after apply)
      + id             = (known after apply)

      + backup_policy {
          + status = "ENABLED"
        }
    }

  # aws_efs_file_system.efs will be created
  + resource "aws_efs_file_system" "efs" {
```

```
      + arn                      = (known after apply)
      + availability_zone_id     = (known after apply)
      + availability_zone_name   = (known after apply)
      + creation_token           = "minghan-eks-cluster-efs-filesystem"
      + dns_name                 = (known after apply)
      + encrypted                = true
      + id                       = (known after apply)
      + kms_key_id               = (known after apply)
      + name                     = (known after apply)
      + number_of_mount_targets = (known after apply)
      + owner_id                 = (known after apply)
      + performance_mode         = (known after apply)
      + size_in_bytes            = (known after apply)
      + tags_all                 = (known after apply)
      + throughput_mode          = "elastic"

      + lifecycle_policy {
          + transition_to_ia = "AFTER_30_DAYS"
        }

      + protection (known after apply)
    }

  # aws_efs_mount_target.efs_mount_target_1 will be created
  + resource "aws_efs_mount_target" "efs_mount_target_1" {
      + availability_zone_id   = (known after apply)
      + availability_zone_name = (known after apply)
      + dns_name               = (known after apply)
      + file_system_arn        = (known after apply)
      + file_system_id         = (known after apply)
      + id                     = (known after apply)
      + ip_address             = (known after apply)
      + mount_target_dns_name  = (known after apply)
      + network_interface_id   = (known after apply)
      + owner_id               = (known after apply)
      + security_groups        = (known after apply)
      + subnet_id              = (known after apply)
    }

  # aws_efs_mount_target.efs_mount_target_2 will be created
  + resource "aws_efs_mount_target" "efs_mount_target_2" {
      + availability_zone_id   = (known after apply)
      + availability_zone_name = (known after apply)
      + dns_name               = (known after apply)
      + file_system_arn        = (known after apply)
      + file_system_id         = (known after apply)
      + id                     = (known after apply)
      + ip_address             = (known after apply)
      + mount_target_dns_name  = (known after apply)
      + network_interface_id   = (known after apply)
      + owner_id               = (known after apply)
      + security_groups        = (known after apply)
      + subnet_id              = (known after apply)
    }

  # aws_security_group.efs_security_group will be created
  + resource "aws_security_group" "efs_security_group" {
      + arn                    = (known after apply)
      + description            = "Security group for EFS with NFS access"
      + egress                 = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
```

```
                    ]
                + from_port        = 0
                + ipv6_cidr_blocks = []
                + prefix_list_ids  = []
                + protocol         = "-1"
                + security_groups  = []
                + self             = false
                + to_port          = 0
                  # (1 unchanged attribute hidden)
              },
          ]
        + id                     = (known after apply)
        + ingress                = [
            + {
                + cidr_blocks      = [
                    + "10.0.0.0/16",
                  ]
                + description      = "Allow NFS access"
                + from_port        = 2049
                + ipv6_cidr_blocks = []
                + prefix_list_ids  = []
                + protocol         = "tcp"
                + security_groups  = []
                + self             = false
                + to_port          = 2049
              },
          ]
        + name                   = "minghan-eks-cluster-efs"
        + name_prefix            = (known after apply)
        + owner_id               = (known after apply)
        + revoke_rules_on_delete = false
        + tags                   = {
            + "Name" = "minghan-eks-cluster-efs-nfs-sg"
          }
        + tags_all               = {
            + "Name" = "minghan-eks-cluster-efs-nfs-sg"
          }
        + vpc_id                 = (known after apply)
      }

    # random_string.suffix will be created
    + resource "random_string" "suffix" {
        + id          = (known after apply)
        + length      = 8
        + lower       = true
        + min_lower   = 0
        + min_numeric = 0
        + min_special = 0
        + min_upper   = 0
        + number      = true
        + numeric     = true
        + result      = (known after apply)
        + special     = false
        + upper       = true
      }

    # module.eks.data.aws_eks_addon_version.this["coredns"] will be read during
        apply
    # (depends on a resource or a module with changes pending)
   <= data "aws_eks_addon_version" "this" {
        + addon_name         = "coredns"
        + id                 = (known after apply)
        + kubernetes_version = "1.30"
```

```
      + version            = (known after apply)
    }

 # module.eks.data.aws_eks_addon_version.this["eks-pod-identity-agent"] will be read during apply
 # (depends on a resource or a module with changes pending)
<= data "aws_eks_addon_version" "this" {
      + addon_name         = "eks-pod-identity-agent"
      + id                 = (known after apply)
      + kubernetes_version = "1.30"
      + version            = (known after apply)
    }

 # module.eks.data.aws_eks_addon_version.this["kube-proxy"] will be read during apply
 # (depends on a resource or a module with changes pending)
<= data "aws_eks_addon_version" "this" {
      + addon_name         = "kube-proxy"
      + id                 = (known after apply)
      + kubernetes_version = "1.30"
      + version            = (known after apply)
    }

 # module.eks.data.aws_eks_addon_version.this["vpc-cni"] will be read during apply
 # (depends on a resource or a module with changes pending)
<= data "aws_eks_addon_version" "this" {
      + addon_name         = "vpc-cni"
      + id                 = (known after apply)
      + kubernetes_version = "1.30"
      + version            = (known after apply)
    }

 # module.eks.data.tls_certificate.this[0] will be read during apply
 # (config refers to values not yet known)
<= data "tls_certificate" "this" {
      + certificates = (known after apply)
      + id           = (known after apply)
      + url          = (known after apply)
    }

 # module.eks.aws_cloudwatch_log_group.this[0] will be created
 + resource "aws_cloudwatch_log_group" "this" {
      + arn               = (known after apply)
      + id                = (known after apply)
      + log_group_class   = (known after apply)
      + name              = (known after apply)
      + name_prefix       = (known after apply)
      + retention_in_days = 90
      + skip_destroy      = false
      + tags              = (known after apply)
      + tags_all          = (known after apply)
    }

 # module.eks.aws_eks_access_entry.this["cluster_creator"] will be created
 + resource "aws_eks_access_entry" "this" {
      + access_entry_arn  = (known after apply)
      + cluster_name      = (known after apply)
      + created_at        = (known after apply)
      + id                = (known after apply)
      + kubernetes_groups = (known after apply)
      + modified_at       = (known after apply)
      + principal_arn     = "arn:aws:iam::861814105207:user/chanming"
```

```
      + tags_all          = (known after apply)
      + type              = "STANDARD"
      + user_name         = (known after apply)
    }

  # module.eks.aws_eks_access_policy_association.this["cluster_creator_admin"]
      will be created
  + resource "aws_eks_access_policy_association" "this" {
      + associated_at = (known after apply)
      + cluster_name  = (known after apply)
      + id            = (known after apply)
      + modified_at   = (known after apply)
      + policy_arn    = "arn:aws:eks::aws:cluster-access-policy/
          AmazonEKSClusterAdminPolicy"
      + principal_arn = "arn:aws:iam::861814105207:user/chanming"

      + access_scope {
          + type = "cluster"
        }
    }

  # module.eks.aws_eks_addon.this["coredns"] will be created
  + resource "aws_eks_addon" "this" {
      + addon_name                  = "coredns"
      + addon_version               = (known after apply)
      + arn                         = (known after apply)
      + cluster_name                = (known after apply)
      + configuration_values        = (known after apply)
      + created_at                  = (known after apply)
      + id                          = (known after apply)
      + modified_at                 = (known after apply)
      + preserve                    = true
      + resolve_conflicts_on_create = "OVERWRITE"
      + resolve_conflicts_on_update = "OVERWRITE"
      + tags_all                    = (known after apply)

      + timeouts {}
    }

  # module.eks.aws_eks_addon.this["eks-pod-identity-agent"] will be created
  + resource "aws_eks_addon" "this" {
      + addon_name                  = "eks-pod-identity-agent"
      + addon_version               = (known after apply)
      + arn                         = (known after apply)
      + cluster_name                = (known after apply)
      + configuration_values        = (known after apply)
      + created_at                  = (known after apply)
      + id                          = (known after apply)
      + modified_at                 = (known after apply)
      + preserve                    = true
      + resolve_conflicts_on_create = "OVERWRITE"
      + resolve_conflicts_on_update = "OVERWRITE"
      + tags_all                    = (known after apply)

      + timeouts {}
    }

  # module.eks.aws_eks_addon.this["kube-proxy"] will be created
  + resource "aws_eks_addon" "this" {
      + addon_name                  = "kube-proxy"
      + addon_version               = (known after apply)
      + arn                         = (known after apply)
      + cluster_name                = (known after apply)
```

```
      + configuration_values        = (known after apply)
      + created_at                  = (known after apply)
      + id                          = (known after apply)
      + modified_at                 = (known after apply)
      + preserve                    = true
      + resolve_conflicts_on_create = "OVERWRITE"
      + resolve_conflicts_on_update = "OVERWRITE"
      + tags_all                    = (known after apply)

      + timeouts {}
    }

  # module.eks.aws_eks_addon.this["vpc-cni"] will be created
  + resource "aws_eks_addon" "this" {
      + addon_name                  = "vpc-cni"
      + addon_version               = (known after apply)
      + arn                         = (known after apply)
      + cluster_name                = (known after apply)
      + configuration_values        = (known after apply)
      + created_at                  = (known after apply)
      + id                          = (known after apply)
      + modified_at                 = (known after apply)
      + preserve                    = true
      + resolve_conflicts_on_create = "OVERWRITE"
      + resolve_conflicts_on_update = "OVERWRITE"
      + tags_all                    = (known after apply)

      + timeouts {}
    }

  # module.eks.aws_eks_cluster.this[0] will be created
  + resource "aws_eks_cluster" "this" {
      + arn                         = (known after apply)
      + certificate_authority       = (known after apply)
      + cluster_id                  = (known after apply)
      + created_at                  = (known after apply)
      + enabled_cluster_log_types = [
          + "api",
          + "audit",
          + "authenticator",
          + "controllerManager",
          + "scheduler",
        ]
      + endpoint                    = (known after apply)
      + id                          = (known after apply)
      + identity                    = (known after apply)
      + name                        = (known after apply)
      + platform_version            = (known after apply)
      + role_arn                    = (known after apply)
      + status                      = (known after apply)
      + tags                        = {
          + "terraform-aws-modules" = "eks"
        }
      + tags_all                    = {
          + "terraform-aws-modules" = "eks"
        }
      + version                     = "1.30"

      + access_config {
          + authentication_mode                         = "API_AND_CONFIG_MAP"
          + bootstrap_cluster_creator_admin_permissions = false
        }

```

```
347           + encryption_config {
348               + resources = [
349                   + "secrets",
350                 ]
351
352               + provider {
353                   + key_arn = (known after apply)
354                 }
355             }
356
357           + kubernetes_network_config {
358               + ip_family         = "ipv4"
359               + service_ipv4_cidr = (known after apply)
360               + service_ipv6_cidr = (known after apply)
361             }
362
363           + timeouts {}
364
365           + vpc_config {
366               + cluster_security_group_id = (known after apply)
367               + endpoint_private_access   = true
368               + endpoint_public_access    = true
369               + public_access_cidrs       = [
370                   + "0.0.0.0/0",
371                 ]
372               + security_group_ids        = (known after apply)
373               + subnet_ids                = (known after apply)
374               + vpc_id                    = (known after apply)
375             }
376         }
377
378       # module.eks.aws_iam_openid_connect_provider.oidc_provider[0] will be created
379       + resource "aws_iam_openid_connect_provider" "oidc_provider" {
380           + arn             = (known after apply)
381           + client_id_list  = [
382               + "sts.amazonaws.com",
383             ]
384           + id              = (known after apply)
385           + tags            = (known after apply)
386           + tags_all        = (known after apply)
387           + thumbprint_list = (known after apply)
388           + url             = (known after apply)
389         }
390
391       # module.eks.aws_iam_policy.cluster_encryption[0] will be created
392       + resource "aws_iam_policy" "cluster_encryption" {
393           + arn              = (known after apply)
394           + attachment_count = (known after apply)
395           + description      = "Cluster encryption policy to allow cluster role to
                utilize CMK provided"
396           + id               = (known after apply)
397           + name             = (known after apply)
398           + name_prefix      = (known after apply)
399           + path             = "/"
400           + policy           = (known after apply)
401           + policy_id        = (known after apply)
402           + tags_all         = (known after apply)
403         }
404
405       # module.eks.aws_iam_role.this[0] will be created
406       + resource "aws_iam_role" "this" {
407           + arn                   = (known after apply)
408           + assume_role_policy    = jsonencode(
```

```
409                 {
410                   + Statement = [
411                       + {
412                           + Action    = "sts:AssumeRole"
413                           + Effect    = "Allow"
414                           + Principal = {
415                               + Service = "eks.amazonaws.com"
416                             }
417                           + Sid       = "EKSClusterAssumeRole"
418                         },
419                     ]
420                   + Version   = "2012-10-17"
421                 }
422             )
423           + create_date           = (known after apply)
424           + force_detach_policies = true
425           + id                    = (known after apply)
426           + managed_policy_arns   = (known after apply)
427           + max_session_duration  = 3600
428           + name                  = (known after apply)
429           + name_prefix           = (known after apply)
430           + path                  = "/"
431           + tags_all              = (known after apply)
432           + unique_id             = (known after apply)
433
434           + inline_policy {
435               + name   = (known after apply)
436               + policy = jsonencode(
437                     {
438                       + Statement = [
439                           + {
440                               + Action   = [
441                                   + "logs:CreateLogGroup",
442                                 ]
443                               + Effect   = "Deny"
444                               + Resource = "*"
445                             },
446                         ]
447                       + Version   = "2012-10-17"
448                     }
449                 )
450             }
451       }
452
453     # module.eks.aws_iam_role_policy_attachment.cluster_encryption[0] will be
          created
454     + resource "aws_iam_role_policy_attachment" "cluster_encryption" {
455         + id         = (known after apply)
456         + policy_arn = (known after apply)
457         + role       = (known after apply)
458       }
459
460     # module.eks.aws_iam_role_policy_attachment.this["AmazonEKSClusterPolicy"] will
          be created
461     + resource "aws_iam_role_policy_attachment" "this" {
462         + id         = (known after apply)
463         + policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
464         + role       = (known after apply)
465       }
466
467     # module.eks.aws_iam_role_policy_attachment.this["AmazonEKSVPCResourceController
          "] will be created
468     + resource "aws_iam_role_policy_attachment" "this" {
```

```
      + id         = (known after apply)
      + policy_arn = "arn:aws:iam::aws:policy/AmazonEKSVPCResourceController"
      + role       = (known after apply)
    }

  # module.eks.aws_security_group.cluster[0] will be created
  + resource "aws_security_group" "cluster" {
      + arn                    = (known after apply)
      + description            = "EKS cluster security group"
      + egress                 = (known after apply)
      + id                     = (known after apply)
      + ingress                = (known after apply)
      + name                   = (known after apply)
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + revoke_rules_on_delete = false
      + tags                   = (known after apply)
      + tags_all               = (known after apply)
      + vpc_id                 = (known after apply)
    }

  # module.eks.aws_security_group.node[0] will be created
  + resource "aws_security_group" "node" {
      + arn                    = (known after apply)
      + description            = "EKS node shared security group"
      + egress                 = (known after apply)
      + id                     = (known after apply)
      + ingress                = (known after apply)
      + name                   = (known after apply)
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + revoke_rules_on_delete = false
      + tags                   = (known after apply)
      + tags_all               = (known after apply)
      + vpc_id                 = (known after apply)
    }

  # module.eks.aws_security_group_rule.cluster["ingress_nodes_443"] will be
    created
  + resource "aws_security_group_rule" "cluster" {
      + description              = "Node groups to cluster API"
      + from_port                = 443
      + id                       = (known after apply)
      + protocol                 = "tcp"
      + security_group_id        = (known after apply)
      + security_group_rule_id   = (known after apply)
      + self                     = false
      + source_security_group_id = (known after apply)
      + to_port                  = 443
      + type                     = "ingress"
    }

  # module.eks.aws_security_group_rule.node["egress_all"] will be created
  + resource "aws_security_group_rule" "node" {
      + cidr_blocks              = [
          + "0.0.0.0/0",
        ]
      + description              = "Allow all egress"
      + from_port                = 0
      + id                       = (known after apply)
      + prefix_list_ids          = []
      + protocol                 = "-1"
      + security_group_id        = (known after apply)
```

```
      + security_group_rule_id   = (known after apply)
      + self                     = false
      + source_security_group_id = (known after apply)
      + to_port                  = 0
      + type                     = "egress"
    }

  # module.eks.aws_security_group_rule.node["ingress_cluster_443"] will be created
  + resource "aws_security_group_rule" "node" {
      + description              = "Cluster API to node groups"
      + from_port                = 443
      + id                       = (known after apply)
      + prefix_list_ids          = []
      + protocol                 = "tcp"
      + security_group_id        = (known after apply)
      + security_group_rule_id   = (known after apply)
      + self                     = false
      + source_security_group_id = (known after apply)
      + to_port                  = 443
      + type                     = "ingress"
    }

  # module.eks.aws_security_group_rule.node["ingress_cluster_4443_webhook"] will
      be created
  + resource "aws_security_group_rule" "node" {
      + description              = "Cluster API to node 4443/tcp webhook"
      + from_port                = 4443
      + id                       = (known after apply)
      + prefix_list_ids          = []
      + protocol                 = "tcp"
      + security_group_id        = (known after apply)
      + security_group_rule_id   = (known after apply)
      + self                     = false
      + source_security_group_id = (known after apply)
      + to_port                  = 4443
      + type                     = "ingress"
    }

  # module.eks.aws_security_group_rule.node["ingress_cluster_6443_webhook"] will
      be created
  + resource "aws_security_group_rule" "node" {
      + description              = "Cluster API to node 6443/tcp webhook"
      + from_port                = 6443
      + id                       = (known after apply)
      + prefix_list_ids          = []
      + protocol                 = "tcp"
      + security_group_id        = (known after apply)
      + security_group_rule_id   = (known after apply)
      + self                     = false
      + source_security_group_id = (known after apply)
      + to_port                  = 6443
      + type                     = "ingress"
    }

  # module.eks.aws_security_group_rule.node["ingress_cluster_8443_webhook"] will
      be created
  + resource "aws_security_group_rule" "node" {
      + description              = "Cluster API to node 8443/tcp webhook"
      + from_port                = 8443
      + id                       = (known after apply)
      + prefix_list_ids          = []
      + protocol                 = "tcp"
      + security_group_id        = (known after apply)
```

```
      + security_group_rule_id   = (known after apply)
      + self                     = false
      + source_security_group_id = (known after apply)
      + to_port                  = 8443
      + type                     = "ingress"
    }

  # module.eks.aws_security_group_rule.node["ingress_cluster_9443_webhook"] will
      be created
  + resource "aws_security_group_rule" "node" {
      + description              = "Cluster API to node 9443/tcp webhook"
      + from_port                = 9443
      + id                       = (known after apply)
      + prefix_list_ids          = []
      + protocol                 = "tcp"
      + security_group_id        = (known after apply)
      + security_group_rule_id   = (known after apply)
      + self                     = false
      + source_security_group_id = (known after apply)
      + to_port                  = 9443
      + type                     = "ingress"
    }

  # module.eks.aws_security_group_rule.node["ingress_cluster_kubelet"] will be
      created
  + resource "aws_security_group_rule" "node" {
      + description              = "Cluster API to node kubelets"
      + from_port                = 10250
      + id                       = (known after apply)
      + prefix_list_ids          = []
      + protocol                 = "tcp"
      + security_group_id        = (known after apply)
      + security_group_rule_id   = (known after apply)
      + self                     = false
      + source_security_group_id = (known after apply)
      + to_port                  = 10250
      + type                     = "ingress"
    }

  # module.eks.aws_security_group_rule.node["ingress_nodes_ephemeral"] will be
      created
  + resource "aws_security_group_rule" "node" {
      + description              = "Node to node ingress on ephemeral ports"
      + from_port                = 1025
      + id                       = (known after apply)
      + prefix_list_ids          = []
      + protocol                 = "tcp"
      + security_group_id        = (known after apply)
      + security_group_rule_id   = (known after apply)
      + self                     = true
      + source_security_group_id = (known after apply)
      + to_port                  = 65535
      + type                     = "ingress"
    }

  # module.eks.aws_security_group_rule.node["ingress_self_all"] will be created
  + resource "aws_security_group_rule" "node" {
      + description              = "Node to node all ports/protocols"
      + from_port                = 0
      + id                       = (known after apply)
      + prefix_list_ids          = []
      + protocol                 = "-1"
      + security_group_id        = (known after apply)
```

```
651           + security_group_rule_id   = (known after apply)
652           + self                     = true
653           + source_security_group_id = (known after apply)
654           + to_port                  = 0
655           + type                     = "ingress"
656         }
657
658       # module.eks.aws_security_group_rule.node["ingress_self_coredns_tcp"] will be
              created
659       + resource "aws_security_group_rule" "node" {
660           + description              = "Node to node CoreDNS"
661           + from_port                = 53
662           + id                       = (known after apply)
663           + prefix_list_ids          = []
664           + protocol                 = "tcp"
665           + security_group_id        = (known after apply)
666           + security_group_rule_id   = (known after apply)
667           + self                     = true
668           + source_security_group_id = (known after apply)
669           + to_port                  = 53
670           + type                     = "ingress"
671         }
672
673       # module.eks.aws_security_group_rule.node["ingress_self_coredns_udp"] will be
              created
674       + resource "aws_security_group_rule" "node" {
675           + description              = "Node to node CoreDNS UDP"
676           + from_port                = 53
677           + id                       = (known after apply)
678           + prefix_list_ids          = []
679           + protocol                 = "udp"
680           + security_group_id        = (known after apply)
681           + security_group_rule_id   = (known after apply)
682           + self                     = true
683           + source_security_group_id = (known after apply)
684           + to_port                  = 53
685           + type                     = "ingress"
686         }
687
688       # module.eks.time_sleep.this[0] will be created
689       + resource "time_sleep" "this" {
690           + create_duration = "30s"
691           + id              = (known after apply)
692           + triggers        = {
693               + "cluster_certificate_authority_data" = (known after apply)
694               + "cluster_endpoint"                   = (known after apply)
695               + "cluster_name"                       = (known after apply)
696               + "cluster_service_cidr"               = (known after apply)
697               + "cluster_version"                    = "1.30"
698             }
699         }
700
701       # module.vpc.aws_default_network_acl.this[0] will be created
702       + resource "aws_default_network_acl" "this" {
703           + arn                    = (known after apply)
704           + default_network_acl_id = (known after apply)
705           + id                     = (known after apply)
706           + owner_id               = (known after apply)
707           + tags                   = {
708               + "Name" = "minghan-eks-cluster-vpc-default"
709             }
710           + tags_all               = {
711               + "Name" = "minghan-eks-cluster-vpc-default"
```

```
            }
          + vpc_id                 = (known after apply)

          + egress {
              + action          = "allow"
              + from_port       = 0
              + ipv6_cidr_block = "::/0"
              + protocol        = "-1"
              + rule_no         = 101
              + to_port         = 0
                # (1 unchanged attribute hidden)
            }
          + egress {
              + action          = "allow"
              + cidr_block      = "0.0.0.0/0"
              + from_port       = 0
              + protocol        = "-1"
              + rule_no         = 100
              + to_port         = 0
                # (1 unchanged attribute hidden)
            }

          + ingress {
              + action          = "allow"
              + from_port       = 0
              + ipv6_cidr_block = "::/0"
              + protocol        = "-1"
              + rule_no         = 101
              + to_port         = 0
                # (1 unchanged attribute hidden)
            }
          + ingress {
              + action          = "allow"
              + cidr_block      = "0.0.0.0/0"
              + from_port       = 0
              + protocol        = "-1"
              + rule_no         = 100
              + to_port         = 0
                # (1 unchanged attribute hidden)
            }
        }

      # module.vpc.aws_default_route_table.default[0] will be created
      + resource "aws_default_route_table" "default" {
          + arn                    = (known after apply)
          + default_route_table_id = (known after apply)
          + id                     = (known after apply)
          + owner_id               = (known after apply)
          + route                  = (known after apply)
          + tags                   = {
              + "Name" = "minghan-eks-cluster-vpc-default"
            }
          + tags_all               = {
              + "Name" = "minghan-eks-cluster-vpc-default"
            }
          + vpc_id                 = (known after apply)

          + timeouts {
              + create = "5m"
              + update = "5m"
            }
        }

```

```
      # module.vpc.aws_default_security_group.this[0] will be created
      + resource "aws_default_security_group" "this" {
          + arn                    = (known after apply)
          + description            = (known after apply)
          + egress                 = (known after apply)
          + id                     = (known after apply)
          + ingress                = (known after apply)
          + name                   = (known after apply)
          + name_prefix            = (known after apply)
          + owner_id               = (known after apply)
          + revoke_rules_on_delete = false
          + tags                   = {
              + "Name" = "minghan-eks-cluster-vpc-default"
            }
          + tags_all               = {
              + "Name" = "minghan-eks-cluster-vpc-default"
            }
          + vpc_id                 = (known after apply)
        }

      # module.vpc.aws_eip.nat[0] will be created
      + resource "aws_eip" "nat" {
          + allocation_id        = (known after apply)
          + arn                  = (known after apply)
          + association_id       = (known after apply)
          + carrier_ip           = (known after apply)
          + customer_owned_ip    = (known after apply)
          + domain               = "vpc"
          + id                   = (known after apply)
          + instance             = (known after apply)
          + network_border_group = (known after apply)
          + network_interface    = (known after apply)
          + private_dns          = (known after apply)
          + private_ip           = (known after apply)
          + ptr_record           = (known after apply)
          + public_dns           = (known after apply)
          + public_ip            = (known after apply)
          + public_ipv4_pool     = (known after apply)
          + tags                 = {
              + "Name" = "minghan-eks-cluster-vpc-ap-southeast-1a"
            }
          + tags_all             = {
              + "Name" = "minghan-eks-cluster-vpc-ap-southeast-1a"
            }
          + vpc                  = (known after apply)
        }

      # module.vpc.aws_internet_gateway.this[0] will be created
      + resource "aws_internet_gateway" "this" {
          + arn      = (known after apply)
          + id       = (known after apply)
          + owner_id = (known after apply)
          + tags     = {
              + "Name" = "minghan-eks-cluster-vpc"
            }
          + tags_all = {
              + "Name" = "minghan-eks-cluster-vpc"
            }
          + vpc_id   = (known after apply)
        }

      # module.vpc.aws_nat_gateway.this[0] will be created
      + resource "aws_nat_gateway" "this" {
```

```
      + allocation_id                       = (known after apply)
      + association_id                      = (known after apply)
      + connectivity_type                   = "public"
      + id                                  = (known after apply)
      + network_interface_id                = (known after apply)
      + private_ip                          = (known after apply)
      + public_ip                           = (known after apply)
      + secondary_private_ip_address_count = (known after apply)
      + secondary_private_ip_addresses      = (known after apply)
      + subnet_id                           = (known after apply)
      + tags                                = {
          + "Name" = "minghan-eks-cluster-vpc-ap-southeast-1a"
        }
      + tags_all                            = {
          + "Name" = "minghan-eks-cluster-vpc-ap-southeast-1a"
        }
    }

  # module.vpc.aws_route.private_nat_gateway[0] will be created
  + resource "aws_route" "private_nat_gateway" {
      + destination_cidr_block = "0.0.0.0/0"
      + id                     = (known after apply)
      + instance_id            = (known after apply)
      + instance_owner_id      = (known after apply)
      + nat_gateway_id         = (known after apply)
      + network_interface_id   = (known after apply)
      + origin                 = (known after apply)
      + route_table_id         = (known after apply)
      + state                  = (known after apply)

      + timeouts {
          + create = "5m"
        }
    }

  # module.vpc.aws_route.public_internet_gateway[0] will be created
  + resource "aws_route" "public_internet_gateway" {
      + destination_cidr_block = "0.0.0.0/0"
      + gateway_id             = (known after apply)
      + id                     = (known after apply)
      + instance_id            = (known after apply)
      + instance_owner_id      = (known after apply)
      + network_interface_id   = (known after apply)
      + origin                 = (known after apply)
      + route_table_id         = (known after apply)
      + state                  = (known after apply)

      + timeouts {
          + create = "5m"
        }
    }

  # module.vpc.aws_route_table.private[0] will be created
  + resource "aws_route_table" "private" {
      + arn              = (known after apply)
      + id               = (known after apply)
      + owner_id         = (known after apply)
      + propagating_vgws = (known after apply)
      + route            = (known after apply)
      + tags             = {
          + "Name" = "minghan-eks-cluster-vpc-private"
        }
      + tags_all         = {
```

```
901               + "Name" = "minghan-eks-cluster-vpc-private"
902             }
903           + vpc_id           = (known after apply)
904         }
905
906       # module.vpc.aws_route_table.public[0] will be created
907       + resource "aws_route_table" "public" {
908           + arn              = (known after apply)
909           + id               = (known after apply)
910           + owner_id         = (known after apply)
911           + propagating_vgws = (known after apply)
912           + route            = (known after apply)
913           + tags             = {
914               + "Name" = "minghan-eks-cluster-vpc-public"
915             }
916           + tags_all         = {
917               + "Name" = "minghan-eks-cluster-vpc-public"
918             }
919           + vpc_id           = (known after apply)
920         }
921
922       # module.vpc.aws_route_table_association.private[0] will be created
923       + resource "aws_route_table_association" "private" {
924           + id             = (known after apply)
925           + route_table_id = (known after apply)
926           + subnet_id      = (known after apply)
927         }
928
929       # module.vpc.aws_route_table_association.private[1] will be created
930       + resource "aws_route_table_association" "private" {
931           + id             = (known after apply)
932           + route_table_id = (known after apply)
933           + subnet_id      = (known after apply)
934         }
935
936       # module.vpc.aws_route_table_association.public[0] will be created
937       + resource "aws_route_table_association" "public" {
938           + id             = (known after apply)
939           + route_table_id = (known after apply)
940           + subnet_id      = (known after apply)
941         }
942
943       # module.vpc.aws_route_table_association.public[1] will be created
944       + resource "aws_route_table_association" "public" {
945           + id             = (known after apply)
946           + route_table_id = (known after apply)
947           + subnet_id      = (known after apply)
948         }
949
950       # module.vpc.aws_subnet.private[0] will be created
951       + resource "aws_subnet" "private" {
952           + arn                                            = (known after apply)
953           + assign_ipv6_address_on_creation                = false
954           + availability_zone                              = "ap-southeast-1a"
955           + availability_zone_id                           = (known after apply)
956           + cidr_block                                     = "10.0.1.0/24"
957           + enable_dns64                                   = false
958           + enable_resource_name_dns_a_record_on_launch    = false
959           + enable_resource_name_dns_aaaa_record_on_launch = false
960           + id                                             = (known after apply)
961           + ipv6_cidr_block_association_id                 = (known after apply)
962           + ipv6_native                                    = false
963           + map_public_ip_on_launch                        = false
```

```
964           + owner_id                                      = (known after apply)
965           + private_dns_hostname_type_on_launch           = (known after apply)
966           + tags                                          = {
967               + "Name"                                = "minghan-eks-cluster-vpc-private-
                      ap-southeast-1a"
968               + "kubernetes.io/role/internal-elb" = "1"
969             }
970           + tags_all                                      = {
971               + "Name"                                = "minghan-eks-cluster-vpc-private-
                      ap-southeast-1a"
972               + "kubernetes.io/role/internal-elb" = "1"
973             }
974           + vpc_id                                        = (known after apply)
975         }
976
977       # module.vpc.aws_subnet.private[1] will be created
978       + resource "aws_subnet" "private" {
979           + arn                                           = (known after apply)
980           + assign_ipv6_address_on_creation               = false
981           + availability_zone                             = "ap-southeast-1b"
982           + availability_zone_id                          = (known after apply)
983           + cidr_block                                    = "10.0.2.0/24"
984           + enable_dns64                                  = false
985           + enable_resource_name_dns_a_record_on_launch   = false
986           + enable_resource_name_dns_aaaa_record_on_launch = false
987           + id                                            = (known after apply)
988           + ipv6_cidr_block_association_id                = (known after apply)
989           + ipv6_native                                   = false
990           + map_public_ip_on_launch                       = false
991           + owner_id                                      = (known after apply)
992           + private_dns_hostname_type_on_launch           = (known after apply)
993           + tags                                          = {
994               + "Name"                                = "minghan-eks-cluster-vpc-private-
                      ap-southeast-1b"
995               + "kubernetes.io/role/internal-elb" = "1"
996             }
997           + tags_all                                      = {
998               + "Name"                                = "minghan-eks-cluster-vpc-private-
                      ap-southeast-1b"
999               + "kubernetes.io/role/internal-elb" = "1"
1000            }
1001          + vpc_id                                        = (known after apply)
1002        }
1003
1004      # module.vpc.aws_subnet.public[0] will be created
1005      + resource "aws_subnet" "public" {
1006          + arn                                           = (known after apply)
1007          + assign_ipv6_address_on_creation               = false
1008          + availability_zone                             = "ap-southeast-1a"
1009          + availability_zone_id                          = (known after apply)
1010          + cidr_block                                    = "10.0.4.0/24"
1011          + enable_dns64                                  = false
1012          + enable_resource_name_dns_a_record_on_launch   = false
1013          + enable_resource_name_dns_aaaa_record_on_launch = false
1014          + id                                            = (known after apply)
1015          + ipv6_cidr_block_association_id                = (known after apply)
1016          + ipv6_native                                   = false
1017          + map_public_ip_on_launch                       = true
1018          + owner_id                                      = (known after apply)
1019          + private_dns_hostname_type_on_launch           = (known after apply)
1020          + tags                                          = {
1021              + "Name"                    = "minghan-eks-cluster-vpc-public-ap-
                      southeast-1a"
```

```
      + "kubernetes.io/role/elb" = "1"
    }
  + tags_all                                              = {
      + "Name"                    = "minghan-eks-cluster-vpc-public-ap-
          southeast-1a"
      + "kubernetes.io/role/elb" = "1"
    }
  + vpc_id                                                = (known after apply)
}

# module.vpc.aws_subnet.public[1] will be created
+ resource "aws_subnet" "public" {
  + arn                                                   = (known after apply)
  + assign_ipv6_address_on_creation                       = false
  + availability_zone                                     = "ap-southeast-1b"
  + availability_zone_id                                  = (known after apply)
  + cidr_block                                            = "10.0.5.0/24"
  + enable_dns64                                          = false
  + enable_resource_name_dns_a_record_on_launch           = false
  + enable_resource_name_dns_aaaa_record_on_launch        = false
  + id                                                    = (known after apply)
  + ipv6_cidr_block_association_id                        = (known after apply)
  + ipv6_native                                           = false
  + map_public_ip_on_launch                               = true
  + owner_id                                              = (known after apply)
  + private_dns_hostname_type_on_launch                   = (known after apply)
  + tags                                                  = {
      + "Name"                    = "minghan-eks-cluster-vpc-public-ap-
          southeast-1b"
      + "kubernetes.io/role/elb" = "1"
    }
  + tags_all                                              = {
      + "Name"                    = "minghan-eks-cluster-vpc-public-ap-
          southeast-1b"
      + "kubernetes.io/role/elb" = "1"
    }
  + vpc_id                                                = (known after apply)
}

# module.vpc.aws_vpc.this[0] will be created
+ resource "aws_vpc" "this" {
  + arn                                  = (known after apply)
  + cidr_block                           = "10.0.0.0/16"
  + default_network_acl_id               = (known after apply)
  + default_route_table_id               = (known after apply)
  + default_security_group_id            = (known after apply)
  + dhcp_options_id                      = (known after apply)
  + enable_dns_hostnames                 = true
  + enable_dns_support                   = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                                   = (known after apply)
  + instance_tenancy                     = "default"
  + ipv6_association_id                  = (known after apply)
  + ipv6_cidr_block                      = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id                  = (known after apply)
  + owner_id                             = (known after apply)
  + tags                                 = {
      + "Name" = "minghan-eks-cluster-vpc"
    }
  + tags_all                             = {
      + "Name" = "minghan-eks-cluster-vpc"
    }
```

```
1082          }
1083
1084        # module.eks.module.eks_managed_node_group["one"].aws_eks_node_group.this[0]
              will be created
1085        + resource "aws_eks_node_group" "this" {
1086            + ami_type               = "AL2_x86_64"
1087            + arn                    = (known after apply)
1088            + capacity_type          = (known after apply)
1089            + cluster_name           = (known after apply)
1090            + disk_size              = (known after apply)
1091            + id                     = (known after apply)
1092            + instance_types         = [
1093                + "t3.medium",
1094              ]
1095            + node_group_name        = (known after apply)
1096            + node_group_name_prefix = "node-group-1-"
1097            + node_role_arn          = (known after apply)
1098            + release_version        = (known after apply)
1099            + resources              = (known after apply)
1100            + status                 = (known after apply)
1101            + subnet_ids             = (known after apply)
1102            + tags                   = {
1103                + "Name" = "node-group-1"
1104              }
1105            + tags_all               = {
1106                + "Name" = "node-group-1"
1107              }
1108            + version                = "1.30"
1109
1110            + launch_template {
1111                + id      = (known after apply)
1112                + name    = (known after apply)
1113                + version = (known after apply)
1114              }
1115
1116            + scaling_config {
1117                + desired_size = 2
1118                + max_size     = 2
1119                + min_size     = 2
1120              }
1121
1122            + timeouts {}
1123
1124            + update_config {
1125                + max_unavailable_percentage = 33
1126              }
1127          }
1128
1129        # module.eks.module.eks_managed_node_group["one"].aws_iam_role.this[0] will be
              created
1130        + resource "aws_iam_role" "this" {
1131            + arn                   = (known after apply)
1132            + assume_role_policy    = jsonencode(
1133                  {
1134                    + Statement = [
1135                        + {
1136                            + Action    = "sts:AssumeRole"
1137                            + Effect    = "Allow"
1138                            + Principal = {
1139                                + Service = "ec2.amazonaws.com"
1140                              }
1141                            + Sid       = "EKSNodeAssumeRole"
1142                          },
```

```
                    ]
                  + Version   = "2012-10-17"
                }
            )
          + create_date           = (known after apply)
          + description           = "EKS managed node group IAM role"
          + force_detach_policies = true
          + id                    = (known after apply)
          + managed_policy_arns   = (known after apply)
          + max_session_duration  = 3600
          + name                  = (known after apply)
          + name_prefix           = "node-group-1-eks-node-group-"
          + path                  = "/"
          + tags_all              = (known after apply)
          + unique_id             = (known after apply)

          + inline_policy (known after apply)
        }

      # module.eks.module.eks_managed_node_group["one"].aws_iam_role_policy_attachment
          .additional["AmazonS3FullAccess"] will be created
      + resource "aws_iam_role_policy_attachment" "additional" {
          + id         = (known after apply)
          + policy_arn = "arn:aws:iam::aws:policy/AmazonS3FullAccess"
          + role       = (known after apply)
        }

      # module.eks.module.eks_managed_node_group["one"].aws_iam_role_policy_attachment
          .additional["AmazonSSMManagedInstanceCore"] will be created
      + resource "aws_iam_role_policy_attachment" "additional" {
          + id         = (known after apply)
          + policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
          + role       = (known after apply)
        }

      # module.eks.module.eks_managed_node_group["one"].aws_iam_role_policy_attachment
          .this["AmazonEC2ContainerRegistryReadOnly"] will be created
      + resource "aws_iam_role_policy_attachment" "this" {
          + id         = (known after apply)
          + policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
          + role       = (known after apply)
        }

      # module.eks.module.eks_managed_node_group["one"].aws_iam_role_policy_attachment
          .this["AmazonEKSWorkerNodePolicy"] will be created
      + resource "aws_iam_role_policy_attachment" "this" {
          + id         = (known after apply)
          + policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
          + role       = (known after apply)
        }

      # module.eks.module.eks_managed_node_group["one"].aws_iam_role_policy_attachment
          .this["AmazonEKS_CNI_Policy"] will be created
      + resource "aws_iam_role_policy_attachment" "this" {
          + id         = (known after apply)
          + policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
          + role       = (known after apply)
        }

      # module.eks.module.eks_managed_node_group["one"].aws_launch_template.this[0]
          will be created
      + resource "aws_launch_template" "this" {
          + arn                    = (known after apply)
```

```
      + default_version        = (known after apply)
      + description            = "Custom launch template for node-group-1 EKS
          managed node group"
      + id                     = (known after apply)
      + latest_version         = (known after apply)
      + name                   = (known after apply)
      + name_prefix            = "one-"
      + tags_all               = (known after apply)
      + update_default_version = true
      + vpc_security_group_ids = (known after apply)
        # (2 unchanged attributes hidden)

      + metadata_options {
          + http_endpoint               = "enabled"
          + http_protocol_ipv6          = (known after apply)
          + http_put_response_hop_limit = 2
          + http_tokens                 = "required"
          + instance_metadata_tags      = (known after apply)
        }

      + monitoring {
          + enabled = true
        }

      + tag_specifications {
          + resource_type = "instance"
          + tags          = {
              + "Name" = "node-group-1"
            }
        }
      + tag_specifications {
          + resource_type = "network-interface"
          + tags          = {
              + "Name" = "node-group-1"
            }
        }
      + tag_specifications {
          + resource_type = "volume"
          + tags          = {
              + "Name" = "node-group-1"
            }
        }
    }

  # module.eks.module.kms.data.aws_iam_policy_document.this[0] will be read during
      apply
  # (config refers to values not yet known)
 <= data "aws_iam_policy_document" "this" {
      + id                        = (known after apply)
      + json                      = (known after apply)
      + override_policy_documents = []
      + source_policy_documents   = []

      + statement {
          + actions   = [
              + "kms:*",
            ]
          + resources = [
              + "*",
            ]
          + sid       = "Default"

          + principals {
```

```
                + identifiers = [
                    + "arn:aws:iam::861814105207:root",
                  ]
                + type        = "AWS"
              }
          }
        + statement {
            + actions   = [
                + "kms:CancelKeyDeletion",
                + "kms:Create*",
                + "kms:Delete*",
                + "kms:Describe*",
                + "kms:Disable*",
                + "kms:Enable*",
                + "kms:Get*",
                + "kms:ImportKeyMaterial",
                + "kms:List*",
                + "kms:Put*",
                + "kms:ReplicateKey",
                + "kms:Revoke*",
                + "kms:ScheduleKeyDeletion",
                + "kms:TagResource",
                + "kms:UntagResource",
                + "kms:Update*",
              ]
            + resources = [
                + "*",
              ]
            + sid       = "KeyAdministration"

            + principals {
                + identifiers = [
                    + "arn:aws:iam::861814105207:user/chanming",
                  ]
                + type        = "AWS"
              }
          }
        + statement {
            + actions   = [
                + "kms:Decrypt",
                + "kms:DescribeKey",
                + "kms:Encrypt",
                + "kms:GenerateDataKey*",
                + "kms:ReEncrypt*",
              ]
            + resources = [
                + "*",
              ]
            + sid       = "KeyUsage"

            + principals {
                + identifiers = [
                    + (known after apply),
                  ]
                + type        = "AWS"
              }
          }
      }

    # module.eks.module.kms.aws_kms_alias.this["cluster"] will be created
    + resource "aws_kms_alias" "this" {
        + arn            = (known after apply)
        + id             = (known after apply)
```

```
          + name           = (known after apply)
          + name_prefix    = (known after apply)
          + target_key_arn = (known after apply)
          + target_key_id  = (known after apply)
        }

      # module.eks.module.kms.aws_kms_key.this[0] will be created
      + resource "aws_kms_key" "this" {
          + arn                                = (known after apply)
          + bypass_policy_lockout_safety_check = false
          + customer_master_key_spec           = "SYMMETRIC_DEFAULT"
          + description                        = (known after apply)
          + enable_key_rotation                = true
          + id                                 = (known after apply)
          + is_enabled                         = true
          + key_id                             = (known after apply)
          + key_usage                          = "ENCRYPT_DECRYPT"
          + multi_region                       = false
          + policy                             = (known after apply)
          + tags                               = {
              + "terraform-aws-modules" = "eks"
            }
          + tags_all                           = {
              + "terraform-aws-modules" = "eks"
            }
        }

      # module.eks.module.eks_managed_node_group["one"].module.user_data.null_resource
          .validate_cluster_service_cidr will be created
      + resource "null_resource" "validate_cluster_service_cidr" {
          + id = (known after apply)
        }

    Plan: 65 to add, 0 to change, 0 to destroy.

    Changes to Outputs:
      + cluster_endpoint          = (known after apply)
      + cluster_name              = (known after apply)
      + cluster_security_group_id = (known after apply)
      + efs_id                    = (known after apply)
      + region                    = "ap-southeast-1"

    Do you want to perform these actions?
      Terraform will perform the actions described above.
      Only 'yes' will be accepted to approve.

      Enter a value:
```

Listing E.1: terraform plan output for `cloudkube` provisioning

Appendix F: `cloudkube` successful provisioning output

The following shows the output when `cloudkube` successfully provisions a Kubernetes cluster. This is output after user approval of `terraform plan`.

```
aws_efs_backup_policy.policy: Destroying... [id=fs-006892326429a9091]
aws_efs_mount_target.efs_mount_target_1: Destroying... [id=fsmt-049b1cfd599b6d02f]
aws_efs_mount_target.efs_mount_target_2: Destroying... [id=fsmt-013a25d1fba50ba69]
module.eks.aws_cloudwatch_log_group.this[0]: Creating...
module.eks.aws_iam_role.this[0]: Creating...
module.eks.aws_cloudwatch_log_group.this[0]: Creation complete after 0s [id=/aws/
    eks/mh-fyp-eks-wFMZsm5Z/cluster]
aws_efs_backup_policy.policy: Destruction complete after 2s
module.eks.aws_iam_role.this[0]: Creation complete after 2s [id=mh-fyp-eks-
    wFMZsm5Z-cluster-20250203032553942600000001]
module.eks.aws_iam_role_policy_attachment.this["AmazonEKSVPCResourceController"]:
    Creating...
module.eks.aws_iam_role_policy_attachment.this["AmazonEKSClusterPolicy"]: Creating
    ...
module.eks.module.kms.data.aws_iam_policy_document.this[0]: Reading...
module.eks.module.kms.data.aws_iam_policy_document.this[0]: Read complete after 0s
     [id=1425458566]
module.eks.module.kms.aws_kms_key.this[0]: Creating...
module.eks.aws_iam_role_policy_attachment.this["AmazonEKSClusterPolicy"]: Creation
     complete after 1s [id=mh-fyp-eks-wFMZsm5Z-cluster
    -20250203032553942600000001-20250203032556718200000002]
module.eks.aws_iam_role_policy_attachment.this["AmazonEKSVPCResourceController"]:
    Creation complete after 1s [id=mh-fyp-eks-wFMZsm5Z-cluster
    -20250203032553942600000001-20250203032557022900000003]
aws_efs_mount_target.efs_mount_target_2: Destruction complete after 21s
aws_efs_mount_target.efs_mount_target_1: Destruction complete after 21s
aws_efs_file_system.efs: Destroying... [id=fs-006892326429a9091]
aws_security_group.efs_security_group: Destroying... [id=sg-05aec177b12885cb7]
aws_security_group.efs_security_group: Destruction complete after 1s
module.vpc.aws_vpc.this[0]: Modifying... [id=vpc-07906e73af4bdfa38]
module.eks.module.kms.aws_kms_key.this[0]: Creation complete after 21s [id=39788
    f43-b716-41c7-b62c-ea716ead9495]
module.eks.module.kms.aws_kms_alias.this["cluster"]: Creating...
module.eks.aws_iam_policy.cluster_encryption[0]: Creating...
module.vpc.aws_vpc.this[0]: Modifications complete after 1s [id=vpc-07906
    e73af4bdfa38]
module.eks.aws_security_group.cluster[0]: Creating...
module.eks.aws_security_group.node[0]: Creating...
module.vpc.aws_default_security_group.this[0]: Modifying... [id=sg-004
    b85b82b9a4e893]
module.vpc.aws_default_route_table.default[0]: Modifying... [id=rtb-0
    ce95e233fcc0893e]
aws_security_group.efs_security_group: Creating...
module.vpc.aws_subnet.public[1]: Modifying... [id=subnet-00ad26679af7193d6]
module.vpc.aws_default_network_acl.this[0]: Modifying... [id=acl-088b7bc72bbae1655
    ]
module.vpc.aws_default_route_table.default[0]: Modifications complete after 0s [id
    =rtb-0ce95e233fcc0893e]
```

```
34  module.vpc.aws_default_network_acl.this[0]: Modifications complete after 0s [id=
        acl-088b7bc72bbae1655]
35  module.vpc.aws_subnet.public[0]: Modifying... [id=subnet-0242d8325c1411258]
36  module.vpc.aws_route_table.public[0]: Modifying... [id=rtb-088893d0f63ae6ae2]
37  module.vpc.aws_default_security_group.this[0]: Modifications complete after 0s [id
        =sg-004b85b82b9a4e893]
38  module.vpc.aws_subnet.private[0]: Modifying... [id=subnet-0406b1e1be0246d54]
39  module.eks.module.kms.aws_kms_alias.this["cluster"]: Creation complete after 0s [
        id=alias/eks/mh-fyp-eks-wFMZsm5Z]
40  module.vpc.aws_subnet.public[1]: Modifications complete after 0s [id=subnet-00
        ad26679af7193d6]
41  module.vpc.aws_route_table.private[0]: Modifying... [id=rtb-0e1db9bfba2e86daf]
42  module.vpc.aws_subnet.private[1]: Modifying... [id=subnet-0da8c6fc54bcca700]
43  module.vpc.aws_route_table.public[0]: Modifications complete after 0s [id=rtb
        -088893d0f63ae6ae2]
44  module.vpc.aws_internet_gateway.this[0]: Modifying... [id=igw-0a42a0a8b33bb9171]
45  module.vpc.aws_route_table.private[0]: Modifications complete after 0s [id=rtb-0
        e1db9bfba2e86daf]
46  module.vpc.aws_subnet.public[0]: Modifications complete after 0s [id=subnet-0242
        d8325c1411258]
47  module.vpc.aws_subnet.private[0]: Modifications complete after 0s [id=subnet-0406
        b1e1be0246d54]
48  module.vpc.aws_internet_gateway.this[0]: Modifications complete after 0s [id=igw-0
        a42a0a8b33bb9171]
49  module.vpc.aws_subnet.private[1]: Modifications complete after 0s [id=subnet-0
        da8c6fc54bcca700]
50  module.vpc.aws_eip.nat[0]: Modifying... [id=eipalloc-00e9979b2f14a9b77]
51  module.eks.aws_iam_policy.cluster_encryption[0]: Creation complete after 0s [id=
        arn:aws:iam::861814105207:policy/mh-fyp-eks-wFMZsm5Z-cluster-
        ClusterEncryption20250203032616588500000004]
52  module.eks.aws_iam_role_policy_attachment.cluster_encryption[0]: Creating...
53  aws_efs_file_system.efs: Destruction complete after 3s
54  aws_efs_file_system.efs: Creating...
55  module.vpc.aws_eip.nat[0]: Modifications complete after 1s [id=eipalloc-00
        e9979b2f14a9b77]
56  module.vpc.aws_nat_gateway.this[0]: Modifying... [id=nat-0a2c160f99e36f111]
57  module.vpc.aws_nat_gateway.this[0]: Modifications complete after 0s [id=nat-0
        a2c160f99e36f111]
58  module.eks.aws_iam_role_policy_attachment.cluster_encryption[0]: Creation complete
         after 1s [id=mh-fyp-eks-wFMZsm5Z-cluster
        -20250203032553942600000001-20250203032617809300000007]
59  module.eks.aws_security_group.cluster[0]: Creation complete after 1s [id=sg-0869
        b037f781256c0]
60  module.eks.aws_security_group.node[0]: Creation complete after 1s [id=sg-06025
        b703daae1f50]
61  module.eks.aws_security_group_rule.node["ingress_self_coredns_udp"]: Creating...
62  module.eks.aws_security_group_rule.node["egress_all"]: Creating...
63  module.eks.aws_security_group_rule.node["ingress_cluster_6443_webhook"]: Creating
        ...
64  module.eks.aws_security_group_rule.node["ingress_cluster_4443_webhook"]: Creating
        ...
65  module.eks.aws_security_group_rule.cluster["ingress_nodes_443"]: Creating...
66  module.eks.aws_security_group_rule.node["ingress_self_all"]: Creating...
67  module.eks.aws_security_group_rule.node["ingress_cluster_9443_webhook"]: Creating
        ...
68  module.eks.aws_security_group_rule.node["ingress_cluster_8443_webhook"]: Creating
        ...
69  aws_security_group.efs_security_group: Creation complete after 2s [id=sg-07071
        d8c70918f239]
70  module.eks.aws_security_group_rule.node["ingress_cluster_6443_webhook"]: Creation
        complete after 1s [id=sgrule-3527609920]
71  module.eks.aws_security_group_rule.cluster["ingress_nodes_443"]: Creation complete
         after 1s [id=sgrule-2853805676]
```

```
72    module.eks.aws_security_group_rule.node["ingress_nodes_ephemeral"]: Creating...
73    module.eks.aws_security_group_rule.node["ingress_cluster_443"]: Creating...
74    module.eks.aws_security_group_rule.node["ingress_self_coredns_tcp"]: Creating...
75    module.eks.aws_security_group_rule.node["ingress_cluster_4443_webhook"]: Creation
          complete after 1s [id=sgrule-453077462]
76    module.eks.aws_security_group_rule.node["ingress_cluster_kubelet"]: Creating...
77    module.eks.aws_security_group_rule.node["ingress_self_all"]: Creation complete
          after 2s [id=sgrule-4254759833]
78    module.eks.aws_security_group_rule.node["egress_all"]: Creation complete after 2s
          [id=sgrule-2240897126]
79    module.eks.aws_security_group_rule.node["ingress_cluster_9443_webhook"]: Creation
          complete after 3s [id=sgrule-2780692906]
80    module.eks.aws_security_group_rule.node["ingress_self_coredns_udp"]: Creation
          complete after 4s [id=sgrule-1320893099]
81    module.eks.aws_security_group_rule.node["ingress_cluster_8443_webhook"]: Creation
          complete after 4s [id=sgrule-3240047201]
82    module.eks.aws_security_group_rule.node["ingress_nodes_ephemeral"]: Creation
          complete after 4s [id=sgrule-2608620767]
83    aws_efs_file_system.efs: Creation complete after 5s [id=fs-078a7048b2c301c27]
84    module.eks.aws_security_group_rule.node["ingress_cluster_443"]: Creation complete
          after 4s [id=sgrule-1773228432]
85    aws_efs_backup_policy.policy: Creating...
86    aws_efs_mount_target.efs_mount_target_2: Creating...
87    aws_efs_mount_target.efs_mount_target_1: Creating...
88    module.eks.aws_security_group_rule.node["ingress_self_coredns_tcp"]: Creation
          complete after 5s [id=sgrule-3940849988]
89    module.eks.aws_security_group_rule.node["ingress_cluster_kubelet"]: Creation
          complete after 5s [id=sgrule-776379637]
90    module.eks.aws_eks_cluster.this[0]: Creating...
91    aws_efs_backup_policy.policy: Creation complete after 2s [id=fs-078a7048b2c301c27]
92    aws_efs_mount_target.efs_mount_target_2: Creation complete after 1m23s [id=fsmt-04
          d2ec3f1e7bed547]
93    aws_efs_mount_target.efs_mount_target_1: Creation complete after 1m23s [id=fsmt-0
          ac906803bf9d1fae]
94    module.eks.aws_eks_cluster.this[0]: Creation complete after 8m3s [id=mh-fyp-eks-
          wFMZsm5Z]
95    module.eks.data.aws_eks_addon_version.this["kube-proxy"]: Reading...
96    module.eks.data.aws_eks_addon_version.this["coredns"]: Reading...
97    module.eks.data.aws_eks_addon_version.this["eks-pod-identity-agent"]: Reading...
98    module.eks.data.aws_eks_addon_version.this["vpc-cni"]: Reading...
99    module.eks.aws_eks_access_entry.this["cluster_creator"]: Creating...
100   module.eks.data.tls_certificate.this[0]: Reading...
101   module.eks.time_sleep.this[0]: Creating...
102   module.eks.data.tls_certificate.this[0]: Read complete after 0s [id=02
          f0b3f30424129b8a14276375b6aebec81a5e17]
103   module.eks.aws_iam_openid_connect_provider.oidc_provider[0]: Creating...
104   module.eks.data.aws_eks_addon_version.this["kube-proxy"]: Read complete after 0s [
          id=kube-proxy]
105   module.eks.data.aws_eks_addon_version.this["eks-pod-identity-agent"]: Read
          complete after 0s [id=eks-pod-identity-agent]
106   module.eks.data.aws_eks_addon_version.this["coredns"]: Read complete after 1s [id=
          coredns]
107   module.eks.data.aws_eks_addon_version.this["vpc-cni"]: Read complete after 1s [id=
          vpc-cni]
108   module.eks.aws_eks_access_entry.this["cluster_creator"]: Creation complete after 1
          s [id=mh-fyp-eks-wFMZsm5Z:arn:aws:iam::861814105207:user/chanming]
109   module.eks.aws_eks_access_policy_association.this["cluster_creator_admin"]:
          Creating...
110   module.eks.aws_eks_access_policy_association.this["cluster_creator_admin"]:
          Creation complete after 0s [id=mh-fyp-eks-wFMZsm5Z#arn:aws:iam::861814105207:
          user/chanming#arn:aws:eks::aws:cluster-access-policy/
          AmazonEKSClusterAdminPolicy]
```

```
111     module.eks.aws_iam_openid_connect_provider.oidc_provider[0]: Creation complete
            after 2s [id=arn:aws:iam::861814105207:oidc-provider/oidc.eks.ap-southeast-1.
            amazonaws.com/id/5862339F7E77382B0A74D8C85BD42D31]
112     module.eks.time_sleep.this[0]: Creation complete after 30s [id=2025-02-03T03:34:57
            Z]
113     module.eks.module.eks_managed_node_group["one"].module.user_data.null_resource.
            validate_cluster_service_cidr: Creating...
114     module.eks.module.eks_managed_node_group["one"].module.user_data.null_resource.
            validate_cluster_service_cidr: Creation complete after 0s [id
            =4506423314006051376]
115     module.eks.module.eks_managed_node_group["one"].aws_launch_template.this[0]:
            Creating...
116     module.eks.module.eks_managed_node_group["one"].aws_launch_template.this[0]:
            Creation complete after 1s [id=lt-08afb7e4e64848ca8]
117     module.eks.module.eks_managed_node_group["one"].aws_eks_node_group.this[0]:
            Creating...
118     module.eks.module.eks_managed_node_group["one"].aws_eks_node_group.this[0]:
            Creation complete after 1m58s [id=mh-fyp-eks-wFMZsm5Z:node-group
            -1-20250203033457630400000000a]
119     module.eks.aws_eks_addon.this["kube-proxy"]: Creating...
120     module.eks.aws_eks_addon.this["coredns"]: Creating...
121     module.eks.aws_eks_addon.this["vpc-cni"]: Creating...
122     module.eks.aws_eks_addon.this["eks-pod-identity-agent"]: Creating...
123     module.eks.aws_eks_addon.this["kube-proxy"]: Creation complete after 7s [id=mh-fyp
            -eks-wFMZsm5Z:kube-proxy]
124     module.eks.aws_eks_addon.this["coredns"]: Creation complete after 14s [id=mh-fyp-
            eks-wFMZsm5Z:coredns]
125     module.eks.aws_eks_addon.this["eks-pod-identity-agent"]: Creation complete after
            44s [id=mh-fyp-eks-wFMZsm5Z:eks-pod-identity-agent]
126     module.eks.aws_eks_addon.this["vpc-cni"]: Creation complete after 44s [id=mh-fyp-
            eks-wFMZsm5Z:vpc-cni]
127     module.eks.aws_security_group_rule.cluster["ingress_nodes_443"] (deposed object
            927cce0b): Destroying... [id=sgrule-3991927841]
128     module.eks.aws_security_group_rule.node["egress_all"] (deposed object 04898922):
            Destroying... [id=sgrule-2250989526]
129     module.eks.aws_security_group_rule.node["ingress_self_coredns_tcp"] (deposed
            object d6e5d236): Destroying... [id=sgrule-1785397800]
130     module.eks.aws_security_group_rule.node["ingress_cluster_443"] (deposed object 746
            f42c4): Destroying... [id=sgrule-3986844362]
131     module.eks.aws_security_group_rule.node["ingress_cluster_9443_webhook"] (deposed
            object e9e2b630): Destroying... [id=sgrule-2172802925]
132     module.eks.aws_security_group_rule.node["ingress_cluster_kubelet"] (deposed object
             d2ee43f4): Destroying... [id=sgrule-4254499759]
133     module.eks.aws_cloudwatch_log_group.this[0] (deposed object 4864f132): Destroying
            ... [id=/aws/eks/minghan-eks-cluster-eks-wFMZsm5Z/cluster]
134     module.eks.aws_security_group_rule.node["ingress_nodes_ephemeral"] (deposed object
             2561e632): Destroying... [id=sgrule-2602521719]
135     module.eks.aws_security_group_rule.node["ingress_self_coredns_udp"] (deposed
            object e5b73c93): Destroying... [id=sgrule-3459555783]
136     module.eks.aws_security_group_rule.node["ingress_cluster_6443_webhook"] (deposed
            object af3a8556): Destroying... [id=sgrule-4135540871]
137     module.eks.aws_cloudwatch_log_group.this[0]: Destruction complete after 0s
138     module.eks.aws_security_group_rule.node["ingress_cluster_8443_webhook"] (deposed
            object 3137c3f8): Destroying... [id=sgrule-3844301990]
139     module.eks.aws_security_group_rule.cluster["ingress_nodes_443"]: Destruction
            complete after 1s
140     module.eks.aws_security_group_rule.node["ingress_self_coredns_tcp"]: Destruction
            complete after 1s
141     module.eks.aws_security_group_rule.node["ingress_cluster_4443_webhook"] (deposed
            object d6aab15d): Destroying... [id=sgrule-1060978449]
142     module.eks.aws_security_group_rule.node["ingress_self_all"] (deposed object 1
            c2a4d48): Destroying... [id=sgrule-2443960524]
```

```
143     module.eks.aws_security_group_rule.node["ingress_cluster_9443_webhook"]:
            Destruction complete after 1s
144     module.eks.aws_security_group_rule.node["ingress_cluster_6443_webhook"]:
            Destruction complete after 2s
145     module.eks.aws_security_group_rule.node["egress_all"]: Destruction complete after
            2s
146     module.eks.aws_security_group_rule.node["ingress_cluster_kubelet"]: Destruction
            complete after 2s
147     module.eks.aws_security_group_rule.node["ingress_nodes_ephemeral"]: Destruction
            complete after 3s
148     module.eks.aws_security_group_rule.node["ingress_cluster_443"]: Destruction
            complete after 3s
149     module.eks.aws_security_group_rule.node["ingress_self_coredns_udp"]: Destruction
            complete after 4s
150     module.eks.aws_security_group_rule.node["ingress_cluster_8443_webhook"]:
            Destruction complete after 4s
151     module.eks.aws_security_group_rule.node["ingress_self_all"]: Destruction complete
            after 4s
152     module.eks.aws_security_group_rule.node["ingress_cluster_4443_webhook"]:
            Destruction complete after 4s
153     module.eks.aws_security_group.cluster[0] (deposed object 5f805fa0): Destroying...
            [id=sg-01c9f8754d3e4815c]
154     module.eks.aws_security_group.node[0] (deposed object dbb128d2): Destroying... [id
            =sg-0ee0a1f7f0109b310]
155     module.eks.aws_security_group.cluster[0]: Destruction complete after 1s
156     module.eks.aws_security_group.node[0]: Destruction complete after 1s
157
158     Apply complete! Resources: 39 added, 13 changed, 20 destroyed.
159
160     Outputs:
161
162     cluster_endpoint = "https://5862339F7E77382B0A74D8C85BD42D31.gr7.ap-southeast-1.
            eks.amazonaws.com"
163     cluster_name = "mh-fyp-eks-wFMZsm5Z"
164     cluster_security_group_id = "sg-0869b037f781256c0"
165     efs_id = "fs-078a7048b2c301c27"
166     region = "ap-southeast-1"
167     [INFO]: Terraform applied successfully
168     Raw Terraform output for 'cluster_name': mh-fyp-eks-wFMZsm5Z
169     Raw Terraform output for 'region': ap-southeast-1
170     region:ap-southeast-1, cluster_name:mh-fyp-eks-wFMZsm5Z
171     [INFO]: kubectl configured successfully
172     [INFO]: install_aws_nginx_controller: Applied resource from https://raw.
            githubusercontent.com/kubernetes/ingress-nginx/controller-v1.12.0-beta.0/
            deploy/static/provider/aws/deploy.yaml successfully
173
```

Listing F.1: `cloudkube` successful provisioning output

Appendix G: `cloudkube` Publishing and Distribution

The following shows the `setup.py` and `workflow.yml` files which are used in the publishing and distribution of `cloudkube`

```
setup(
    name="cloudkube",
    version="0.16.5",
    author="Chan Ming Han",
    email="chanminghan00@gmail.com",
    description="minikube for the cloud",
    python_requires=">=3.11",
    packages=find_packages(),
    entry_points={
        "console_scripts": [
            "cloudkube=cloudkube:main",
        ],
    },
    install_requires=[
        "boto3",
        "python_terraform",
        "kubernetes",
        "requests",
        "GitPython",
    ],
    package_data={"cloudkube": ["terraform/*"]},
    include_package_data=True,
)
```

Listing G.1: `setup.py` file for Python Packaging

```
name: Publish Python distribution to PyPI

on: push

jobs:
  build:
    name: Build distribution
    runs-on: ubuntu-latest
    outputs:
        tag_ver: ${{ steps.bump_ver.outputs.tag_ver }}

    steps:
      - uses: actions/checkout@v4
        with:
          persist-credentials: false
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.x"
      - name: Install pypa/build
        run: >-
          python3 -m
          pip install
```

```
          build
          --user
      - name: Install bump for automatic versioning
        run: pip install bump
      - name: Bump Version
        id: bump_ver
        run: |
          TAG_VER=$(bump)
          echo "TAG_VER=$TAG_VER" >> $GITHUB_ENV
          echo "::set-output name=tag_ver::$TAG_VER"
      - name: Commit Version Bump
        run: |
          git config user.name "github-actions[bot]"
          git config user.email "github-actions[bot]@users.noreply.github.com"
          git add setup.py
          git commit -m "Bump version via GitHub Actions"
          git remote set-url origin https://x-access-token:${{ secrets.GITHUB_TOKEN
              }}@github.com/${{ github.repository }}
          git push origin main

      - name: Build a binary wheel and a source tarball
        run: python3 -m build
      - name: Store the distribution packages
        uses: actions/upload-artifact@v4
        with:
          name: python-package-distributions
          path: dist/

  publish-to-pypi:
    name: >-
      Publish Python distribution to PyPI
    needs:
      - build
    runs-on: ubuntu-latest
    environment:
      name: pypi
      url: https://pypi.org/p/cloudkube
    permissions:
      id-token: write # IMPORTANT: mandatory for trusted publishing

    steps:
      - name: Download all the dists
        uses: actions/download-artifact@v4
        with:
          name: python-package-distributions
          path: dist/
      - name: Publish distribution to PyPI
        uses: pypa/gh-action-pypi-publish@release/v1

  github-release:
    name: >-
      Sign the Python distribution with Sigstore
      and upload them to GitHub Release
    needs:
      - build
      - publish-to-pypi
    runs-on: ubuntu-latest

    permissions:
      contents: write # IMPORTANT: mandatory for making GitHub Releases
      id-token: write # IMPORTANT: mandatory for sigstore

    steps:
```

```
      - name: Download all the dists
        uses: actions/download-artifact@v4
        with:
          name: python-package-distributions
          path: dist/
      - name: Sign the dists with Sigstore
        uses: sigstore/gh-action-sigstore-python@v3.0.0
        with:
          inputs: >-
            ./dist/*.tar.gz
            ./dist/*.whl
      - name: Create GitHub Release
        env:
          GITHUB_TOKEN: ${{ github.token }}
        run: >-
          gh release create
          "${{ needs.build.outputs.tag_ver }}"
          --repo "$GITHUB_REPOSITORY"
          --notes ""
      - name: Upload artifact signatures to GitHub Release
        env:
          GITHUB_TOKEN: ${{ github.token }}
        # Upload to GitHub Release using the ‘gh‘ CLI.
        # ‘dist/‘ contains the built packages, and the
        # sigstore-produced signatures and certificates.
        run: >-
          gh release upload
          "${{ needs.build.outputs.tag_ver }}" dist/**
          --repo "$GITHUB_REPOSITORY"

```

Listing G.2: `workflow.yml` file for `cloudkube` release

Appendix H: Kubernetes Manifest for SpeechLab ASR Application

---

The following shows the Kubernetes Manifest for deploying the SpeechLab ASR Application.

```
apiVersion: v1
kind: Namespace
metadata:
    name: decoding-sdk

---
apiVersion: v1
kind: ConfigMap
metadata:
    name: decoding-sdk-config
    namespace: decoding-sdk
data:
    INSTANCE_NUM: "2"
    MASTER: "decoding-sdk-server:8010"
    MODEL_DIR: "genericasr_apr2021"

---
apiVersion: v1
kind: PersistentVolume
metadata:
    name: model-storage
spec:
    capacity:
    storage: 5Gi
    accessModes:
    - ReadWriteMany # EFS supports multi-node access
    storageClassName: efs-sc
    csi:
    driver: efs.csi.aws.com
    volumeHandle: fs-015b986c551bc3640
---
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
    name: efs-sc
provisioner: efs.csi.aws.com
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
    name: model-storage-claim
    namespace: decoding-sdk
spec:
    accessModes:
    - ReadWriteMany
    storageClassName: efs-sc
    resources:
    requests:
        storage: 1Gi

```

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
    name: decoding-sdk-server
    namespace: decoding-sdk
spec:
    replicas: 1
    selector:
    matchLabels:
        app: decoding-sdk-server
    template:
    metadata:
        labels:
        app: decoding-sdk-server
    spec:
        imagePullSecrets:
        - name: dockerhub-secret
        containers:
        - name: decoding-sdk-server
            image: minghan2000/decoding-sdk:2.0
            ports:
            - containerPort: 8010
            command: ["/home/speechuser/start_master.sh", "-p", "8010"]

---
apiVersion: v1
kind: Service
metadata:
    name: decoding-sdk-server
    namespace: decoding-sdk
spec:
    selector:
    app: decoding-sdk-server
    ports:
    - protocol: TCP
        port: 8010
        targetPort: 8010
    type: LoadBalancer

---
apiVersion: apps/v1
kind: Deployment
metadata:
    name: decoding-sdk-worker
    namespace: decoding-sdk
spec:
    replicas: 2
    selector:
    matchLabels:
        app: decoding-sdk-worker
    template:
    metadata:
        labels:
        app: decoding-sdk-worker
    spec:
        imagePullSecrets:
        - name: dockerhub-secret
        containers:
        - name: decoding-sdk-worker
            image: minghan2000/decoding-sdk:2.0
            envFrom:
            - configMapRef:
```

```
                name: decoding-sdk-config
            volumeMounts:
            - name: model-storage
                mountPath: /opt/models/
            command:
            [
                "/home/speechuser/start_worker.sh",
                "-m",
                "decoding-sdk-server",
                "-p",
                "8010",
            ]
        initContainers:
        - name: copy-models
            image: amazonlinux
            command: ["/bin/sh", "-c"]
            args:
            - |
                yum install -y aws-cli
                aws s3 cp s3://mh-decoding-sdk-s3-models/models/ /opt/models/
                    --recursive
            volumeMounts:
            - name: model-storage
                mountPath: /opt/models/
        volumes:
        - name: model-storage
            persistentVolumeClaim:
            claimName: model-storage-claim


```

Listing H.1: Kubernetes Manifest for deploying SpeechLab ASR Application.